

Algorithms: A Complete Series

Parts 1–10

History • Theory • Implementation • Systems • Practice

Contents

1	Algorithms	23
1.0.1	When, Why, How, and What—From Ancient Roots to Modern Computing	23
1.1	What Is an Algorithm?	23
1.2	Why Algorithms Matter	23
1.3	Historical Milestones	24
1.3.1	Ancient Foundations (~300 BCE – 9th Century CE)	24
1.3.2	The Mechanical Era (17th – 19th Century)	24
1.3.3	The Theoretical Revolution (1930s – 1940s)	25
1.3.4	The Birth of Modern Computing (1940s – 1960s)	25
1.3.5	Complexity Theory and the P vs NP Question (1960s – 1970s)	26
1.3.6	The Algorithm Age (1980s – 2000s)	26
1.3.7	Machine Learning and the Modern Era (2000s – Present)	27
1.4	How Algorithms Are Analyzed	27
1.5	Categories of Algorithms	28
1.5.1	By Design Paradigm	28
1.5.2	By Problem Domain	29
1.6	The Theory Beneath It All	30
1.6.1	Computability Theory	30
1.6.2	Complexity Theory	30
1.6.3	Information Theory	31
1.6.4	Algorithm Design Theory	31
1.7	The Significance of Evolution	31
1.8	A Note on What Comes Next	32

2 Algorithms: Categories and Implementations	33
2.0.1 A Deep Dive into Design Patterns, Tradeoffs, and How They Work	33
2.1 How to Read This Document	33
3 Part I: Sorting and Searching	35
3.1 Sorting	35
3.1.1 The Problem	35
3.1.2 Why Sorting Matters	35
3.1.3 Bubble Sort	35
3.1.4 Insertion Sort	36
3.1.5 Merge Sort	36
3.1.6 Quicksort	37
3.1.7 Heapsort	38
3.1.8 Radix Sort	38
3.1.9 Timsort	38
3.1.10 Sorting Algorithm Comparison	39
3.2 Searching	39
3.2.1 Linear Search	39
3.2.2 Binary Search	39
3.2.3 Hash-Based Lookup	40
4 Part II: Graph Algorithms	41
4.1 Graph Traversal	41
4.1.1 Breadth-First Search (BFS)	41
4.1.2 Depth-First Search (DFS)	42
4.2 Shortest Path Algorithms	42
4.2.1 Dijkstra's Algorithm	42
4.2.2 Bellman-Ford Algorithm	43
4.2.3 Floyd-Warshall Algorithm	43
4.2.4 A* (A-Star)	43
4.3 Minimum Spanning Trees	44
4.3.1 Kruskal's Algorithm	44
4.3.2 Prim's Algorithm	44
4.4 Topological Sort	44
4.5 Union-Find (Disjoint Set Union)	44

5	Part III: Dynamic Programming	47
5.1	Two Approaches	47
5.2	Classic DP Problems	47
5.2.1	Fibonacci Numbers	47
5.2.2	Longest Common Subsequence (LCS)	48
5.2.3	Edit Distance (Levenshtein Distance)	48
5.2.4	0/1 Knapsack Problem	48
5.2.5	Coin Change	49
5.2.6	Matrix Chain Multiplication	49
5.2.7	Interval DP and Tree DP	49
6	Part IV: Greedy Algorithms	51
6.1	When Greedy Works	51
6.2	Classic Greedy Algorithms	51
6.2.1	Activity Selection (Interval Scheduling)	51
6.2.2	Huffman Coding	52
6.2.3	Dijkstra's Algorithm	52
6.2.4	Fractional Knapsack	52
6.2.5	Minimum Spanning Trees (Prim's and Kruskal's)	52
7	Part V: Divide and Conquer	53
7.1	Master Theorem	53
7.2	Key Divide and Conquer Algorithms	53
7.2.1	Binary Search	53
7.2.2	Merge Sort and Quicksort	54
7.2.3	Karatsuba Multiplication	54
7.2.4	Fast Fourier Transform (FFT)	54
7.2.5	Closest Pair of Points	54
8	Part VI: Backtracking	55
8.1	The Backtracking Template	55
8.2	Classic Backtracking Problems	55
8.2.1	N-Queens	55
8.2.2	Sudoku Solver	56
8.2.3	Permutations and Combinations	56
8.2.4	Constraint Satisfaction Problems (CSPs)	56

9	Part VII: String Algorithms	57
9.1	Pattern Matching	57
9.1.1	Naive Algorithm	57
9.1.2	Knuth-Morris-Pratt (KMP)	57
9.1.3	Boyer-Moore	57
9.1.4	Rabin-Karp	58
9.2	String Data Structures	58
9.2.1	Suffix Array	58
9.2.2	Trie (Prefix Tree)	58
9.3	Edit Distance and Sequence Alignment	58
10	Part VIII: Randomized Algorithms	59
10.1	Types	59
10.2	Key Randomized Algorithms	60
10.2.1	Randomized Quicksort	60
10.2.2	Miller-Rabin Primality Test	60
10.2.3	Hash Functions and Universal Hashing	60
10.2.4	Randomized Min-Cut (Karger's Algorithm)	60
11	Part IX: Computational Geometry	61
11.1	Key Problems and Algorithms	61
11.1.1	Convex Hull	61
11.1.2	Line Intersection (Sweep Line)	61
11.1.3	Voronoi Diagram and Delaunay Triangulation	62
12	Part X: Machine Learning Algorithms	63
12.1	Supervised Learning	63
12.1.1	Linear Regression	63
12.1.2	Logistic Regression	63
12.1.3	Decision Trees	63
12.1.4	Random Forests	64
12.1.5	Support Vector Machines (SVMs)	64
12.1.6	Neural Networks and Gradient Descent	64
12.2	Unsupervised Learning	64
12.2.1	K-Means Clustering	64
12.2.2	Principal Component Analysis (PCA)	64
12.2.3	Autoencoders and Generative Models	65

12.3	Reinforcement Learning	65
13	Part XI: Cryptographic Algorithms	67
13.1	Symmetric Encryption	67
13.2	Public Key Cryptography	67
13.2.1	RSA	67
13.2.2	Elliptic Curve Cryptography (ECC)	68
13.2.3	Diffie-Hellman Key Exchange	68
13.3	Hash Functions	68
14	Design Patterns Summary	69
14.1	What Makes an Algorithm "Good"?	69
14.2	What Comes Next	70
15	Algorithms: Data Structures	71
15.0.1	The Containers That Make Algorithms Work	71
15.1	Why Data Structures Are Half the Story	71
16	Part I: Primitives and Linear Structures	73
16.1	Arrays	73
16.2	Linked Lists	73
16.3	Stacks	74
16.4	Queues	75
16.4.1	Deque (Double-Ended Queue)	75
16.5	Circular Buffers	75
17	Part II: Trees	77
17.1	Binary Trees: The Foundation	77
17.2	Binary Search Trees (BST)	77
17.3	AVL Trees	78
17.4	Red-Black Trees	78
17.5	B-Trees and B+ Trees	79
17.6	Heaps	79
17.7	Tries (Prefix Trees)	80
17.8	Segment Trees	81
17.9	Fenwick Trees (Binary Indexed Trees)	82
17.10	Suffix Trees and Suffix Arrays	82

17.11 Union-Find (Disjoint Set Union)	83
18 Part III: Hash-Based Structures	85
18.1 Hash Tables	85
18.2 Hash Sets	86
18.3 Bloom Filters	86
18.4 Count-Min Sketch	86
18.5 Consistent Hashing	87
19 Part IV: Graphs as Data Structures	89
19.1 Adjacency Matrix	89
19.2 Adjacency List	89
19.3 Special Graph Structures	90
20 Part V: Advanced and Specialized Structures	91
20.1 Skip Lists	91
20.2 Disjoint Sets	91
20.3 Sparse Tables	92
20.4 Interval Trees and Segment Trees (for Geometry)	92
20.5 K-D Trees (k-dimensional trees)	92
20.6 R-Trees	92
20.7 Van Emde Boas Trees	93
20.8 Fibonacci Heaps	93
20.9 Persistent Data Structures	93
20.10 Lock-Free Data Structures	94
21 Part VI: The Space-Time Tradeoff Landscape	95
22 Part VII: Choosing the Right Data Structure	97
23 Part VIII: Data Structures in the Wild	99
23.1 What Comes Next	100
24 Algorithms: Complexity Theory	101
24.0.1 Why Hard Problems Are Hard—and What "Hard" Really Means	101
24.1 The Central Question	101

25 Part I: The Framework—What Are We Measuring?	103
25.1 Computational Models	103
25.2 Input Size	104
25.3 Asymptotic Notation—A Precise Review	104
26 Part II: Time Complexity Classes	105
26.1 P—Polynomial Time	105
26.2 NP—Nondeterministic Polynomial Time	106
26.3 The P vs NP Question	107
26.4 NP-Hard and NP-Complete	108
26.4.1 Cook-Levin Theorem (1971)	108
26.4.2 Karp’s 21 NP-Complete Problems (1972)	108
26.5 Reductions—The Language of Complexity	109
27 Part III: Beyond NP—The Complexity Zoo	111
27.1 co-NP	111
27.2 PSPACE	111
27.3 EXPTIME	112
27.4 BPP—Bounded-Error Probabilistic Polynomial Time	112
27.5 RP and ZPP	112
27.6 #P—Counting Complexity	113
27.7 The Polynomial Hierarchy (PH)	113
27.8 NC and P-completeness	114
27.9 EXPSPACE, 2-EXPTIME, and Beyond	114
28 Part IV: Lower Bounds—Proving Problems Are Hard	115
28.1 Comparison-Based Lower Bounds	115
28.2 Information-Theoretic Lower Bounds	116
28.3 Adversary Arguments	116
28.4 NP-Hardness Reductions as Lower Bounds	116
28.5 Circuit Complexity	117
29 Part V: Dealing with Hardness—When You Can’t Avoid NP	119
29.1 1. Approximation Algorithms	119
29.2 2. Parameterized Complexity (FPT)	120
29.3 3. Exact Exponential Algorithms	120
29.4 4. Heuristics and SAT Solvers	121

30 Part VI: Special Complexity Results Worth Knowing	123
30.1 The PCP Theorem	123
30.2 Unique Games Conjecture (UGC)	123
30.3 Ladner's Theorem (1975)	124
30.4 Savitch's Theorem	124
30.5 Time and Space Hierarchy Theorems	125
30.6 Immerman-Szelepcsényi Theorem (1988)	125
31 Part VII: Quantum Complexity	127
31.1 BQP—Bounded-Error Quantum Polynomial Time	127
31.2 Shor's Algorithm (1994)	127
31.3 Grover's Algorithm (1996)	128
31.4 Post-Quantum Cryptography	128
32 Part VIII: The Complexity Landscape—A Map	129
33 Part IX: Why This Matters Beyond Theory	131
33.1 The Deepest Insight	132
33.2 What Comes Next	132
34 Algorithms: Advanced Design Techniques	135
34.0.1 Network Flow, Linear Programming, Online Algorithms, and the Art of Approximation	135
34.1 What "Advanced" Means Here	135
35 Part I: Network Flow	137
35.1 The Maximum Flow Problem	137
35.2 The Max-Flow Min-Cut Theorem	138
35.3 Ford-Fulkerson and Augmenting Paths	138
35.4 Edmonds-Karp Algorithm	139
35.5 Push-Relabel Algorithm	139
35.6 Applications of Max Flow	139
35.6.1 Bipartite Matching	139
35.6.2 Minimum Cut in Graphs	140
35.6.3 Project Selection / Closure Problem	140
35.6.4 Circulation with Demands	140
35.6.5 Multi-Commodity Flow	140

35.7	Matching Beyond Bipartite Graphs	140
36	Part II: Linear Programming	143
36.1	What Is Linear Programming?	143
36.2	The Simplex Method	143
36.3	The Ellipsoid Method and Interior Point Methods	144
36.4	LP Duality	144
36.5	Integer Linear Programming (ILP)	145
36.6	LP in Algorithm Design	145
36.7	The Ellipsoid Method in Combinatorics	146
37	Part III: Online Algorithms	147
37.1	The Online Setting	147
37.2	Competitive Analysis	147
37.3	The Ski Rental Problem	148
37.4	Paging and Caching	148
37.5	The k-Server Problem	149
37.6	Secretary Problem	149
37.7	Online Bipartite Matching	150
37.8	Load Balancing Online	150
37.9	Adversary Models	150
38	Part IV: Approximation Algorithm Design	153
38.1	The Goal	153
38.2	Greedy Approximations	153
38.2.1	Set Cover	153
38.2.2	Load Balancing (again)	154
38.3	LP Relaxation and Rounding	154
38.3.1	Weighted Vertex Cover	154
38.3.2	Randomized Rounding for MAX-SAT	154
38.4	Primal-Dual Method for Approximation	155
38.4.1	Primal-Dual for Shortest Path (Dijkstra!)	155
38.4.2	Primal-Dual for Steiner Tree	155
38.4.3	Primal-Dual for Facility Location	156
38.5	PTAS and FPTAS	156
38.6	Semidefinite Programming (SDP) Relaxations	157
38.6.1	Goemans-Williamson MAX-CUT (1995)	157

38.7	Hardness of Approximation	157
38.7.1	The PCP Theorem’s Role	157
38.7.2	Key Inapproximability Results	158
38.8	The Approximation Ratio Landscape	158
39	Part V: Randomized Algorithm Design—Deeper	161
39.1	The Probabilistic Method	161
39.2	Hashing and Randomness	161
39.3	Randomized Data Structures	162
39.4	Monte Carlo Integration and Sampling	162
40	Part VI: Streaming Algorithms	163
40.1	The Streaming Model	163
40.2	Frequency Estimation	163
40.3	Distinct Element Counting	163
40.4	Finding the Median	164
41	Part VII: Putting It Together—Advanced Design in Practice	165
41.1	What Comes Next	166
42	Algorithms: Analysis in Depth	167
42.0.1	Amortized, Probabilistic, Smoothed, and Competi- tive Analysis—Proving Algorithms Are Fast	167
42.1	Why Analysis Deserves Its Own Document	167
43	Part I: Amortized Analysis	169
43.1	The Core Idea	169
43.2	Technique 1: Aggregate Analysis	169
43.2.1	Dynamic Array (ArrayList) Appends	169
43.2.2	Binary Counter Increments	170
43.3	Technique 2: The Accounting Method	170
43.3.1	Dynamic Array with Accounting	171
43.4	Technique 3: The Potential Method	171
43.4.1	Dynamic Array with Potential	172
43.4.2	Splay Trees	172
43.5	Amortized Analysis of Union-Find	172
43.6	Lazy Deletion and Amortized Cleanup	173

44 Part II: Recurrence Analysis	175
44.1 Solving Recurrences	175
44.2 Substitution Method	175
44.3 Recursion Tree Method	176
44.4 The Master Theorem	176
44.5 Akra-Bazzi Method	177
44.6 Linear Recurrences and Fibonacci	177
45 Part III: Probabilistic Analysis	179
45.1 Average-Case vs. Worst-Case	179
45.2 Quicksort Average-Case Analysis	179
45.3 Harmonic Numbers and the Coupon Collector	180
45.4 Birthday Paradox and Hash Collisions	180
45.5 High-Probability Bounds	181
45.5.1 Markov's Inequality	181
45.5.2 Chebyshev's Inequality	181
45.5.3 Chernoff Bounds	181
45.6 The Union Bound	182
46 Part IV: Smoothed Analysis	183
46.1 The Problem with Worst-Case and Average-Case	183
46.2 Smoothed Complexity	183
46.3 Smoothed Analysis of Other Algorithms	184
47 Part V: Competitive Analysis—A Deeper Look	185
47.1 Potential Functions for Online Algorithms	185
47.2 Working-Set Model and Beyond Worst-Case	185
47.3 Algorithms with Predictions (Learning-Augmented Algorithms)	186
47.4 Yao's Minimax Principle	186
48 Part VI: Information-Theoretic Lower Bounds	189
48.1 Decision Tree Lower Bounds	189
48.2 Communication Complexity	190
48.3 Entropy Lower Bounds	190
49 Part VII: Fine-Grained Complexity	191

49.1	Beyond NP-Hardness	191
49.2	The Strong Exponential Time Hypothesis (SETH)	191
49.3	The 3SUM Conjecture	192
49.4	Matrix Multiplication and ω	192
49.5	APSP Conjecture	192
50	Part VIII: The Analysis Toolkit—Summary	195
50.1	The Relationship Between Analysis Techniques	196
50.2	What Comes Next	196
51	Algorithms: Systems and Engineering	197
51.0.1	Where Theory Meets Hardware, Scale, and the Messy Real World	197
51.1	The Gap Between Theory and Practice	197
52	Part I: The Memory Hierarchy	199
52.1	Why Memory Hierarchy Exists	199
52.2	Locality: The Key to Cache Performance	200
52.3	Cache-Aware Algorithm Design	200
52.3.1	Matrix Multiplication—Naive vs. Cache-Friendly	200
52.4	Cache-Oblivious Algorithms	201
52.4.1	Cache-Oblivious Matrix Multiplication	201
52.4.2	Cache-Oblivious Sorting: Funnelsort	202
52.4.3	Cache-Oblivious Search Trees	202
52.5	External Memory Algorithms	202
52.5.1	External Sorting	203
52.5.2	B-Trees as Optimal External Search Structures	203
53	Part II: Parallel Algorithms	205
53.1	Why Parallelism Now	205
53.2	Models of Parallel Computation	205
53.3	Parallel Sorting	206
53.3.1	Parallel Merge Sort	206
53.3.2	Bitonic Sort	206
53.3.3	Sample Sort (Parallel Quicksort)	206
53.4	Parallel Graph Algorithms	207
53.4.1	Parallel BFS	207

53.4.2	Parallel Minimum Spanning Tree	207
53.4.3	Parallel Prefix Sum (Scan)	207
53.4.4	List Ranking and Euler Tour	208
53.5	GPU Computing	208
53.6	Concurrent Data Structures	209
54	Part III: Distributed Algorithms	211
54.1	The Distributed Setting	211
54.2	The CAP Theorem	211
54.3	Consensus—The Core Problem	212
54.3.1	FLP Impossibility (Fischer, Lynch, Paterson, 1985)	212
54.3.2	Paxos (Lamport, 1989/1998)	213
54.3.3	Raft (Ongaro and Ousterhout, 2014)	213
54.3.4	Byzantine Fault Tolerance (BFT)	214
54.4	Distributed Data Structures	214
54.4.1	Consistent Hashing	214
54.4.2	Distributed Hash Tables (DHTs)	215
54.4.3	CRDTs—Conflict-Free Replicated Data Types	215
54.5	MapReduce and Large-Scale Data Processing	215
54.6	Distributed Databases and Transactions	216
54.6.1	ACID vs. BASE	216
54.6.2	Two-Phase Commit (2PC)	216
54.6.3	Three-Phase Commit (3PC)	217
54.6.4	Distributed Transactions with Paxos / Raft	217
54.6.5	Google Spanner: TrueTime	217
54.7	The Logical Clock Perspective	217
55	Part IV: Database Internals	219
55.1	The Storage Layer	219
55.1.1	Row vs. Column Storage	219
55.1.2	LSM Trees (Log-Structured Merge Trees)	219
55.1.3	Write-Ahead Logging (WAL)	220
55.2	Query Execution	220
55.2.1	The Query Planner	220
55.2.2	Join Algorithms	221
55.2.3	Query Compilation	221
55.3	Indexes Beyond B+ Trees	221

56 Part V: Compiler Algorithms	223
56.1 Why Compilers Matter Here	223
56.2 Lexing and Parsing	223
56.3 Intermediate Representation and SSA	224
56.4 Key Compiler Optimizations	224
56.5 Dataflow Analysis	225
57 Part VI: Making Algorithms Fast in Practice	227
57.1 The Optimization Hierarchy	227
57.2 Benchmarking and Profiling	228
57.3 SIMD and Bit Manipulation	228
57.4 Memory Allocators	229
57.5 String Processing Optimizations	229
57.6 Real-World Case Studies	230
58 Part VII: The Systems Perspective—Pulling It Together	231
58.1 The Stack of Abstractions	231
58.2 The Lessons	232
58.3 What Comes Next	232
59 Algorithms: Domain Deep Dives	235
59.0.1 How Algorithmic Thinking Transforms Five Fields	235
59.1 Why Domains Matter	235
60 Part I: Bioinformatics	237
60.1 The Data of Life	237
60.2 Sequence Alignment	237
60.2.1 Needleman-Wunsch (Global Alignment)	237
60.2.2 Smith-Waterman (Local Alignment)	238
60.2.3 Affine Gap Penalties	238
60.2.4 BLAST—Practical Heuristic Alignment	238
60.3 Genome Assembly	239
60.3.1 de Bruijn Graph Assembly	239
60.3.2 Overlap-Layout-Consensus (OLC)	240
60.4 Read Mapping	240
60.4.1 Suffix Array + BWT Alignment (BWA, Bowtie2) . .	241
60.5 Variant Calling	241

60.6	Protein Structure Prediction—AlphaFold	242
60.7	Phylogenetics	242
61	Part II: Cryptography	245
61.1	The Algorithmic Foundation of Trust	245
61.2	Symmetric Cryptography	245
61.2.1	AES—The Advanced Encryption Standard	245
61.2.2	Stream Ciphers and ChaCha20	246
61.3	Asymmetric (Public Key) Cryptography	246
61.3.1	RSA—Number Theory as Security	246
61.3.2	Elliptic Curve Cryptography (ECC)	247
61.3.3	Diffie-Hellman Key Exchange	247
61.4	Cryptographic Hash Functions	248
61.4.1	SHA-256 and SHA-3	248
61.4.2	HMAC—Hash-Based Message Authentication	248
61.5	Zero-Knowledge Proofs	249
61.6	Post-Quantum Cryptography	250
61.7	Side-Channel Attacks—Algorithms Must Be Constant-Time	250
62	Part III: Computer Graphics	253
62.1	The Rendering Pipeline	253
62.2	Rasterization—The Real-Time Pipeline	253
62.2.1	Rasterization—The Triangle Scan-Line Algorithm	254
62.2.2	Texture Mapping and Mipmaps	254
62.3	Ray Tracing—Physically Accurate Rendering	255
62.3.1	Bounding Volume Hierarchies (BVH)	255
62.3.2	Monte Carlo Path Tracing	255
62.4	Global Illumination Techniques	256
62.4.1	Radiosity	256
62.4.2	Photon Mapping (Jensen, 1996)	256
62.4.3	Spherical Harmonics for Irradiance	257
62.5	Geometry Processing	257
62.5.1	Mesh Simplification	257
62.5.2	Subdivision Surfaces	257
62.5.3	Signed Distance Fields (SDFs)	257
62.6	Physics Simulation	258
62.6.1	Rigid Body Dynamics	258

62.6.2	Fluid Simulation	258
63	Part IV: Information Retrieval and Search Engines	259
63.1	The Scale of Search	259
63.2	Indexing—The Inverted Index	259
63.2.1	Intersection and Union—Boolean Retrieval	260
63.3	Ranking—From Retrieval to Relevance	260
63.3.1	TF-IDF	260
63.3.2	BM25 (Best Match 25)	261
63.3.3	Learning to Rank (LTR)	261
63.3.4	PageRank—The Structural Signal	261
63.4	Approximate Nearest Neighbor Search	262
63.4.1	Locality Sensitive Hashing (LSH)	262
63.4.2	HNSW (Hierarchical Navigable Small World)	262
63.4.3	Product Quantization (PQ)	263
63.5	Document Representation and Neural Retrieval	263
63.5.1	TF-IDF Vectors and Sparse Retrieval	263
63.5.2	Dense Retrieval (Neural Embeddings)	263
63.5.3	Hybrid Retrieval	264
63.6	Query Understanding and Spelling Correction	264
64	Part V: Cross-Domain Themes	265
64.1	Probabilistic Models Appear Everywhere	265
64.2	Graph Algorithms Underlie Everything	265
64.3	Dimensionality is the Universal Enemy	266
64.4	Approximation and Exactness in Tension	266
64.5	What Comes Next	266
65	Algorithms: Machine Learning in Depth	269
65.0.1	Optimization, Architecture, Generalization, and the Theory of Learning	269
65.1	A Different Kind of Algorithm	269
66	Part I: The Learning Problem—Foundations	271
66.1	What We’re Trying to Do	271
66.2	Statistical Learning Theory	271
66.3	The Bias-Variance Tradeoff	272

66.4	VC Dimension and Sample Complexity	272
66.5	PAC Learning	273
67	Part II: Optimisation—How Models Are Trained	275
67.1	The Loss Landscape	275
67.2	Gradient Descent and Its Variants	275
67.2.1	The Learning Rate—The Critical Hyperparameter	276
67.3	Momentum and Acceleration	276
67.4	Adaptive Learning Rate Methods	277
67.4.1	AdaGrad (Duchi et al., 2011)	277
67.4.2	RMSProp (Hinton, 2012)	277
67.4.3	Adam (Kingma and Ba, 2014)	277
67.4.4	Convergence Theory of Adam	278
67.5	Second-Order Methods	278
67.6	Backpropagation—The Engine of Deep Learning	279
67.7	The Loss Landscape of Deep Networks	280
68	Part III: Neural Network Architectures	283
68.1	The Universal Approximation Theorem	283
68.2	Fully Connected Networks	283
68.3	Convolutional Neural Networks (CNNs)	284
68.3.1	Classic CNN Architectures	285
68.4	Recurrent Neural Networks (RNNs)	285
68.4.1	LSTM (Long Short-Term Memory, Hochreiter and Schmidhuber, 1997)	286
68.5	The Transformer Architecture	286
68.5.1	Self-Attention	287
68.5.2	Multi-Head Attention	287
68.5.3	Positional Encoding	288
68.5.4	The Full Transformer Block	288
68.5.5	Encoder, Decoder, Encoder-Decoder	288
68.6	Scaling Laws	289
68.7	Attention Variants—Scaling to Longer Sequences	289
68.8	Vision Transformers and Beyond	290
69	Part IV: Regularisation and Generalisation	293
69.1	The Generalisation Problem	293

69.2	Classical Regularisation	293
69.3	Dropout (Srivastava et al., 2014)	293
69.4	Batch Normalisation (Ioffe and Szegedy, 2015)	294
69.5	Data Augmentation	295
69.6	Transfer Learning and Fine-Tuning	295
69.6.1	Parameter-Efficient Fine-Tuning (PEFT)	296
70	Part V: Self-Supervised and Unsupervised Learning	297
70.1	Self-Supervised Learning	297
70.2	Representation Learning and Embeddings	298
70.3	Generative Models	298
71	Part VI: Reinforcement Learning Algorithms	301
71.1	The RL Framework	301
71.2	Value-Based Methods	301
71.2.1	Deep Q-Networks (DQN, Mnih et al., 2013)	302
71.3	Policy Gradient Methods	303
71.3.1	Actor-Critic Methods	303
71.3.2	SAC (Soft Actor-Critic)	304
71.4	Model-Based RL	304
71.5	RLHF—Reinforcement Learning from Human Feedback	304
71.6	AlphaGo / AlphaZero—Deep RL at the Frontier	305
72	Part VII: Large Language Models—The Current Frontier	307
72.1	The Pre-Training Paradigm	307
72.2	Tokenization	307
72.3	Context Length and Attention Efficiency	308
72.4	Inference—Making Generation Fast	308
72.5	Emergent Capabilities and Chain-of-Thought	309
73	Part VIII: The Theory Behind Practice	311
73.1	Why Does Any of This Work?	311
73.2	What Comes Next	312
74	Algorithms: Problem Solving and Pattern Recognition	313
74.0.1	The Art and Science of Algorithmic Thinking	313
74.1	What This Part Is About	313

75 Part I: The Pattern Taxonomy	315
75.1 Pattern 1: The Optimisation Problem	315
75.2 Pattern 2: The Counting Problem	316
75.3 Pattern 3: The Search Problem	316
75.4 Pattern 4: The Graph Problem	317
75.5 Pattern 5: The String or Sequence Problem	318
75.6 Pattern 6: The Interval or Geometric Problem	319
75.7 Pattern 7: The Resource Allocation Problem	319
75.8 Pattern 8: The Recursive or Fractal Problem	320
75.9 Pattern 9: The Probability or Randomness Problem	320
75.10 Pattern 10: The Data Structure Selection Problem	321
76 Part II: The Problem-Solving Framework	323
76.1 Step 1: Understand the Problem—Really	323
76.2 Step 2: Classify the Problem	324
76.3 Step 3: Reduce to a Known Problem	324
76.4 Step 4: Design the Algorithm	325
76.5 Step 5: Prove Correctness	326
76.6 Step 6: Analyse Complexity	326
76.7 Step 7: Implement and Test	327
77 Part III: Canonical Reductions and Transformations	329
77.1 The Reduction Library	329
77.1.1 Binary Search on the Answer	329
77.1.2 Reduce to Shortest Path	330
77.1.3 Reduce to Max Flow	330
77.1.4 Reduce to Dynamic Programming	331
77.1.5 Reduce to Minimum Spanning Tree	332
77.1.6 Reduce to Sorting	332
77.1.7 Reduce to Convex Hull	333
77.1.8 Reduce to String Matching	334
77.1.9 Reduce to Matrix Operations	334
78 Part IV: The Deeper Patterns	335
78.1 Duality—The Same Problem, Two Views	335
78.2 Relaxation—The Art of Letting Go	336
78.3 Symmetry and Invariants	336

78.4	The Power of Preprocessing	337
78.5	The Role of Randomness	337
79	Part V: The Meta-Skills of Algorithmic Thinking	339
79.1	Simplify First, Generalise Later	339
79.2	Know When to Stop	340
79.3	Read the Problem's Constraints	340
79.4	The Problem-Solving Disposition	341
80	Coda: The Unity of Algorithmic Thinking	343
80.1	Series Index	344

Chapter 1

Algorithms

1.0.1 When, Why, How, and What—From Ancient Roots to Modern Computing

1.1 What Is an Algorithm?

An algorithm is a finite, ordered sequence of well-defined instructions designed to solve a problem or accomplish a task. The word itself is surprisingly old—it derives from the name of the 9th-century Persian mathematician **Muhammad ibn Musa al-Khwarizmi**, whose work on arithmetic and algebra introduced systematic problem-solving methods to the Western world. His name, Latinized, became *algorismus*, and eventually *algorithm*.

At its core, an algorithm is not about computers. It is about *process*. A recipe is an algorithm. Long division is an algorithm. The rules of chess are an algorithm. What computers did was make the execution of algorithms extraordinarily fast—and in doing so, transformed algorithms from a mathematical curiosity into the invisible infrastructure of the modern world.

1.2 Why Algorithms Matter

The significance of algorithms lies in their generality. A well-designed algorithm can be applied to any instance of a problem—not just one specific case. This distinction between *solving a problem* and *describing how to solve any problem of that type* is foundational.

Algorithms matter because they determine:

- **Feasibility**—whether a problem can be solved at all
- **Efficiency**—how much time and memory a solution requires
- **Scalability**—whether a solution still works as the input grows large
- **Correctness**—whether the result is always provably right

Without algorithmic thinking, modern computing would be brute force at best. With it, we can search billions of web pages in milliseconds, encrypt global communications, train neural networks, and route packages across continents.

1.3 Historical Milestones

1.3.1 Ancient Foundations (~300 BCE – 9th Century CE)

The earliest known algorithms predate computers by millennia. **Euclid's algorithm** for finding the greatest common divisor (~300 BCE) is one of the oldest algorithms still in common use today. It is elegant, efficient, and provably correct—a perfect example of algorithmic thinking long before the word existed.

The **Sieve of Eratosthenes** (~240 BCE) offered a method for finding all prime numbers up to a given limit—a genuine algorithm with clear steps and termination conditions.

Al-Khwarizmi's 9th-century treatises formalized arithmetic procedures and introduced what we would now call procedural thinking to mathematics, laying the cultural groundwork for algorithmic science.

1.3.2 The Mechanical Era (17th – 19th Century)

Gottfried Wilhelm Leibniz and **Blaise Pascal** built mechanical calculators in the 17th century, demonstrating that arithmetic procedures could be mechanized. More importantly, Leibniz proposed that all reasoning might be reducible to calculation—a philosophical idea that would echo through the centuries.

Charles Babbage designed his Difference Engine (1822) and later the Analytical Engine (1837), a general-purpose mechanical computer that was never fully built in his lifetime. His collaborator **Ada Lovelace** wrote what many consider the first algorithm intended for a machine—a method for computing Bernoulli numbers—making her arguably the first programmer.

George Boole formalized logic as algebra (1847), giving us Boolean logic: the 0s and 1s, ANDs and ORs that underpin every computer today.

1.3.3 The Theoretical Revolution (1930s – 1940s)

This era produced the conceptual breakthroughs that defined what computation *is*.

Kurt Gödel (1931) proved his incompleteness theorems—showing that some mathematical truths can never be proven within a formal system. This hinted at limits on what algorithms could achieve.

Alan Turing (1936) introduced the **Turing Machine**—an abstract model of computation that described what it means for something to be computable. He also defined the **halting problem**, proving that no general algorithm can determine whether an arbitrary program will halt or run forever. This was the first formal proof of algorithmic limitation.

Alonzo Church independently developed **lambda calculus** (1936), an equivalent model of computation that became the theoretical foundation for functional programming languages.

Claude Shannon (1948) established **information theory**, quantifying information, entropy, and the limits of data compression and transmission—giving algorithms a mathematical framework for working with data.

1.3.4 The Birth of Modern Computing (1940s – 1960s)

With working computers came the need for practical algorithms. Key milestones:

John von Neumann (1945) described the stored-program architecture that most computers still follow today, and contributed early sorting algorithms including **merge sort**.

Edsger Dijkstra (1956) published his famous shortest-path algorithm—**Dijkstra’s algorithm**—which remains one of the most widely used graph algorithms in existence, powering GPS navigation and network routing.

Donald Knuth began writing *The Art of Computer Programming* in 1962, the most comprehensive treatment of algorithmic analysis ever attempted. He introduced the concept of **algorithm analysis** as a rigorous discipline.

The **Fast Fourier Transform (FFT)** was rediscovered and popularized by Cooley and Tukey in 1965, reducing the complexity of Fourier analysis from $O(n^2)$ to $O(n \log n)$ —a breakthrough that enabled digital signal processing, audio compression, and communications.

1.3.5 Complexity Theory and the P vs NP Question (1960s – 1970s)

Perhaps the deepest theoretical development in algorithms came when researchers began asking not just *how* to solve problems but *how hard* problems fundamentally are.

Stephen Cook (1971) introduced **NP-completeness**, proving that certain problems (like the Boolean satisfiability problem, SAT) are in a class where solutions can be verified quickly but no efficient algorithm for finding solutions is known. **Richard Karp** (1972) showed 21 fundamental problems are all NP-complete—meaning if you solve one efficiently, you solve them all.

The **P vs NP** question—whether every problem whose solution can be quickly verified can also be quickly solved—remains the greatest unsolved problem in computer science, and one of the Millennium Prize Problems.

Donald Knuth formalized **Big-O notation** as the standard way to express algorithmic complexity, giving the field a common language.

1.3.6 The Algorithm Age (1980s – 2000s)

Algorithms became the commercial backbone of the digital economy:

RSA encryption (1977, published 1978) gave us public-key cryptography—the mathematical foundation for secure internet communication, banking,

and e-commerce. It relies on the algorithmic hardness of factoring large numbers.

The PageRank algorithm (Larry Page and Sergey Brin, 1998) ranked web pages by their link structure, founding Google and transforming how humanity accesses information.

Quicksort, invented by Tony Hoare in 1959-1960, became the dominant practical sorting algorithm and is still widely used today due to its cache performance.

Dynamic programming (Richard Bellman, 1950s) emerged as a paradigm for solving complex optimization problems by breaking them into overlapping subproblems—used in everything from DNA sequence alignment to financial modeling.

1.3.7 Machine Learning and the Modern Era (2000s – Present)

The most significant algorithmic shift of the 21st century has been the rise of **learned algorithms**—programs that improve through data rather than being explicitly programmed.

Gradient descent (an old optimization method) became the workhorse of neural network training. The **backpropagation** algorithm, formalized in the 1980s, allowed deep neural networks to learn from error signals.

The Transformer architecture (Vaswani et al., 2017) introduced the **attention mechanism**, enabling models to process sequences in parallel and learn contextual relationships at scale. It underpins GPT, BERT, and most modern large language models.

AlphaGo (DeepMind, 2016) combined Monte Carlo Tree Search with deep reinforcement learning to defeat world champions at Go—a game long considered beyond the reach of algorithms due to its combinatorial complexity.

1.4 How Algorithms Are Analyzed

Analyzing an algorithm means asking: how does its resource consumption scale as input size grows?

Time complexity measures how the number of operations grows with input size n . **Space complexity** measures memory usage. We express these using **asymptotic notation**:

- **$O(1)$** —constant time (e.g., array lookup)
- **$O(\log n)$** —logarithmic (e.g., binary search)
- **$O(n)$** —linear (e.g., linear scan)
- **$O(n \log n)$** —linearithmic (e.g., merge sort)
- **$O(n^2)$** —quadratic (e.g., bubble sort)
- **$O(2^n)$** —exponential (e.g., brute-force subset enumeration)

The gap between these classes is enormous at scale. An $O(n^2)$ algorithm processing one million items does one trillion operations. An $O(n \log n)$ algorithm does about twenty million. This difference is the difference between a calculation that takes seconds and one that takes decades.

Best, average, and worst case analysis gives a fuller picture—an algorithm might perform well on average but degrade badly on adversarial inputs (the classic case being quicksort’s $O(n^2)$ worst case on already-sorted data).

1.5 Categories of Algorithms

Algorithms are grouped by the *design paradigm* or *problem domain* they address.

1.5.1 By Design Paradigm

Brute Force—try all possibilities. Correct but often wildly inefficient. Useful as a baseline or when input size is guaranteed small.

Divide and Conquer—split the problem into smaller subproblems, solve them recursively, combine the results. Examples: merge sort, quicksort, binary search, Karatsuba multiplication, FFT.

Dynamic Programming—solve overlapping subproblems once and store results. Used when a problem has optimal substructure. Examples: Fi-

bonacci (memoized), longest common subsequence, knapsack problem, Bellman-Ford shortest paths.

Greedy Algorithms—make the locally optimal choice at each step, hoping to reach a global optimum. Efficient but not always correct. Examples: Dijkstra’s algorithm, Kruskal’s and Prim’s spanning tree algorithms, Huffman coding.

Backtracking—explore all possibilities by building candidates incrementally and abandoning paths that violate constraints. Examples: N-Queens, Sudoku solvers, constraint satisfaction problems.

Randomized Algorithms—incorporate randomness to achieve good average-case performance or probabilistic guarantees. Examples: randomized quicksort, Monte Carlo methods, Miller-Rabin primality testing.

Approximation Algorithms—for NP-hard problems where exact solutions are too expensive, find solutions guaranteed to be within some factor of optimal. Examples: approximation algorithms for TSP, set cover, vertex cover.

1.5.2 By Problem Domain

Sorting and Searching—the oldest and most studied category. Sorting algorithms include bubble sort, insertion sort, merge sort, quicksort, heap-sort, radix sort, and timsort. Searching includes linear search, binary search, and hash-based lookup.

Graph Algorithms—algorithms operating on networks of nodes and edges. Includes breadth-first search (BFS), depth-first search (DFS), shortest path (Dijkstra, Bellman-Ford, Floyd-Warshall), minimum spanning trees (Kruskal, Prim), topological sorting, and network flow (Ford-Fulkerson).

String Algorithms—pattern matching, string searching, and text processing. Includes Knuth-Morris-Pratt (KMP), Boyer-Moore, Rabin-Karp, suffix arrays, and dynamic programming approaches for edit distance and sequence alignment.

Numerical and Mathematical Algorithms—root finding (Newton’s method), numerical integration, matrix operations, linear algebra (Gaussian

elimination, LU decomposition), FFT, primality testing, and cryptographic algorithms.

Compression Algorithms—lossless (Huffman coding, LZ77/LZ78, DEFLATE, LZW) and lossy (JPEG’s DCT, MP3’s psychoacoustic model). These balance algorithmic cleverness with information-theoretic limits.

Cryptographic Algorithms—algorithms designed to be computationally hard to reverse. Symmetric key (AES, DES), public key (RSA, elliptic curve cryptography), hash functions (SHA-256, MD5), and zero-knowledge proofs.

Machine Learning Algorithms—a major modern category: supervised learning (linear regression, decision trees, SVMs, neural networks), unsupervised learning (k-means, PCA, autoencoders), and reinforcement learning (Q-learning, policy gradients).

Distributed and Parallel Algorithms—algorithms designed for multiple processors or machines. Includes MapReduce, consensus algorithms (Paxos, Raft), and parallel sorting and graph algorithms.

1.6 The Theory Beneath It All

1.6.1 Computability Theory

Asks what can be computed at all. The Turing machine model defines the limits: the halting problem is undecidable, meaning no algorithm can solve it in general. Some problems are simply beyond what any algorithm can do.

1.6.2 Complexity Theory

Asks what can be computed *efficiently*. The complexity zoo includes P (polynomial time), NP (nondeterministic polynomial time), NP-complete, NP-hard, PSPACE, EXPTIME, and many others. The relationships between these classes describe the deep structure of computational difficulty.

1.6.3 Information Theory

Asks what can be compressed or transmitted. Shannon's entropy provides a lower bound on lossless compression, and channel capacity theorems describe the limits of reliable communication.

1.6.4 Algorithm Design Theory

Asks how to construct good algorithms. Covers recurrence relations, adversarial analysis, amortized analysis, competitive analysis (for online algorithms), and the theory of approximation.

1.7 The Significance of Evolution

The evolution of algorithms reflects humanity's growing understanding of what computation is, what it can do, and what its limits are. Each era has expanded this understanding:

The ancients showed that systematic reasoning could solve problems reliably. The 19th century showed that reasoning could be mechanized. The 1930s showed that computation has fundamental limits—and defined those limits precisely. The mid-20th century showed that algorithms could be analyzed, compared, and optimized as rigorous mathematical objects. The late 20th century built industries on this foundation. And the 21st century has begun to blur the line between algorithms that are written and algorithms that are learned.

We are still in the middle of this story. Quantum computing promises to reshape complexity classes. Algorithmic fairness and bias have become ethical and political questions. The algorithms that recommend, rank, price, and decide have become as consequential as any law.

Understanding algorithms is no longer just a technical skill. It is a form of literacy.

1.8 A Note on What Comes Next

This document has focused on the landscape—the when, why, and what of algorithms in broad strokes. From here, we can go deeper in many directions: specific implementations and their tradeoffs, the mathematics of complexity proofs, the design patterns of dynamic programming, the mechanics of machine learning, or the engineering of distributed systems. The foundation is here. The details are where it gets interesting.

Next: Specific Algorithm Categories and Implementations

Chapter 2

Algorithms: Categories and Implementations

2.0.1 A Deep Dive into Design Patterns, Tradeoffs, and How They Work

This is Part 2 of the Algorithms series. Part 1 covers the history, theory, and landscape.

2.1 How to Read This Document

For each major category, we cover:

- **What problem it solves** and why it matters
- **The core design idea**—the insight that makes the approach work
- **Key algorithms** with pseudocode or plain-language descriptions of the logic
- **Complexity**—time and space, best/average/worst where relevant
- **Tradeoffs**—what each algorithm is good at, where it breaks down
- **Real-world uses**—where you actually encounter these today

Chapter 3

Part I: Sorting and Searching

Sorting and searching are the foundation. They appear as sub-routines inside almost every other algorithm. Understanding them deeply—not just their names but why they work and when to use which—is the prerequisite for everything else.

3.1 Sorting

3.1.1 The Problem

Given a collection of items, arrange them in a defined order (numeric, lexicographic, custom comparator). Sounds simple. The nuance is enormous.

3.1.2 Why Sorting Matters

Sorted data enables binary search ($O(\log n)$ vs $O(n)$). It simplifies deduplication, merging, and set operations. Many algorithms assume sorted input as a precondition. Sorting is so fundamental that most languages have a built-in sort—and the choice of which algorithm to use under the hood involves careful engineering tradeoffs.

3.1.3 Bubble Sort

Idea: Repeatedly scan the array, swapping adjacent elements that are out of order. Each pass "bubbles" the largest remaining element to its correct position.

```
for i from 0 to n-1:
    for j from 0 to n-i-2:
        if arr[j] > arr[j+1]:
            swap(arr[j], arr[j+1])
```

Complexity: $O(n^2)$ time in all practical cases. $O(1)$ space. **Verdict:** Almost never used in practice. Its only virtues are simplicity and the fact that it can detect an already-sorted array in $O(n)$. Good for teaching.

3.1.4 Insertion Sort

Idea: Build the sorted array one element at a time. Take the next unsorted element and insert it into its correct position among the already-sorted elements—like sorting a hand of playing cards.

```
for i from 1 to n-1:
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j -= 1
    arr[j+1] = key
```

Complexity: $O(n^2)$ average/worst. $O(n)$ best case (already sorted). $O(1)$ space. **Verdict:** Excellent for small arrays ($< \sim 20$ elements) and nearly-sorted data. Low overhead, cache-friendly, stable. Used as the base case in many hybrid sorts.

3.1.5 Merge Sort

Idea: Divide the array in half, recursively sort each half, then merge the two sorted halves. The merge step is the key: two sorted arrays can be merged in $O(n)$ time by walking through both simultaneously.

```
mergeSort(arr):
    if len(arr) <= 1: return arr
```

```

mid = len(arr) // 2
left = mergeSort(arr[:mid])
right = mergeSort(arr[mid:])
return merge(left, right)

merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i++
        else:
            result.append(right[j]); j++
    return result + left[i:] + right[j:]

```

Complexity: $O(n \log n)$ in all cases. $O(n)$ space (the merge step needs auxiliary storage). **Verdict:** Reliable, predictable, stable. The guaranteed $O(n \log n)$ worst case makes it preferable to quicksort in situations where worst-case behavior matters. Default choice for linked lists (where random access is expensive). Used in Python's Timsort.

3.1.6 Quicksort

Idea: Choose a *pivot* element. Partition the array so everything less than the pivot is on the left, everything greater is on the right. Recursively sort both halves. No merge step needed—the partitioning does the work.

```

quickSort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        quickSort(arr, low, pivot_index - 1)
        quickSort(arr, pivot_index + 1, high)

partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j from low to high-1:
        if arr[j] <= pivot:
            i++

```

```
        swap(arr[i], arr[j])
    swap(arr[i+1], arr[high])
    return i + 1
```

Complexity: $O(n \log n)$ average. $O(n^2)$ worst case (when pivot is always the min or max—e.g., sorted input with naive pivot choice). $O(\log n)$ stack space. **Verdict:** In practice, often the fastest sort due to excellent cache locality and low constant factors. The worst case is avoided with randomized pivot selection. Unstable. The default sort in many C/C++ standard libraries.

3.1.7 Heapsort

Idea: Build a max-heap from the array, then repeatedly extract the maximum element to build the sorted array.

Complexity: $O(n \log n)$ in all cases. $O(1)$ space (in-place). **Verdict:** Optimal time and space but poor cache performance in practice. Rarely the first choice but important as the basis for priority queues.

3.1.8 Radix Sort

Idea: Sort by individual digits (or characters) from least significant to most significant, using a stable counting sort at each digit level. Does not use comparisons at all.

Complexity: $O(nk)$ where k is the number of digits. For integers with fixed-size keys, this is effectively $O(n)$. **Verdict:** Faster than comparison-based sorts for integers or fixed-length strings when k is small. Cannot handle arbitrary comparison-based ordering. Used in systems software and databases.

3.1.9 Timsort

Idea: A hybrid of merge sort and insertion sort. Finds naturally occurring sorted "runs" in the data, uses insertion sort to extend short runs, then merges them using a merge sort variant with several clever optimizations.

Complexity: $O(n \log n)$ worst case. $O(n)$ on nearly-sorted data. **Verdict:** The default sort in Python (`list.sort()`), Java (`Arrays.sort()` for objects), and many other languages. Exceptional real-world performance because real data is rarely random—it tends to have existing structure that Timsort exploits.

3.1.10 Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Educational
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Best for small
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Predictable, l
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Fast in practi
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	In-place but p
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes	Non-compari
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Best general-

3.2 Searching

3.2.1 Linear Search

Scan every element. $O(n)$. Works on unsorted data. Use when the list is small or unsorted and you're searching once.

3.2.2 Binary Search

Idea: On sorted data, compare the target to the middle element. If it matches, done. If the target is smaller, search the left half; if larger, search the right half. Repeat.

```

binarySearch(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: low = mid + 1

```

```
        else: high = mid - 1  
    return -1
```

Complexity: $O(\log n)$. Requires sorted input. **Verdict:** Extraordinarily powerful. Searching a billion elements takes about 30 comparisons. Used everywhere sorted data exists—databases, dictionaries, game AI (bisection), autocomplete.

3.2.3 Hash-Based Lookup

Not a search algorithm in the traditional sense, but achieves $O(1)$ average lookup using a hash function to map keys to array indices. Used in hash maps, sets, caches, and symbol tables. The tradeoff is $O(n)$ worst case (hash collisions), memory overhead, and lack of ordering.

Chapter 4

Part II: Graph Algorithms

Graphs are everywhere: social networks, road maps, the internet, dependency trees, molecular structures, circuit layouts. Graph algorithms are among the most powerful and widely applied in computer science.

A graph is a set of **nodes** (vertices) connected by **edges**. Edges can be directed or undirected, weighted or unweighted.

4.1 Graph Traversal

4.1.1 Breadth-First Search (BFS)

Idea: Explore all neighbors of the current node before moving deeper. Uses a queue. Processes nodes level by level.

```
BFS(graph, start):
    queue = [start]
    visited = {start}
    while queue:
        node = queue.pop(front)
        process(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Complexity: $O(V + E)$ where V = vertices, E = edges. **What it's good for:**

Finding the shortest path in an unweighted graph, level-order traversal, finding all nodes reachable from a source, detecting bipartiteness.

4.1.2 Depth-First Search (DFS)

Idea: Go as deep as possible along one path before backtracking. Uses a stack (or recursion).

```
DFS(graph, node, visited):
    visited.add(node)
    process(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            DFS(graph, neighbor, visited)
```

Complexity: $O(V + E)$. **What it's good for:** Detecting cycles, topological sorting, finding connected components, solving mazes, generating spanning trees, backtracking problems.

4.2 Shortest Path Algorithms

4.2.1 Dijkstra's Algorithm

Idea: Find the shortest path from a source node to all other nodes in a weighted graph with non-negative edge weights. Use a priority queue (min-heap). Always process the unvisited node with the smallest known distance.

```
Dijkstra(graph, source):
    dist = {node: inf for node in graph}
    dist[source] = 0
    priority_queue = [(0, source)]
    while priority_queue:
        d, node = extract_min(priority_queue)
        for neighbor, weight in graph[node]:
            if dist[node] + weight < dist[neighbor]:
                dist[neighbor] = dist[node] + weight
```

```
        insert(priority_queue, (dist[neighbor], neighbor))
    return dist
```

Complexity: $O((V + E) \log V)$ with a binary heap. **What it's good for:** GPS navigation, network routing (OSPF protocol), game AI pathfinding (A* is an extension). **Cannot handle negative edge weights.**

4.2.2 Bellman-Ford Algorithm

Idea: Relax all edges $V-1$ times. Each iteration guarantees shortest paths of at most k edges for increasing k .

Complexity: $O(VE)$ —slower than Dijkstra but handles negative weights and detects negative cycles. **What it's good for:** Financial arbitrage detection, routing protocols (RIP), any graph with negative weights.

4.2.3 Floyd-Warshall Algorithm

Idea: Dynamic programming. Compute shortest paths between *all pairs* of nodes. For each intermediate node k , update all pairs (i, j) : if going through k is shorter, update.

Complexity: $O(V^3)$ time, $O(V^2)$ space. **What it's good for:** Dense graphs, all-pairs shortest paths, transitive closure, detecting negative cycles.

4.2.4 A* (A-Star)

Idea: An extension of Dijkstra that uses a **heuristic function** $h(n)$ estimating the cost from node n to the goal. Instead of purely minimizing known distance, it minimizes $g(n) + h(n)$ where $g(n)$ is the known cost from the source. With an admissible heuristic (one that never overestimates), A* finds the optimal path faster than Dijkstra by focusing the search toward the goal.

What it's good for: Game pathfinding, robotics, GPS (with geographic heuristics like straight-line distance). The dominant pathfinding algorithm in practice.

4.3 Minimum Spanning Trees

A spanning tree of a graph connects all nodes with exactly $V-1$ edges and no cycles. A *minimum* spanning tree minimizes the total edge weight.

4.3.1 Kruskal's Algorithm

Sort all edges by weight. Add each edge to the spanning tree if it doesn't create a cycle (use a Union-Find data structure to check).

Complexity: $O(E \log E)$. Better for sparse graphs.

4.3.2 Prim's Algorithm

Start from any node. Repeatedly add the lowest-weight edge connecting the current tree to a new node.

Complexity: $O(E \log V)$ with a heap. Better for dense graphs.

Real-world uses: Network design (laying cables, pipelines), clustering algorithms (single-linkage clustering is equivalent to MST), approximation algorithms for TSP.

4.4 Topological Sort

Problem: Given a directed acyclic graph (DAG), order the nodes such that for every edge $(u \rightarrow v)$, u comes before v .

Idea: Use DFS. When a node's subtree is fully explored, add it to the front of the result list.

What it's good for: Build systems (make, gradle), dependency resolution (package managers), task scheduling, spreadsheet cell evaluation order.

4.5 Union-Find (Disjoint Set Union)

Not a graph traversal but a fundamental data structure for graph algorithms. Tracks which nodes belong to the same connected component with two

operations: **Find** (which component is this in?) and **Union** (merge two components). With path compression and union by rank, both operations run in near- $O(1)$ amortized time.

Used in: Kruskal's algorithm, network connectivity, percolation problems, Kruskal's MST.

Chapter 5

Part III: Dynamic Programming

Dynamic programming (DP) is one of the most powerful algorithm design techniques. It is applicable whenever a problem has two properties: **optimal substructure** (the optimal solution contains optimal solutions to subproblems) and **overlapping subproblems** (the same subproblems are solved multiple times).

The key insight: solve each subproblem once, store the result, and reuse it.

5.1 Two Approaches

Top-down (Memoization): Write the recursive solution naturally, but cache the result of each subproblem so it is only computed once.

Bottom-up (Tabulation): Identify the order in which subproblems must be solved, build a table starting from the base cases, and fill in larger subproblems using already-computed values.

5.2 Classic DP Problems

5.2.1 Fibonacci Numbers

The canonical example. Naively: $O(2^n)$ recursive calls. With memoization: $O(n)$ calls, $O(n)$ space. With bottom-up DP: $O(n)$ time, $O(1)$ space (only need the last two values).

5.2.2 Longest Common Subsequence (LCS)

Problem: Given two strings, find the length of their longest common subsequence (characters don't need to be contiguous).

Idea: Build a 2D table $dp[i][j]$ = LCS of first i characters of string 1 and first j characters of string 2. Fill it in order.

```
if s1[i] == s2[j]:
    dp[i][j] = dp[i-1][j-1] + 1
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Complexity: $O(mn)$ time and space where m, n are string lengths. **Uses:** Diff tools (git diff), DNA sequence alignment (bioinformatics), file comparison.

5.2.3 Edit Distance (Levenshtein Distance)

Problem: Minimum number of insertions, deletions, or substitutions to transform one string into another.

Complexity: $O(mn)$. **Uses:** Spell checkers, DNA alignment, fuzzy string matching, natural language processing.

5.2.4 0/1 Knapsack Problem

Problem: Given items with weights and values, and a weight capacity, maximize the total value without exceeding capacity. Each item can be taken or left.

Idea: $dp[i][w]$ = maximum value using the first i items with capacity w .

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i])
```

Complexity: $O(nW)$ where n = items, W = capacity. **Uses:** Resource allocation, portfolio optimization, cargo loading, cutting stock problems.

5.2.5 Coin Change

Problem: Given coin denominations and a target amount, find the minimum number of coins to make the amount.

Complexity: $O(\text{amount} \times \text{num_coins})$. **Uses:** Currency systems, scheduling (minimum number of intervals), cutting problems.

5.2.6 Matrix Chain Multiplication

Problem: Given a sequence of matrices to multiply, find the optimal parenthesization that minimizes the number of scalar multiplications.

Uses: Compiler optimization, graphics pipelines (matrix transforms), scientific computing.

5.2.7 Interval DP and Tree DP

DP generalizes naturally to intervals (e.g., optimal polygon triangulation, burst balloons) and trees (e.g., maximum independent set on a tree, tree diameter). These are more advanced patterns that appear in competitive programming and systems like compilers and parsers.

Chapter 6

Part IV: Greedy Algorithms

A greedy algorithm makes the locally optimal choice at each step with the hope that local optima lead to a global optimum. Greedy algorithms are fast and simple, but they don't always work—proving correctness requires careful analysis (usually via an exchange argument: showing that any solution can be transformed into the greedy solution without getting worse).

6.1 When Greedy Works

Greedy works when the problem has the **greedy choice property**—a globally optimal solution can be reached by always making the locally optimal choice—and **optimal substructure**.

6.2 Classic Greedy Algorithms

6.2.1 Activity Selection (Interval Scheduling)

Problem: Given a set of activities with start and end times, select the maximum number of non-overlapping activities.

Greedy choice: Always select the activity with the earliest end time that doesn't conflict with the last selected activity.

Why it works: By choosing the activity that ends earliest, we leave the maximum possible time for future activities.

Complexity: $O(n \log n)$ to sort, $O(n)$ to scan. **Uses:** Job scheduling, calendar optimization, resource allocation.

6.2.2 Huffman Coding

Problem: Assign variable-length binary codes to characters such that more frequent characters get shorter codes, minimizing total encoded length.

Greedy choice: Always merge the two nodes with the lowest frequency into a new combined node (using a min-heap). Repeat until one tree remains.

Why it works: Provably optimal prefix-free code via an exchange argument.

Complexity: $O(n \log n)$. **Uses:** ZIP, gzip, PNG, JPEG, and most compression formats use Huffman coding as a component.

6.2.3 Dijkstra's Algorithm

Dijkstra is fundamentally a greedy algorithm: at each step, commit to the shortest currently-known path. Its correctness relies on the fact that edge weights are non-negative (so we can never find a shorter path by waiting).

6.2.4 Fractional Knapsack

Unlike the 0/1 knapsack (which requires DP), when items can be fractionally included, the greedy approach works: sort by value/weight ratio, take items greedily from highest ratio down.

6.2.5 Minimum Spanning Trees (Prim's and Kruskal's)

Both are greedy algorithms. Kruskal's always takes the cheapest available edge that doesn't create a cycle. Prim's always adds the cheapest edge connecting the current tree to a new node. Both provably find the global minimum.

Chapter 7

Part V: Divide and Conquer

The pattern: split the problem into smaller subproblems, solve them recursively, combine the results. Unlike DP, subproblems in divide and conquer are typically *independent* (not overlapping).

The mathematics of divide and conquer is captured by recurrence relations, solved with the Master Theorem.

7.1 Master Theorem

For a recurrence $T(n) = aT(n/b) + f(n)$:

- If $f(n) = O(n^{\log_b a} - \epsilon)$: $T(n) = \Theta(n^{\log_b a})$
- If $f(n) = \Theta(n^{\log_b a})$: $T(n) = \Theta(n^{\log_b a} \log n)$
- If $f(n) = \Omega(n^{\log_b a} + \epsilon)$: $T(n) = \Theta(f(n))$

Merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$. Binary search: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.

7.2 Key Divide and Conquer Algorithms

7.2.1 Binary Search

Already covered under searching. The archetypal divide and conquer algorithm.

7.2.2 Merge Sort and Quicksort

Already covered under sorting.

7.2.3 Karatsuba Multiplication

Problem: Multiply two n -digit numbers. The naive algorithm is $O(n^2)$.

Idea: Split each number into two halves. Compute three multiplications instead of four using algebraic manipulation. Recurse.

Complexity: $O(n^{1.585})$ —significantly faster for large numbers. **Uses:** Big integer libraries (Python’s built-in int uses Karatsuba), cryptographic operations on large numbers.

7.2.4 Fast Fourier Transform (FFT)

Idea: The Discrete Fourier Transform naively takes $O(n^2)$. The Cooley-Tukey FFT algorithm recursively splits the DFT into smaller DFTs, achieving $O(n \log n)$.

Complexity: $O(n \log n)$. **Uses:** Audio and video compression (MP3, JPEG use it), signal processing, polynomial multiplication, solving partial differential equations. One of the most important algorithms ever discovered.

7.2.5 Closest Pair of Points

Problem: Given n points in a plane, find the two closest.

Naive: $O(n^2)$. Divide and conquer: $O(n \log n)$. Split points by x -coordinate, recursively find closest pair in each half, then check a narrow strip around the dividing line (this step requires careful analysis to show it runs in $O(n)$).

Chapter 8

Part VI: Backtracking

Backtracking is a systematic method for solving constraint satisfaction and combinatorial problems by building solutions incrementally and abandoning ("pruning") branches that cannot lead to valid solutions.

The structure is always: choose, explore, unchoose.

8.1 The Backtracking Template

```
backtrack(state, choices):
    if is_solution(state):
        record(state)
        return
    for choice in choices:
        if is_valid(state, choice):
            make_choice(state, choice)
            backtrack(state, remaining_choices)
            undo_choice(state, choice)  <- the "backtrack" step
```

8.2 Classic Backtracking Problems

8.2.1 N-Queens

Place N queens on an $N \times N$ chessboard so no two queens attack each other. Backtrack whenever a placement leads to a conflict.

8.2.2 Sudoku Solver

Fill in the grid so every row, column, and 3×3 box contains digits 1-9. For each empty cell, try each valid digit; backtrack if none work.

8.2.3 Permutations and Combinations

Generate all permutations of a set, all combinations of size k , all subsets (the power set). Backtracking naturally enumerates these.

8.2.4 Constraint Satisfaction Problems (CSPs)

Map coloring (color regions so no adjacent regions share a color), scheduling, and many planning problems reduce to CSPs solved by backtracking with constraint propagation.

Pruning strategies are critical for performance:

- **Forward checking:** After each assignment, eliminate inconsistent values from other variables' domains.
- **Arc consistency (AC-3):** More aggressive constraint propagation.
- **Minimum remaining values (MRV) heuristic:** Choose the variable with the fewest remaining valid values next.

Chapter 9

Part VII: String Algorithms

Text is everywhere, and processing it efficiently is one of the core problems of computing.

9.1 Pattern Matching

9.1.1 Naive Algorithm

For each position in the text, check if the pattern matches starting there.

Complexity: $O(nm)$ where n = text length, m = pattern length.

9.1.2 Knuth-Morris-Pratt (KMP)

Idea: Precompute a *failure function* for the pattern that encodes the longest proper prefix that is also a suffix for each prefix of the pattern. Use this to avoid re-examining characters in the text after a mismatch.

Complexity: $O(n + m)$ —linear time. Preprocessing is $O(m)$. **Uses:** Text editors, grep, DNA sequence search.

9.1.3 Boyer-Moore

Idea: Scan the pattern right-to-left. Use two heuristics—the bad character rule and the good suffix rule—to skip large portions of the text on mismatches.

Complexity: $O(nm)$ worst case but $O(n/m)$ in practice on natural text—often faster than KMP in real-world use. **Uses:** `grep` (default in many implementations), text editors, file search.

9.1.4 Rabin-Karp

Idea: Use hashing. Compute a rolling hash of the current window of text and compare to the pattern's hash. Only do a full comparison on hash matches.

Complexity: $O(nm)$ worst case (hash collisions), $O(n + m)$ expected. **Uses:** Plagiarism detection, finding multiple patterns simultaneously, rolling hash applications.

9.2 String Data Structures

9.2.1 Suffix Array

An array of all suffixes of a string sorted lexicographically. Enables $O(m \log n)$ pattern search after $O(n \log n)$ construction. **Uses:** Bioinformatics (genomic sequence analysis), text indexing, longest common substring.

9.2.2 Trie (Prefix Tree)

A tree where each node represents a prefix of a string. Enables $O(m)$ lookup, insertion, and prefix search where m = length of the query string. **Uses:** Autocomplete, spell checkers, IP routing (longest prefix match), word games.

9.3 Edit Distance and Sequence Alignment

Already covered under dynamic programming, but worth emphasizing: the algorithms for comparing strings (edit distance, LCS, Smith-Waterman, Needleman-Wunsch) are foundational to bioinformatics, diff tools, machine translation, and speech recognition.

Chapter 10

Part VIII: Randomized Algorithms

Randomized algorithms use random numbers as part of their logic. They often achieve better average-case performance or simpler implementations than deterministic alternatives, at the cost of probabilistic (rather than certain) correctness.

10.1 Types

Las Vegas algorithms always produce the correct answer; randomness only affects running time. *Randomized quicksort* is the canonical example—random pivot selection guarantees $O(n \log n)$ expected time while eliminating adversarial worst-case inputs.

Monte Carlo algorithms always terminate quickly but may produce incorrect results with some small probability. *Miller-Rabin primality testing* is a Monte Carlo algorithm: it can declare a composite number prime (but with probability at most $1/4$ per round, driven to negligible levels with repeated rounds).

10.2 Key Randomized Algorithms

10.2.1 Randomized Quicksort

Choose the pivot uniformly at random. Expected $O(n \log n)$ regardless of input. Used in practice to neutralize adversarial inputs.

10.2.2 Miller-Rabin Primality Test

Problem: Is a given number n prime? Trial division is $O(\sqrt{n})$ —infeasible for cryptographic-size numbers.

Idea: Probabilistically test properties that every prime must satisfy. If n fails, it's definitely composite. If it passes k rounds, it's prime with probability at least $1 - (1/4)^k$.

Complexity: $O(k \log^2 n)$ per test. **Uses:** RSA key generation, generating large primes for cryptography.

10.2.3 Hash Functions and Universal Hashing

Random hash functions can achieve expected $O(1)$ lookup even against adversarial data. The theory of universal hash families guarantees low collision probability regardless of the input distribution.

10.2.4 Randomized Min-Cut (Karger's Algorithm)

Repeatedly contract random edges to find the minimum cut in a graph. Elegant proof that a simple random process solves a non-trivial graph problem.

Chapter 11

Part IX: Computational Geometry

Algorithms for geometric objects: points, lines, polygons, circles.

11.1 Key Problems and Algorithms

11.1.1 Convex Hull

Problem: Find the smallest convex polygon containing all points.

Graham Scan: Sort by polar angle, sweep counterclockwise, maintain a stack. $O(n \log n)$. **Quickhull:** Divide-and-conquer approach analogous to quicksort. $O(n \log n)$ expected.

Uses: Collision detection, shape analysis, gift wrapping problems, clustering.

11.1.2 Line Intersection (Sweep Line)

Problem: Given n line segments, find all intersections.

Naive: $O(n^2)$. **Sweep line (Bentley-Ottmann):** $O((n + k) \log n)$ where k = intersections. The sweep line paradigm—moving a vertical line across the plane and maintaining an active data structure—applies to many geometric problems.

11.1.3 Voronoi Diagram and Delaunay Triangulation

Voronoi diagram: Partition the plane into regions, each region being the set of points closest to one particular input point. Fortune's algorithm: $O(n \log n)$ sweep line.

Delaunay triangulation: The dual of the Voronoi diagram. Maximizes the minimum angle of triangles, avoiding thin sliver triangles.

Uses: GPS, nearest-neighbor queries, mesh generation, geographic information systems, robotics path planning.

Chapter 12

Part X: Machine Learning Algorithms

Machine learning algorithms are distinguished by the fact that they learn from data rather than following explicitly programmed rules. The algorithm specifies a model structure and a training procedure; the model's behavior is determined by data.

12.1 Supervised Learning

12.1.1 Linear Regression

Fits a linear function to minimize squared error. Solved analytically (closed-form solution: the normal equations) or iteratively (gradient descent). The simplest learned algorithm.

12.1.2 Logistic Regression

Despite the name, a classification algorithm. Applies the sigmoid function to a linear combination of features to produce a probability. Trained with gradient descent on the cross-entropy loss.

12.1.3 Decision Trees

Recursively split data by the feature and threshold that best separates classes (measured by information gain or Gini impurity). Interpretable but

prone to overfitting.

12.1.4 Random Forests

Ensemble of decision trees trained on random subsets of data and features. Reduces overfitting through averaging. Robust, accurate, and widely used.

12.1.5 Support Vector Machines (SVMs)

Find the hyperplane that maximizes the margin between classes. The kernel trick extends SVMs to non-linear boundaries. State-of-the-art for many classification tasks before deep learning.

12.1.6 Neural Networks and Gradient Descent

The universal approximation theorem states that a sufficiently wide neural network can approximate any continuous function. Training uses **back-propagation** (efficient computation of gradients via the chain rule) and **gradient descent** (update weights in the direction that decreases the loss).

Variants of gradient descent: SGD, Adam, RMSProp—all variations on how to estimate and apply gradients efficiently.

12.2 Unsupervised Learning

12.2.1 K-Means Clustering

Idea: Assign each point to its nearest centroid; update centroids to the mean of their assigned points; repeat.

Complexity: $O(nkt)$ where n = points, k = clusters, t = iterations. Converges to a local optimum (not necessarily global).

12.2.2 Principal Component Analysis (PCA)

Find the directions of maximum variance in high-dimensional data. Reduces dimensionality while preserving structure. Computed via eigendecomposition or SVD of the covariance matrix.

12.2.3 Autoencoders and Generative Models

Neural network-based unsupervised learning. Autoencoders learn compressed representations. VAEs and GANs learn to generate new data samples.

12.3 Reinforcement Learning

Q-Learning: Learn a value function $Q(s, a)$ estimating the long-term reward of taking action a in state s . Update using the Bellman equation.

Policy Gradient Methods: Directly optimize a policy (probability distribution over actions) by gradient ascent on expected reward.

AlphaGo / AlphaZero: Combine Monte Carlo Tree Search (MCTS) with deep neural networks, trained via self-play. Demonstrated superhuman performance in Go, Chess, and Shogi.

Chapter 13

Part XI: Cryptographic Algorithms

Cryptography is the science of secure communication. Cryptographic algorithms are specifically designed to be computationally hard to reverse without a secret key.

13.1 Symmetric Encryption

AES (Advanced Encryption Standard): The current standard for symmetric key encryption. Uses a series of substitution, permutation, and mixing operations (a substitution-permutation network) over 128/192/256-bit keys. Designed to be efficient in hardware and software while resisting all known attacks.

13.2 Public Key Cryptography

13.2.1 RSA

Idea: Multiplication is easy; factoring is hard. Generate two large primes p and q . Publish $n = p \times q$ and an exponent e as the public key. Keep d (the modular inverse of e modulo $(p-1)(q-1)$) as the private key. Encrypting is exponentiation mod n ; decrypting requires knowing the factorization.

Security: Based on the conjectured hardness of factoring. A 2048-bit RSA

key is currently secure; quantum computers (Shor's algorithm) could break it.

13.2.2 Elliptic Curve Cryptography (ECC)

Achieves equivalent security to RSA with much smaller keys (256-bit ECC $\approx$ 3072-bit RSA) using the algebraic structure of elliptic curves over finite fields. The basis of most modern TLS, Bitcoin, and signal protocol.

13.2.3 Diffie-Hellman Key Exchange

Allows two parties to establish a shared secret over a public channel without ever transmitting the secret. Based on the discrete logarithm problem.

13.3 Hash Functions

A cryptographic hash function maps arbitrary data to a fixed-size digest. Properties: deterministic, fast to compute, pre-image resistant (can't find input from output), collision resistant (can't find two inputs with the same output).

SHA-256: The workhorse of cryptographic hashing. Used in digital signatures, TLS, Bitcoin's proof-of-work, password storage (combined with salting), and data integrity verification.

Chapter 14

Design Patterns Summary

Stepping back, most algorithms fall into a small number of design patterns. Recognizing which pattern applies is the first skill of algorithm design.

Pattern	Core Idea	Key Examples
Brute Force	Try all possibilities	Naive polynomial
Divide and Conquer	Split, recurse, combine	Merge sort
Dynamic Programming	Reuse overlapping subproblem solutions	Longest Common Subsequence, knapsack
Greedy	Locally optimal choice at each step	Huffman coding
Backtracking	Build incrementally, prune dead ends	N-Queens
Randomization	Use randomness to improve average case	Randomized algorithms
Two Pointers	Maintain two indices to avoid nested loops	Sorted arrays
Sliding Window	Maintain a moving window over sequential data	Maximum sum subarray
Sweep Line	Move a line across space, process events	Intersection of rectangles
Reduction	Transform one problem to another known problem	NP-hard problems

14.1 What Makes an Algorithm "Good"?

Correctness, efficiency, and simplicity are the three virtues—and they are frequently in tension. Quicksort is not as simple as insertion sort. Fibonacci with memoization is not as simple as the naive recursion. The FFT is not simple at all.

Beyond the academic benchmarks, real-world algorithm selection involves: cache behavior, parallelizability, implementation complexity, input dis-

tribution, and whether approximate solutions are acceptable. The "best" algorithm is always relative to the problem context.

14.2 What Comes Next

This document has covered the major categories—sorting, searching, graphs, dynamic programming, greedy, divide and conquer, backtracking, strings, randomization, geometry, ML, and cryptography—with enough depth to understand how each works and when to reach for it.

From here, the natural directions are:

- **Specific deep dives**—going fully inside one category (e.g., all graph algorithms in detail, or the full ML training pipeline)
- **Data structures**—the structures that algorithms operate on (heaps, trees, hash tables, skip lists, bloom filters) are half the story
- **Complexity theory**—understanding *why* certain problems are hard, what NP-completeness really means, and what the theoretical limits are
- **Algorithmic problem solving**—working through canonical problem sets and building the pattern recognition to apply the right tool

Next: Data Structures—the other half of the algorithmic story

Chapter 15

Algorithms: Data Structures

15.0.1 The Containers That Make Algorithms Work

This is Part 3 of the Algorithms series. Part 1 covers history and theory. Part 2 covers algorithm categories and implementations.

15.1 Why Data Structures Are Half the Story

An algorithm is a procedure. A data structure is an organization. They are inseparable—the same algorithm can be fast or slow, elegant or convoluted, depending entirely on how the data it operates on is organized.

Dijkstra's algorithm is $O(V^2)$ with an array and $O((V+E) \log V)$ with a binary heap. Merging two sorted lists is $O(n)$ with a linked list and $O(n)$ with an array—but searching is $O(n)$ in a linked list and $O(\log n)$ in a sorted array. Every data structure makes certain operations fast and others slow. The skill is knowing what operations your algorithm needs and choosing accordingly.

Data structures can be understood along two axes:

By shape: Arrays, linked structures, trees, graphs, hash-based structures.

By purpose: General containers, ordered collections, priority structures, associative maps, specialized index structures.

This document covers both—starting from the primitives and building to the sophisticated.

Chapter 16

Part I: Primitives and Linear Structures

16.1 Arrays

The most fundamental data structure: a contiguous block of memory divided into fixed-size cells, each accessible in $O(1)$ by index.

Strengths: $O(1)$ random access by index. Excellent cache locality (elements are contiguous in memory—the CPU prefetcher loves this). Simple to reason about.

Weaknesses: Fixed size (in most languages, unless using dynamic arrays). Insertion and deletion in the middle require $O(n)$ shifting.

Dynamic Arrays (ArrayList, Vec, Python list): Wrap an array and automatically resize when capacity is exceeded. The trick is *doubling* the capacity on resize—amortized analysis shows that even though occasional resizes cost $O(n)$, the average cost per insertion is $O(1)$. This is one of the cleanest examples of amortized analysis.

16.2 Linked Lists

A sequence of nodes where each node holds a value and a pointer to the next node (singly linked) or both next and previous nodes (doubly linked).

```
Node:
  value
  next -> Node
        value
        next -> Node
              value
              next -> null
```

Strengths: $O(1)$ insertion and deletion at the head (or at a known pointer). No need to shift elements. Dynamic size with no wasted capacity.

Weaknesses: $O(n)$ access by index—no random access. Poor cache performance (nodes are scattered in memory). Extra memory per element for pointers.

When to use: Implementing queues and dequeues. When you need frequent insertion/deletion at arbitrary positions and have a pointer to the location. Undo stacks. The base structure for many more advanced structures.

Real-world note: In modern hardware, the cache penalty of linked lists is significant. For small to medium collections, arrays almost always win in practice despite worse asymptotic complexity for some operations.

16.3 Stacks

Abstract data type: Last In, First Out (LIFO). Operations: push (add to top), pop (remove from top), peek (view top without removing).

Implementation: Array (with a top-of-stack index) or linked list (adding/removing at the head).

Complexity: $O(1)$ push, pop, peek.

Uses: Function call stack (the reason recursion works at all—the CPU maintains a call stack). Expression evaluation and parsing. Undo/redo systems. DFS traversal. Balancing parentheses. Browser back button (roughly). Compiler symbol tables.

16.4 Queues

Abstract data type: First In, First Out (FIFO). Operations: enqueue (add to back), dequeue (remove from front).

Implementation: Circular array (efficient, fixed size) or doubly linked list. A naive array implementation that doesn't recycle space wastes memory; the circular array wraps around.

Complexity: $O(1)$ enqueue, dequeue.

Uses: BFS traversal. Task scheduling (operating system process queues). Print queues. Message queues (Kafka, RabbitMQ). Network packet buffers. Simulations.

16.4.1 Deque (Double-Ended Queue)

Supports efficient insertion and removal at both ends. Python's `collections.deque` and C++'s `std::deque` implement this. Used for sliding window algorithms (keeping a window of the last k elements efficiently).

16.5 Circular Buffers

A fixed-size array used as a queue by maintaining head and tail indices that wrap around. $O(1)$ enqueue and dequeue with $O(1)$ space overhead. Common in embedded systems, audio buffers, and network I/O.

Chapter 17

Part II: Trees

Trees are hierarchical structures—one of the most powerful organizing principles in computer science. A tree is a connected graph with no cycles. Most tree data structures are rooted (one designated root node) with parent-child relationships.

17.1 Binary Trees: The Foundation

A binary tree has at most two children per node (left and right). This simple constraint enables a remarkable variety of efficient algorithms.

Terminology: Root (top node), leaf (no children), height (longest root-to-leaf path), depth (distance from root), subtree (a node and all its descendants).

Full binary tree: Every node has 0 or 2 children. **Complete binary tree:** All levels fully filled except possibly the last, which is filled left to right.

Balanced binary tree: Height is $O(\log n)$.

17.2 Binary Search Trees (BST)

Property: For every node, all values in the left subtree are smaller; all values in the right subtree are larger.



```
graph TD; 8 -- / --> L; 8 -- \ --> R;
```



Operations: Search, insert, delete—all $O(h)$ where h = height.

The catch: A naive BST can degenerate. Insert elements in sorted order and you get a linked list (height n , all operations $O(n)$). This is why *balanced* BSTs matter.

17.3 AVL Trees

The first self-balancing BST (Adelson-Velsky and Landis, 1962). Maintains the invariant that for every node, the heights of left and right subtrees differ by at most 1.

After every insertion or deletion, the tree may need to be rebalanced via **rotations**—local restructuring operations that restore balance in $O(1)$ time. At most $O(\log n)$ rotations needed per operation.

Complexity: $O(\log n)$ search, insert, delete. Guaranteed.

Tradeoffs: Strictly balanced, so searches are fast. More rotations on insertion/deletion than Red-Black trees. Used when reads significantly outnumber writes.

17.4 Red-Black Trees

A self-balancing BST with looser balance constraints than AVL. Each node is colored red or black, and a set of coloring rules ensures the tree's height is at most $2 \log(n+1)$.

Red-Black properties:

1. Every node is red or black.
2. The root is black.

3. Every leaf (null) is black.
4. If a node is red, both children are black.
5. All paths from any node to its leaf descendants have the same number of black nodes.

Complexity: $O(\log n)$ search, insert, delete. Fewer rotations than AVL on average.

Uses: The default implementation of ordered maps and sets in most standard libraries: C++ `std::map`, Java `TreeMap`, Linux kernel's process scheduler (CFS), memory allocators.

17.5 B-Trees and B+ Trees

B-trees generalize BSTs to nodes with more than two children. A B-tree of order m allows between $\lceil m/2 \rceil$ and m children per node.

Why they exist: Disk I/O is orders of magnitude slower than memory access, and it reads data in large blocks (pages). B-trees are designed so that each node fits in one disk page, minimizing the number of disk reads for any operation.

B+ Trees store all values in leaf nodes (internal nodes store only keys for routing), and leaf nodes are linked together for efficient range scans.

Complexity: $O(\log_m n)$ for all operations. For large m (e.g., $m = 1000$), this means a tree of height 3-4 can index billions of records.

Uses: Nearly every relational database index (PostgreSQL, MySQL, SQLite, Oracle) uses a B+ tree. Filesystems (NTFS, ext4, HFS+, APFS). This is one of the most practically important data structures ever invented.

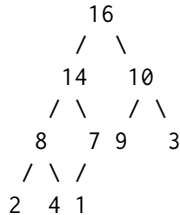
17.6 Heaps

A heap is a complete binary tree satisfying the heap property: in a max-heap, every parent is greater than or equal to its children; in a min-heap, every parent is less than or equal to its children.

Key insight: Heaps are stored as arrays—a complete binary tree maps perfectly to an array. For node at index i : left child at $2i+1$, right child at $2i+2$, parent at $\lfloor (i-1)/2 \rfloor$.

Max-heap array: [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]

Visualized:



Operations:

- Insert: Add to end, sift up. $O(\log n)$.
- Extract max/min: Remove root, put last element at root, sift down. $O(\log n)$.
- Peek max/min: $O(1)$.
- Build heap from array: $O(n)$ —surprisingly, building a heap is linear, not $O(n \log n)$.

Uses: Priority queues (the canonical use). Heapsort. Dijkstra’s and Prim’s algorithms. Job schedulers. Event-driven simulations. The `heapq` module in Python, `PriorityQueue` in Java.

17.7 Tries (Prefix Trees)

A trie stores strings by decomposing them into characters. Each edge represents a character; each path from root to a marked node represents a stored string.

Storing: "cat", "car", "card", "care", "bat"

root



Operations: Insert, search, prefix search—all $O(m)$ where m = string length. Prefix search is the killer feature: find all strings with a given prefix in $O(m + k)$ where k = number of results.

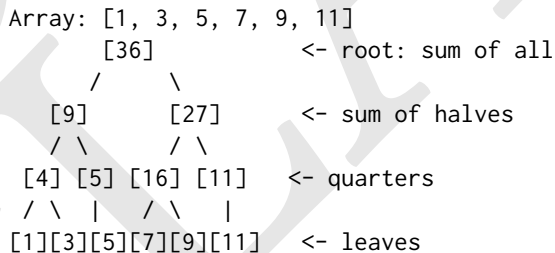
Space: Can be large—each node may have up to alphabet-size children.

Compressed tries (Patricia trees, radix trees) merge nodes with single children, reducing space.

Uses: Autocomplete. Spell checkers. IP routing (longest prefix matching—the internet relies on this). Word games (Boggle solvers). DNA sequence analysis. Symbol tables in compilers.

17.8 Segment Trees

A segment tree is a binary tree for range queries and point updates on an array. Each node represents a range $[l, r]$ and stores aggregate information (sum, min, max) for that range.



Operations:

- Build: $O(n)$
- Point update: $O(\log n)$
- Range query (sum/min/max of any subarray): $O(\log n)$

Uses: Range sum queries, range minimum queries. Competitive programming. Databases (range aggregations). Computational geometry.

Lazy propagation extends segment trees to support range updates (e.g., add 5 to all elements in $[l, r]$) in $O(\log n)$ by deferring updates.

17.9 Fenwick Trees (Binary Indexed Trees)

A simpler, more memory-efficient alternative to segment trees for the specific case of prefix sum queries and point updates.

Idea: Use the binary representation of indices to cleverly partition responsibilities among array cells.

Complexity: $O(\log n)$ update and prefix query. $O(n)$ space. Constant factor smaller than segment trees. **Uses:** Order statistics, frequency tables, inversion counting, competitive programming.

17.10 Suffix Trees and Suffix Arrays

Suffix tree: A compressed trie of all suffixes of a string. Enables $O(m)$ pattern search in a text of length n after $O(n)$ construction. Also solves longest common substring, longest palindromic substring, and many other string problems.

Suffix array: A sorted array of all suffixes. More space-efficient than a suffix tree. Combined with an LCP (longest common prefix) array, achieves equivalent functionality.

Uses: Bioinformatics (whole genome search, read mapping). Search engines. Text editors (large file search). DNA sequence databases.

17.11 Union-Find (Disjoint Set Union)

Already mentioned in graph algorithms, but worth treating fully as a data structure.

Operations: Make-set, Find (with path compression), Union (by rank or size).

```
find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])    <- path compression
    return parent[x]

union(x, y):
    rx, ry = find(x), find(y)
    if rank[rx] < rank[ry]: swap(rx, ry)
    parent[ry] = rx
    if rank[rx] == rank[ry]: rank[rx]++
```

Complexity: $O(\alpha(n))$ per operation, where α is the inverse Ackermann function—essentially $O(1)$ for all practical purposes. One of the most efficient data structures ever devised.

Uses: Kruskal's MST. Network connectivity. Percolation theory. Image segmentation. Social network connected components.

Chapter 18

Part III: Hash-Based Structures

18.1 Hash Tables

The most practically important data structure in everyday programming. Maps keys to values using a hash function.

Core idea: Compute $h(\text{key})$ to get an array index. Store the value there. Lookup, insertion, and deletion are $O(1)$ average.

The hard part: Collisions—two different keys hashing to the same index. Two strategies:

Chaining: Each array cell holds a linked list of all key-value pairs that hash there. Simple, handles high load factors gracefully. $O(1+\alpha)$ expected time where $\alpha = n/m$ (load factor).

Open addressing: On collision, probe for the next open slot according to a probe sequence (linear probing, quadratic probing, double hashing). Better cache performance than chaining. Degrades at high load factors.

Load factor management: When α exceeds a threshold (typically 0.7–0.75), resize the table (usually double the size and rehash). Amortized $O(1)$ insertions.

Hash functions: A good hash function distributes keys uniformly. Bad hash functions create clustering. Cryptographic hash functions (SHA-256) are not used for hash tables—they are too slow. Instead: polynomial rolling hashes, MurmurHash, FNV, xxHash.

Security concern: Deterministic hash functions are vulnerable to hash flooding attacks (adversarially crafted inputs that all hash to the same bucket). Modern languages use randomized hash seeds (SipHash in Python 3.3+, Rust).

Uses: Dictionaries in every modern language. Database hash indexes. Caches. Symbol tables. Sets. Deduplication.

18.2 Hash Sets

A hash table storing only keys (no values). $O(1)$ average insert, delete, and membership test.

Uses: Removing duplicates. Fast membership testing. Implementing set operations (union, intersection, difference) in $O(n)$ expected time.

18.3 Bloom Filters

A probabilistic data structure that tests set membership. Uses multiple hash functions and a bit array.

Insert: Hash the element with k hash functions, set those k bits. **Query:** Hash with k functions, check if all k bits are set. If any is 0, definitely not in set. If all are 1, *probably* in set (false positives possible, false negatives impossible).

Properties: $O(k)$ insert and query. Space-efficient. Never false negatives. False positive rate controlled by size and k .

Uses: Database query optimization (avoid disk reads for keys that don't exist). Web crawlers (visited URL tracking). Spell checkers. Network routers (packet filtering). Bitcoin SPV nodes. CDN cache filtering.

18.4 Count-Min Sketch

A probabilistic data structure for approximate frequency counting with sublinear space. Uses multiple hash functions and a 2D count array.

Uses: Heavy hitter detection in network traffic. Approximate word frequency in large text corpora. Stream processing.

18.5 Consistent Hashing

A technique for distributing keys across a changing set of servers. When a server is added or removed, only a fraction of keys need to be remapped ($O(k/n)$ where k = keys, n = servers). Standard hash tables would require remapping $O(k)$ keys.

Uses: Distributed caches (Memcached, Redis Cluster). Load balancing. Distributed databases. Content delivery networks.

Chapter 19

Part IV: Graphs as Data Structures

Algorithms operating on graphs need the graph itself stored efficiently. Two main representations:

19.1 Adjacency Matrix

An $n \times n$ matrix where entry $[i][j] = 1$ (or the edge weight) if there is an edge from i to j .

Strengths: $O(1)$ edge existence check. Simple. Good for dense graphs.

Weaknesses: $O(V^2)$ space—prohibitive for sparse graphs. $O(V^2)$ to iterate all edges.

19.2 Adjacency List

An array (or hash map) where each entry is a list of that node's neighbors.

Strengths: $O(V+E)$ space. $O(\text{degree}(v))$ to iterate neighbors. Efficient for sparse graphs (most real-world graphs are sparse). **Weaknesses:** $O(\text{degree}(v))$ to check edge existence (unless augmented with a hash set).

Most graph algorithms use adjacency lists. For social networks, the internet, road networks—graphs are sparse, and adjacency lists win.

19.3 Special Graph Structures

Trees (as graphs): Stored as adjacency lists; special tree traversal algorithms apply.

DAGs (Directed Acyclic Graphs): Enable topological sort, DP on the DAG.

Bipartite graphs: Two disjoint sets of nodes with edges only between sets. Enable efficient matching algorithms (Hopcroft-Karp).

Chapter 20

Part V: Advanced and Specialized Structures

20.1 Skip Lists

A probabilistic alternative to balanced BSTs. A linked list with multiple levels—higher levels skip over more elements, enabling $O(\log n)$ expected search, insert, delete.

Level 3: head	-----	9	-----	tail												
Level 2: head	----	3	-----	7	-----	9	-----	tail								
Level 1: head	--	1	-	3	--	5	-	7	--	8	-	9	--	12	--	tail

Complexity: $O(\log n)$ expected for all operations. $O(n)$ space. **Advantages over balanced BSTs:** Simpler to implement. Easier to make concurrent (lock-free skip lists are easier than lock-free red-black trees).

Uses: Redis uses skip lists for sorted sets. LevelDB. MemSQL. Concurrent data structures.

20.2 Disjoint Sets

(Covered in Part II under Union-Find—central enough to appear in both graph algorithms and data structure discussions.)

20.3 Sparse Tables

A static data structure for range minimum queries (RMQ) after $O(n \log n)$ preprocessing. Query in $O(1)$. Cannot handle updates.

Idea: Precompute minimums for all ranges of length 2^k . Any range query decomposes into two overlapping power-of-2 ranges.

Uses: Lowest common ancestor (LCA) in trees. Static RMQ in competitive programming. Preprocessing step in suffix array algorithms.

20.4 Interval Trees and Segment Trees (for Geometry)

Interval tree: Stores intervals (ranges) and enables efficient querying: find all intervals overlapping a given point or range. $O(\log n + k)$ query where k = overlapping intervals.

Uses: Database temporal queries. Calendar overlap detection. Computational geometry. Game collision detection (broad phase).

20.5 K-D Trees (k-dimensional trees)

A BST that partitions k -dimensional space. Each level splits on a different dimension.

Operations: Nearest neighbor search, range search— $O(\log n)$ average, $O(\sqrt{n})$ worst case for 2D.

Uses: Nearest neighbor queries. Collision detection. 3D graphics (ray tracing, photon mapping). Machine learning (k -NN classifier). Geographic information systems.

20.6 R-Trees

Spatial index for multi-dimensional bounding boxes. Groups nearby objects by bounding rectangles.

Uses: Geographic databases (PostGIS, spatial indexes). Game engines. Spatial query acceleration.

20.7 Van Emde Boas Trees

A tree structure that achieves $O(\log \log U)$ time for all operations on integer keys in the universe $0, \dots, U-1$.

Complexity: $O(\log \log U)$ insert, delete, search, successor, predecessor.

Downside: $O(U)$ space (or $O(n \log \log U)$ with hashing).

Uses: Competitive programming. Integer priority queues. Theoretical computer science. IP routing with bounded universe.

20.8 Fibonacci Heaps

A lazy heap variant that achieves amortized $O(1)$ for insert, union, and decrease-key (vs $O(\log n)$ for binary heaps). Extract-min is $O(\log n)$ amortized.

Significance: Enables Dijkstra's algorithm to run in $O(E + V \log V)$ —theoretically optimal for dense graphs. **Reality:** High constant factors and complex implementation make binary heaps faster in practice for most graph sizes. An important theoretical result; less important practically.

20.9 Persistent Data Structures

A data structure that preserves all previous versions when modified. Any update creates a new version while sharing structure with old versions (path copying).

Persistent array: $O(\log n)$ access and update, $O(\log n)$ space per version.

Persistent BST: Each insertion creates a new root and copies the path from root to the modified node ($O(\log n)$ nodes). All other nodes are shared.

Uses: Version control systems (conceptually). Functional programming languages (Clojure, Haskell use persistent structures internally). Computational geometry (persistent segment trees for offline algorithms).

20.10 Lock-Free Data Structures

Data structures designed for concurrent access without locks, using atomic hardware operations (compare-and-swap, CAS).

Lock-free stack: A singly linked list where push/pop use CAS. **Lock-free queue:** Michael-Scott queue—the basis for Java’s `ConcurrentLinkedQueue`.

Uses: High-performance concurrent systems. Operating systems. Databases. Real-time systems where lock contention is unacceptable.

Chapter 21

Part VI: The Space-Time Trade-off Landscape

Every data structure represents a choice on the spectrum between time and space:

Structure	Lookup	Insert	Delete	Space	Ordered
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	No
Sorted Array	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$	Yes
Linked List	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$	No
BST (unbalanced)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Yes
AVL / Red-Black Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Yes
B+ Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Yes
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	No
Skip List	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Yes
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Partial
Trie	$O(m)$	$O(m)$	$O(m)$	$O(\text{ALPHABET} \cdot n)$	Yes
Bloom Filter	$O(k)$	$O(k)$	N/A	$O(m)$ bits	No

Chapter 22

Part VII: Choosing the Right Data Structure

The most important decision in algorithm design is not which algorithm to use—it is which data structure to organize your data around. That choice determines which algorithms become natural and which become awkward.

Ask these questions:

What are my most frequent operations? If you need $O(1)$ lookup by key, use a hash table. If you need ordered traversal or range queries, use a tree. If you need $O(1)$ min/max, use a heap.

Do I need ordering? Hash tables are faster but unordered. Balanced BSTs, B+ trees, and skip lists provide sorted order with $O(\log n)$ operations.

What is the size? For small collections (<100 elements), a sorted array beats everything in practice due to cache performance and simplicity. For disk-resident data, B+ trees dominate.

Do I need approximate answers? Bloom filters and count-min sketches trade accuracy for dramatic space savings.

Is the data dynamic or static? Static data enables preprocessing (sparse tables for $O(1)$ RMQ, perfect hashing for $O(1)$ lookup with no collisions).

Is concurrency required? Immutable/persistent structures, lock-free structures, or MVCC (multi-version concurrency control) may be needed.

Chapter 23

Part VIII: Data Structures in the Wild

Putting it together—the data structures that power systems you use every day:

Your filesystem uses B+ trees (directory indexes) and bitmaps (block allocation tracking).

Your database uses B+ trees for most indexes, hash indexes for equality queries, and LSM-trees (log-structured merge trees) for write-heavy workloads (LevelDB, RocksDB, Cassandra).

Your browser uses a DOM tree (representing the HTML structure), a hash map for CSS rules, and priority queues for scheduling rendering tasks.

DNS uses a distributed trie-like structure (the domain name hierarchy) backed by hash maps at each resolver.

Git is fundamentally a directed acyclic graph (DAG) of commit objects, with objects stored in a content-addressed hash table (every object is named by its SHA-1 hash).

Redis uses skip lists for sorted sets, hash tables for hashes and sets, linked lists for lists, and integers for plain values.

The Linux kernel uses red-black trees for the virtual memory manager, process scheduler (CFS), and file descriptor table.

Your CPU cache uses set-associative hash-based mapping. Your branch predictor uses a small hash table. The MMU's TLB is a hardware hash table.

23.1 What Comes Next

With history and theory (Part 1), algorithm categories (Part 2), and data structures (Part 3) in hand, the foundation is complete. The natural next directions are:

Complexity Theory in Depth—NP-completeness, reductions, approximation hardness, the complexity zoo. Understanding not just what algorithms exist but why certain problems resist them.

Algorithm Design Techniques—Deeper treatment of advanced techniques: network flow, linear programming, approximation algorithm design, online algorithms and competitive analysis, parameterized complexity.

Systems and Engineering—How these structures are implemented and tuned in real systems: cache-oblivious data structures, memory allocators, database internals, compiler design.

Problem Solving Practice—Working through canonical problems that develop the pattern recognition needed to apply these tools under pressure.

Next: Complexity Theory—Understanding Why Hard Problems Are Hard

Chapter 24

Algorithms: Complexity Theory

24.0.1 Why Hard Problems Are Hard—and What "Hard" Really Means

This is Part 4 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures.

24.1 The Central Question

In Part 2, we classified algorithms by their running time: $O(n \log n)$ for merge sort, $O(V + E)$ for BFS, $O(n^2)$ for naive string matching. This is *algorithm analysis*—measuring how fast a specific solution runs.

Complexity theory asks a different and deeper question: **how fast can any solution possibly run?**

Not "is this algorithm efficient?" but "is there a fundamental barrier that prevents any algorithm from being efficient?" Not "can we do better?" but "can we *prove* that no one can do better?"

This is the difference between engineering and science. Algorithm design builds solutions. Complexity theory maps the landscape of what is and is not solvable, and why.

The results are sometimes deeply counterintuitive. Problems that look similar in structure turn out to be vastly different in difficulty. Problems

that look hard turn out to be easy with the right insight. And some problems sit in a strange middle ground where no one knows if they're hard or easy—and proving which would be the greatest mathematical achievement of our era.

Chapter 25

Part I: The Framework—What Are We Measuring?

25.1 Computational Models

To reason formally about complexity, we need a precise model of what "computation" is.

The Turing Machine (Alan Turing, 1936) is the canonical model: an infinite tape divided into cells, a read/write head, and a finite set of states. At each step, the machine reads the current cell, writes a symbol, moves left or right, and transitions to a new state. Simple and universal—it can compute anything computable.

The RAM Model (Random Access Machine) is more practical: an idealized computer with an infinite array of registers, each holding an integer, and standard operations (arithmetic, comparison, memory access) each taking $O(1)$ time. Most algorithm analysis uses this model because it matches real computers more closely.

The key result: These models are polynomially equivalent. An algorithm that runs in polynomial time on one runs in polynomial time on the other (though with different constants). This equivalence is the foundation of complexity theory—it makes polynomial time a robust notion of efficiency.

25.2 Input Size

Complexity is measured as a function of input size n . But what counts as "size"?

For sorting: n = number of elements. For graph algorithms: n = number of vertices V plus edges E . For arithmetic: n = number of *digits* (not the value itself—this distinction is crucial).

The number of bits needed to represent the input is the most precise definition. An integer value k has size $O(\log k)$ because that's how many bits it takes to write it down. This matters enormously: an algorithm that runs in time proportional to the *value* of its input (pseudo-polynomial time) may be exponential in the actual input size.

25.3 Asymptotic Notation—A Precise Review

We measure worst-case performance unless stated otherwise.

$O(f(n))$ —asymptotic upper bound. $T(n) = O(f(n))$ means $T(n) \leq c \cdot f(n)$ for some constant c and all sufficiently large n .

$\Omega(f(n))$ —asymptotic lower bound. $T(n) = \Omega(f(n))$ means $T(n) \geq c \cdot f(n)$ for some c and all large n .

$\Theta(f(n))$ —tight bound. Both O and Ω . The algorithm runs in exactly that complexity class.

$o(f(n))$ and $\omega(f(n))$ —strict upper and lower bounds (the ratio goes to 0 or ∞).

The goal of complexity theory is to prove tight bounds—to show both that a problem can be solved in $O(f(n))$ time (by exhibiting an algorithm) *and* that no algorithm can solve it in less than $\Omega(f(n))$ time (by proving a lower bound).

Chapter 26

Part II: Time Complexity Classes

26.1 P—Polynomial Time

Definition: The class of decision problems (yes/no problems) solvable by a deterministic algorithm in $O(n^k)$ time for some constant k .

In plain terms: Problems that can be solved efficiently. Polynomial time is the formal definition of "tractable."

Examples of problems in P:

- Sorting ($O(n \log n)$)
- Shortest path (Dijkstra, $O((V+E) \log V)$)
- Maximum matching in bipartite graphs ($O(E\sqrt{V})$)
- Linear programming (ellipsoid method, $O(n^6)$)
- Primality testing (AKS algorithm, $O(n^6)$ —a landmark 2002 result proving primes are in P)
- 2-SAT (determine if a 2-variable boolean formula is satisfiable— $O(V+E)$)

The choice of polynomial rather than, say, linear or quadratic is deliberate: polynomial is closed under composition (a polynomial of a polynomial is a polynomial), matches the notion of "efficiently reducible," and is robust across computational models.

The caveat: An $O(n^{100})$ algorithm is technically polynomial but practically useless. Polynomial time is necessary but not sufficient for tractability. In practice, most useful polynomial algorithms have low-degree polynomials.

26.2 NP—Nondeterministic Polynomial Time

Definition: The class of decision problems where a **yes** answer has a **certificate** (a proposed solution) that can be **verified** in polynomial time by a deterministic algorithm.

Intuition: NP problems are "easy to verify, possibly hard to solve." If someone hands you a proposed solution, you can check it quickly—but finding that solution yourself might take exponential time.

Equivalent definition: Problems solvable in polynomial time by a *non-deterministic* Turing machine—one that can "guess" the right answer and verify it. This is where "NP" gets its name (Nondeterministic Polynomial).

Examples of problems in NP:

- **3-SAT:** Given a boolean formula in conjunctive normal form with 3 literals per clause, is it satisfiable? *Verify:* Given an assignment, check all clauses in $O(n)$. *Solve:* No known polynomial algorithm.
- **Traveling Salesman Problem (decision version):** Is there a tour of all cities with total distance $\leq k$? *Verify:* Check that a given tour visits all cities and has distance $\leq k$.
- **Integer factoring:** Does n have a factor between 2 and k ? *Verify:* Given a factor, multiply to check.
- **Graph coloring:** Can a graph be colored with k colors? *Verify:* Given a coloring, check adjacent nodes have different colors.
- **Subset sum:** Does any subset of numbers sum to target T ? *Verify:* Given a subset, sum it.
- **Hamiltonian path:** Does a graph have a path visiting every vertex exactly once? *Verify:* Given a path, check it visits all vertices.

Crucially: $P \subseteq NP$. Every problem in P is also in NP (if you can solve it in polynomial time, you can certainly verify a solution in polynomial time—just solve it yourself). The question is whether the containment is strict.

26.3 The P vs NP Question

The question: Does $P = NP$?

What it's asking: If solutions to a problem can be *verified* in polynomial time, can they also be *found* in polynomial time? Is verification fundamentally easier than discovery?

The prevailing belief: $P \neq NP$. Almost all researchers believe there are problems in NP that are genuinely not in P —that verification is fundamentally easier than solution for some problems. But no one has proved it.

Why it matters practically: If $P = NP$:

- Cryptography as we know it collapses. RSA, elliptic curve cryptography, and most internet security rely on problems believed to be in NP but not P (factoring, discrete log). If $P = NP$, these are all solvable in polynomial time.
- Drug discovery, protein folding, scheduling, logistics, and AI planning problems become tractably solvable.
- Mathematical proof discovery becomes automated.

Why it's hard to prove: Proving $P \neq NP$ requires showing that no polynomial algorithm exists for some problem—a statement about all possible algorithms, including ones not yet invented. Current proof techniques are not equipped to make such sweeping statements. Natural proofs, relativization, and algebrization results show that entire families of proof strategies cannot resolve P vs NP .

The Millennium Prize: The Clay Mathematics Institute offers \$1,000,000 for a proof that $P = NP$ or $P \neq NP$. It is one of seven Millennium Prize Problems. As of 2025, it remains open.

26.4 NP-Hard and NP-Complete

NP-Hard: A problem X is NP-hard if every problem in NP can be reduced to X in polynomial time. In other words, X is "at least as hard as every NP problem." An NP-hard problem does not need to be in NP itself (it might not even be a decision problem).

NP-Complete: A problem is NP-complete if it is both NP-hard AND in NP. NP-complete problems are the hardest problems in NP.

The significance: If you could solve any NP-complete problem in polynomial time, you could solve *every* NP problem in polynomial time ($P = NP$). NP-complete problems are the "locks"—break any one, and you break them all.

26.4.1 Cook-Levin Theorem (1971)

Stephen Cook (and independently Leonid Levin) proved that **3-SAT is NP-complete**—the first problem proven to be NP-complete. This was a landmark result: it showed that NP-complete problems exist and that SAT is a natural one.

The proof idea: Any NP problem can be encoded as a 3-SAT instance. The computation of a nondeterministic Turing machine can be written as a boolean formula—variables represent tape cells, machine states, and time steps. If the machine accepts, the formula is satisfiable.

26.4.2 Karp's 21 NP-Complete Problems (1972)

Richard Karp showed 21 fundamental problems are all NP-complete by reducing each to another. This demonstrated that NP-completeness was not a curiosity but a pervasive phenomenon affecting core problems in combinatorics, graph theory, and operations research.

The original 21 include:

- 3-SAT
- Clique (does a graph contain a clique of size k ?)
- Vertex Cover

- Set Cover
- Exact Cover
- 3-Dimensional Matching
- Integer Linear Programming
- Subset Sum
- Partition
- Max Cut
- Hamiltonian Cycle
- Traveling Salesman
- Graph Coloring

26.5 Reductions—The Language of Complexity

Polynomial-time reduction ($\leq_p$): Problem A reduces to problem B ($A \leq_p B$) if any instance of A can be transformed into an instance of B in polynomial time, such that the answer to B gives the answer to A.

Intuition: "B is at least as hard as A." If you can solve B efficiently, you can solve A efficiently. If A is hard, B must be at least as hard.

To prove NP-hardness: Take a known NP-hard problem X and show $X \leq_p$ your new problem Y. Now Y is at least as hard as X—it must be NP-hard.

The web of reductions: NP-complete problems form an interconnected web. 3-SAT reduces to Independent Set, which reduces to Vertex Cover, which reduces to Set Cover. Clique reduces to Graph Coloring. Subset Sum reduces to Partition and Knapsack. This web means that hardness is not isolated—the same fundamental difficulty appears in disguise across dozens of apparently unrelated problems.

Chapter 27

Part III: Beyond NP—The Complexity Zoo

The P vs NP question is just one landmark in a rich landscape of complexity classes. Here is a map of the most important ones.

27.1 co-NP

The class of problems whose *complement* is in NP. If NP is "easy to verify yes answers," co-NP is "easy to verify no answers."

Example: The complement of 3-SAT—"is this formula unsatisfiable?"—is in co-NP. Given a purported proof of unsatisfiability, verification is hard (you'd need to check all 2^n assignments). But given a satisfying assignment (a "no to unsatisfiability" certificate), you can verify in polynomial time.

Key question: Does $\text{NP} = \text{co-NP}$? Again, unknown. Most believe $\text{NP} \neq \text{co-NP}$.

Integer factoring is believed to be in $\text{NP} \cap \text{co-NP}$ but not in P—a rare intermediate position.

27.2 PSPACE

Problems solvable with polynomial *space* (memory), regardless of time.

Examples:

- **Quantified Boolean Formula (QBF):** Determine if a boolean formula with alternating quantifiers ($\forall x_1 \exists x_2 \forall x_3 \dots$) is true. QBF is PSPACE-complete.
- Determining the winner in generalized versions of chess, checkers, and Go (these are actually EXPTIME, but similar in flavor).
- Many planning and AI problems.

Relationships: $P \subseteq NP \subseteq PSPACE$. We know $P \neq PSPACE$ (by the space hierarchy theorem), but not where exactly NP sits relative to PSPACE.

27.3 EXPTIME

Problems solvable in exponential time $O(2^{n^k})$. Strictly larger than PSPACE.

Examples: Generalized chess and Go with $n \times n$ boards are EXPTIME-complete. Certain model-checking problems.

27.4 BPP—Bounded-Error Probabilistic Polynomial Time

Problems solvable by a randomized algorithm in polynomial time with error probability at most $1/3$ (on both yes and no instances). The error can be driven arbitrarily low by repeating.

Relationship to P: It is strongly believed that $BPP = P$ —that randomness doesn't fundamentally help for decision problems. But this is unproven.

Example: Polynomial identity testing (is this arithmetic expression identically zero?) has an efficient randomized algorithm but no known deterministic polynomial algorithm.

27.5 RP and ZPP

RP (Randomized Polynomial): Yes instances accept with probability $\geq 1/2$; no instances always reject. One-sided error.

ZPP (Zero-error Probabilistic Polynomial): Always correct, expected polynomial time (Las Vegas). $ZPP = RP \cap co-RP$.

27.6 #P—Counting Complexity

#P (pronounced "sharp P") is the class of *counting* problems corresponding to NP decision problems.

Example: Not "is there a perfect matching?" (in P) but "how many perfect matchings are there?" (#P-complete).

Surprising hardness: Many counting problems are #P-hard even when the corresponding decision problem is in P. Counting the number of satisfying assignments to a 2-SAT formula is #P-complete, even though deciding if any exists is in P. Counting perfect matchings in a bipartite graph (the permanent of a 0-1 matrix) is #P-complete.

Toda's theorem (1991): Every problem in the polynomial hierarchy can be solved in polynomial time with a single #P oracle call. Counting is at least as powerful as the entire polynomial hierarchy—a stunning result.

27.7 The Polynomial Hierarchy (PH)

NP and co-NP are the first level. The polynomial hierarchy extends upward:

- $\Sigma_0^P = \Pi_0^P = P$
- $\Sigma_1^P = NP$, $\Pi_1^P = co-NP$
- Σ_2^P : Problems where you $\exists$ a witness such that $\forall$ second witnesses a polynomial verifier accepts. "There exists a circuit of size k that computes f " (you $\exists$ the circuit, an adversary $\forall$ checks all inputs).
- Π_k^P, Σ_k^P for all k , alternating $\exists$ and $\forall$ quantifiers.

PH = $\cup_k \Sigma_k^P$.

If PH collapses—if $\Sigma_k^P = \Sigma_{k+1}^P$ for some k —then all higher levels collapse to that level. It is strongly believed the hierarchy does not collapse. If $P = NP$, the entire hierarchy collapses to P.

27.8 NC and P-completeness

NC (Nick's Class): Problems solvable in polylogarithmic time $O(\log^k n)$ using a polynomial number of parallel processors.

$NC \subseteq P$. NC represents problems that are *efficiently parallelizable*.

P-complete problems are the hardest problems in P to parallelize—reducing any problem in P to them. The canonical example is Circuit Value Problem (given a boolean circuit and input, what is the output?).

Significance: Most sorting algorithms are in NC (parallel merge sort). But evaluating a sequential computation is P-complete—inherently sequential.

27.9 EXPSPACE, 2-EXPTIME, and Beyond

The hierarchy continues indefinitely. Regular expression equivalence is PSPACE-complete. Some problems in formal verification are EXPSPACE-complete. Presburger arithmetic (first-order theory of integers with addition) is doubly exponential. These are theoretical landmarks more than practical concerns.

Chapter 28

Part IV: Lower Bounds—Proving Problems Are Hard

Showing an algorithm is fast (upper bound) requires constructing one. Showing no fast algorithm can exist (lower bound) requires proving something about all possible algorithms—a fundamentally harder task.

28.1 Comparison-Based Lower Bounds

For sorting by comparisons, we can prove $\Omega(n \log n)$ is optimal.

Proof by decision tree: Any comparison-based sorting algorithm can be modeled as a binary decision tree—each internal node is a comparison, each leaf is a permutation of the output. To correctly sort all $n!$ possible input orderings, the tree must have at least $n!$ leaves. A binary tree of height h has at most 2^h leaves. So $h \geq \log_2(n!) = \Omega(n \log n)$ by Stirling's approximation. This proves no comparison-based sorting algorithm can beat $O(n \log n)$.

This is a beautiful and complete lower bound—it explains exactly why merge sort and heapsort are optimal (among comparison-based sorts) and why radix sort can break the barrier (it's not comparison-based).

28.2 Information-Theoretic Lower Bounds

A related technique: if the output requires $\Omega(f(n))$ bits of information, the algorithm must take at least $\Omega(f(n))$ time just to write the output. Binary search is optimal ($\Omega(\log n)$) because distinguishing among n possible positions requires $\log_2 n$ bits of information.

28.3 Adversary Arguments

Idea: Imagine an adversary who watches your algorithm and answers queries adversarially (consistently, but trying to force you to make as many queries as possible). If any consistent adversary strategy requires $\Omega(f(n))$ queries, no algorithm can do better.

Example: Finding the minimum in an unsorted array requires $\Omega(n)$ comparisons. Adversary strategy: always claim a new element is smaller than all previously seen. Any algorithm that doesn't examine an element might miss the minimum.

Example: Finding both minimum and maximum simultaneously can be done in $\lfloor 3n/2 \rfloor - 2$ comparisons—better than the naive $2n - 2$. An adversary argument proves this is optimal.

28.4 NP-Hardness Reductions as Lower Bounds

Proving a problem is NP-hard is a conditional lower bound: "if $P \neq NP$, then this problem has no polynomial algorithm." This is the workhorse of complexity lower bounds in practice.

The reduction-based methodology has produced thousands of NP-hardness proofs. Almost every combinatorial optimization problem of practical interest has been analyzed this way.

28.5 Circuit Complexity

To prove $P \neq NP$, many researchers have focused on **circuit complexity**—proving that certain functions require exponential-size boolean circuits.

It is known that most Boolean functions require exponential circuits (by a counting argument—there are more functions than small circuits). But proving *specific* explicit functions require large circuits has been surprisingly difficult. The best lower bounds for explicit functions are only slightly superlinear.

Natural proofs barrier (Razborov-Rudich, 1994): A large class of proof techniques ("natural proofs") cannot prove superpolynomial lower bounds unless cryptographically secure functions don't exist. This result showed that most existing circuit complexity techniques are fundamentally limited.

Chapter 29

Part V: Dealing with Hardness— When You Can't Avoid NP

For NP-hard problems in practice, there are four strategies:

29.1 1. Approximation Algorithms

Accept a solution that is provably near-optimal rather than exactly optimal.

Approximation ratio: An algorithm has ratio α if it always returns a solution within factor α of optimal. ($\alpha = 1$ means exact; $\alpha = 2$ means at most twice the optimal cost.)

Vertex Cover: The problem of finding the smallest set of vertices that covers all edges. NP-complete. But a simple greedy gives a 2-approximation: repeatedly pick any uncovered edge, add both endpoints, remove covered edges. Proved optimal under $P \neq NP$ (assuming the Unique Games Conjecture).

Set Cover: Greedily always pick the set covering the most uncovered elements. Achieves $O(\log n)$ -approximation. Proved that no polynomial algorithm can do better unless $P = NP$.

Traveling Salesman (metric TSP): For instances satisfying the triangle inequality, the Christofides algorithm achieves a $3/2$ -approximation—a 1976 result that stood for 47 years until a 2023 breakthrough achieved $(3/2 - \epsilon)$ for the first time.

Inapproximability: Some NP-hard problems are hard to even approximate. The PCP theorem proves that 3-SAT cannot be approximated within some constant factor unless $P = NP$. This established the field of inapproximability—proving lower bounds on approximation ratios.

29.2 2. Parameterized Complexity (FPT)

Many NP-hard problems become tractable when a natural parameter k is small.

Fixed Parameter Tractable (FPT): A problem is FPT with parameter k if it can be solved in $f(k) \cdot n^c$ time—polynomial in n , with the exponential explosion confined to k .

Vertex Cover parameterized by k : Is there a vertex cover of size $\leq k$? Solvable in $O(2^k \cdot n)$. For small k (say, $k \leq 30$), this is fast even for large n .

Kernelization: Reduce the problem instance to a kernel whose size is bounded by a function of k alone (independent of n). Vertex Cover has a kernel of size $O(k^2)$ —an instance with more than k^2 edges definitely has no cover of size k .

Tree-width: Many NP-hard graph problems become polynomial (even linear) when restricted to graphs of bounded tree-width—a measure of how "tree-like" a graph is.

Significance: Parameterized complexity explains why NP-hard problems are sometimes easy in practice—the natural structure of real instances keeps the relevant parameter small.

29.3 3. Exact Exponential Algorithms

Accept exponential time but be smarter than brute force.

3-SAT: Brute force checks all 2^n assignments. The DPLL algorithm (Davis-Putnam-Logemann-Loveland) with clever branching runs in $O(1.33^n)$. Current best: $O(1.308^n)$.

Traveling Salesman: Brute force is $O(n!)$. The Held-Karp algorithm uses dynamic programming to solve TSP in $O(2^n \cdot n^2)$ —exponential but dramatically better than factorial.

Meet in the Middle: Split the problem into two halves of size $n/2$, solve each, combine. Reduces $O(2^n)$ to $O(2^{n/2})$ —the "square root" trick for exponential problems.

29.4 4. Heuristics and SAT Solvers

In practice, industrial-strength SAT solvers (like MiniSAT, Z3, Glucose) solve instances with millions of variables that would theoretically be intractable. They combine:

- **DPLL with backtracking**
- **Clause learning** (CDCL—Conflict-Driven Clause Learning): record the reason for each conflict as a new clause, preventing the solver from making the same mistake twice
- **VSIDS heuristic** for variable ordering
- **Random restarts**

These techniques make SAT solvers practical despite NP-completeness. SAT solvers are now used in hardware verification, circuit testing, AI planning, and software model checking.

Chapter 30

Part VI: Special Complexity Results Worth Knowing

30.1 The PCP Theorem

PCP (Probabilistic Checkable Proofs) Theorem (1992): Every NP problem has a proof system where a verifier can check a proof by reading only $O(\log n)$ bits of the proof and making a constant number of queries, with the guarantee that valid proofs always pass and invalid proofs pass with probability at most $1/2$.

Why it matters: The PCP theorem is the technical tool behind most inapproximability results. It shows that approximating many NP-hard problems to within constant factors is itself NP-hard.

The significance is philosophical: Even checking a "proof" of an NP solution can be done probabilistically with minimal reading. This reframes verification as a probabilistic process with locality.

30.2 Unique Games Conjecture (UGC)

Conjectured by Subhash Khot (2002): The problem of distinguishing nearly-satisfiable from highly-unsatisfiable "unique games" instances (a specific constraint satisfaction problem) is NP-hard.

Why it matters: Assuming UGC, almost every known approximation

algorithm for NP-hard problems is optimal—you cannot do better. The 2-approximation for Vertex Cover cannot be improved to $(2-\varepsilon)$. The semidefinite programming relaxations used for Max-Cut and other problems are tight.

The UGC has become the central organizing conjecture of approximation algorithms theory. It is widely believed but unproven.

30.3 Ladner's Theorem (1975)

If $P \neq NP$, there exist problems that are in NP but neither in P nor NP-complete. These are called **NP-intermediate** problems.

Candidate NP-intermediate problems (assuming $P \neq NP$):

- **Integer factoring:** Given n , find a non-trivial factor. No polynomial algorithm known; not known to be NP-complete.
- **Graph isomorphism:** Are two graphs structurally identical? No polynomial algorithm known; not known to be NP-complete.
- **Discrete logarithm:** Given $g^x \bmod p$, find x . The basis of Diffie-Hellman and elliptic curve cryptography.

These problems occupy a strange and important middle ground: believed hard, but possibly in a different class of hardness than NP-complete problems.

30.4 Savitch's Theorem

$NSPACE(f(n)) \subseteq DSPACE(f(n)^2)$. Nondeterministic space can be simulated deterministically with only a quadratic overhead in space. This implies $NSPACE = PSPACE$ —unlike the time case, nondeterminism doesn't help for space beyond a quadratic factor.

30.5 Time and Space Hierarchy Theorems

Time hierarchy theorem: More time means more problems. If $f(n)$ is a "nice" function and $g(n)$ grows faster, then $\text{DTIME}(g(n))$ strictly contains $\text{DTIME}(f(n))$. This proves that the time complexity hierarchy is infinite and non-collapsing—there are always harder problems that need more time.

Space hierarchy theorem: Similarly, $\text{DSPACE}(g(n))$ strictly contains $\text{DSPACE}(f(n))$ when g grows faster. This is how we know $P \neq \text{PSPACE}$ —because $O(\log n)$ space is less than polynomial time, which is less than polynomial space.

30.6 Immerman-Szelepcsényi Theorem (1988)

$\text{NSPACE}(f(n))$ is closed under complement—nondeterministic space classes equal their complements. $\text{NL} = \text{co-NL}$ where NL is nondeterministic logspace. This was a surprise—the analogous result for time ($\text{NP} = \text{co-NP}$) is a major open problem.

Chapter 31

Part VII: Quantum Complexity

Quantum computers manipulate quantum states (superpositions of 0 and 1) and exploit quantum phenomena (superposition, entanglement, interference) to solve certain problems faster.

31.1 BQP—Bounded-Error Quantum Polynomial Time

Definition: The class of problems solvable in polynomial time on a quantum computer with error probability at most $1/3$.

Relationship: $P \subseteq BQP \subseteq PSPACE$. It is widely believed that $BQP \not\subseteq NP$ and $NP \not\subseteq BQP$ —quantum computers are not NP oracle machines.

31.2 Shor’s Algorithm (1994)

Peter Shor’s quantum algorithm for integer factoring runs in $O((\log n)^3)$ —polynomial in the number of digits. On a sufficiently powerful quantum computer, this breaks RSA and discrete-log-based cryptography.

Significance: Factoring is believed to be NP-intermediate (not NP-complete). Shor’s algorithm shows it’s in BQP. This is the clearest example where

quantum computation dramatically outperforms classical computation for a problem of cryptographic importance.

31.3 Grover's Algorithm (1996)

Quantum algorithm for unstructured search. Finds a marked element in a list of n items in $O(\sqrt{n})$ time—a quadratic speedup over classical $O(n)$.

Significance: Proves quantum computers provide some speedup for NP problems. But it's only quadratic, and it doesn't put NP into BQP—solving NP-complete problems by Grover would still be exponential.

31.4 Post-Quantum Cryptography

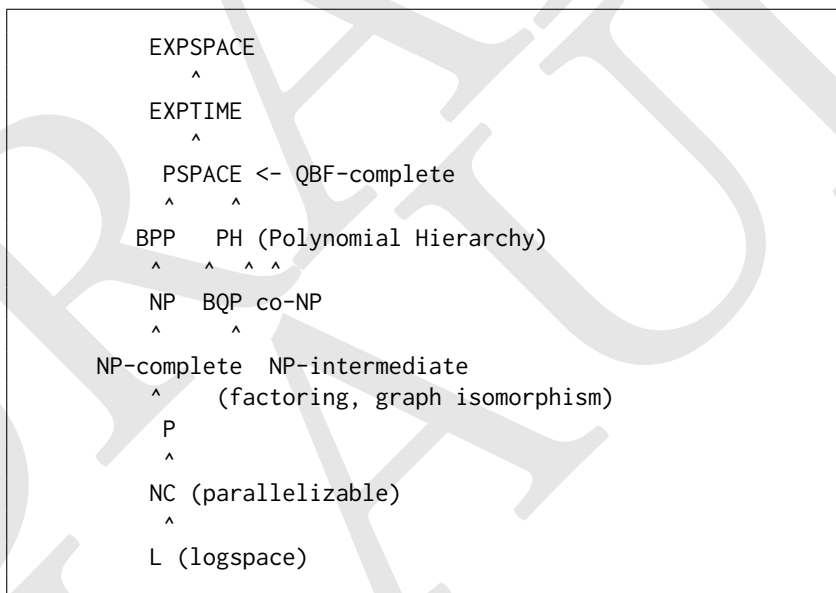
In anticipation of large-scale quantum computers, NIST has standardized post-quantum cryptographic algorithms based on problems believed to be hard even for quantum computers:

- **Lattice-based cryptography** (CRYSTALS-Kyber, CRYSTALS-Dilithium)
- **Hash-based signatures** (SPHINCS+)
- **Code-based cryptography**

These are not believed to be in BQP, though the theory here is less mature than for classical hardness.

Chapter 32

Part VIII: The Complexity Landscape A Map



Known strict separations: $L \subsetneq PSPACE$, $P \subsetneq EXPTIME$. Almost everything else (P vs NP, NP vs co-NP, NP vs PSPACE, BPP vs P) is open.

Chapter 33

Part IX: Why This Matters Beyond Theory

Complexity theory is not just abstract mathematics. Its results shape how we build systems.

Cryptography rests entirely on computational hardness assumptions. RSA relies on factoring being hard (believed NP-intermediate). AES security relies on related-key attacks being hard. If $P = NP$, all of cryptography collapses. The field explicitly assumes $P \neq NP$ and specific hardness properties.

Verification and model checking—proving software and hardware correct—involves problems in PSPACE and EXPTIME. The tools used (SAT solvers, SMT solvers, BDD libraries) are engineering around NP-hardness.

Operations research—supply chains, scheduling, routing, resource allocation—involves NP-hard problems daily. The entire field of integer linear programming is a practical attack on NP-hardness, using branch-and-bound, cutting planes, and LP relaxations.

Machine learning—training neural networks involves non-convex optimization that is in general NP-hard. In practice, gradient descent finds good local minima. The theoretical gap between what is provably hard and what works empirically is one of the deepest open questions in ML theory.

Bioinformatics—genome assembly, protein structure prediction, RNA folding, phylogenetic inference all involve NP-hard problems solved by

heuristics in practice.

Game theory—computing Nash equilibria is PPAD-complete (a complexity class between P and NP). This is why finding optimal strategies in large games remains computationally challenging.

33.1 The Deepest Insight

Complexity theory reveals something profound about the nature of knowledge and computation: **understanding is hard in a precise, mathematical sense**. Verifying a solution is easier than finding one. The universe of problems is stratified by difficulty in ways that resist simple characterization. And the greatest open question in mathematics—P vs NP—is ultimately a question about whether creativity and insight can be mechanized.

The informal argument for $P \neq NP$ is almost philosophical: mathematicians have spent centuries finding proofs, and clever algorithms have not automated mathematics. If $P = NP$, that gap would close. But our collective inability to solve NP-hard problems efficiently despite enormous incentive is itself informal evidence that something deep prevents it.

Whether or not $P = NP$ will ever be resolved, the framework it provides—complexity classes, reductions, hardness, approximation—is the right language for thinking about computational difficulty.

33.2 What Comes Next

With complexity theory (Part 4) complete, the series has covered:

- History and theory (Part 1)
- Algorithm categories and implementations (Part 2)
- Data structures (Part 3)
- Computational complexity (Part 4)

The natural continuation is **Algorithm Design Techniques in Depth**—advanced methods like network flow, linear programming relaxations,

randomized algorithms theory, online algorithms, and the design of approximation algorithms—where complexity theory meets engineering to produce algorithms that are both theoretically grounded and practically powerful.

Next: Advanced Algorithm Design Techniques—Network Flow, Linear Programming, and Beyond

Chapter 34

Algorithms: Advanced Design Techniques

34.0.1 Network Flow, Linear Programming, Online Algorithms, and the Art of Approximation

This is Part 5 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures. Part 4: Complexity Theory.

34.1 What "Advanced" Means Here

Parts 1–4 built the foundation: what algorithms are, how they work, what structures they operate on, and what theoretical limits constrain them. This part moves into the territory where algorithm design becomes genuinely difficult—where problems resist simple greedy or DP solutions, where the gap between theory and practice narrows, and where the most elegant results live.

Four major themes:

1. **Network Flow**—a powerful modeling framework that unifies dozens of seemingly unrelated problems
2. **Linear Programming**—the workhorse of optimization, and the bridge between combinatorial problems and continuous mathematics

3. **Online Algorithms**—algorithms that must make decisions without seeing the future, and the theory of how well they can do
4. **Approximation Algorithm Design**—the systematic craft of building provably near-optimal algorithms for NP-hard problems

Each of these is a field unto itself. This document gives you the core ideas, the key results, and the conceptual tools to go deeper.

Chapter 35

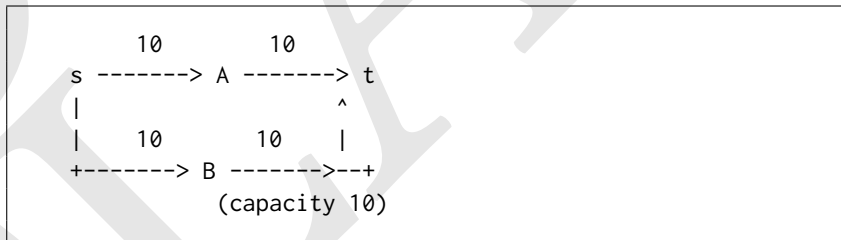
Part I: Network Flow

35.1 The Maximum Flow Problem

A **flow network** is a directed graph where each edge has a capacity—the maximum amount of “flow” it can carry. There is a designated source s (where flow originates) and a sink t (where flow terminates).

Maximum flow problem: Find the maximum amount of flow that can be sent from s to t , subject to:

- **Capacity constraints:** Flow on each edge $\leq$ its capacity.
- **Conservation constraints:** For every node except s and t , flow in = flow out.



In this graph, the max flow is 20—you can send 10 units via $s \rightarrow A \rightarrow t$ and 10 via $s \rightarrow B \rightarrow t$.

35.2 The Max-Flow Min-Cut Theorem

A **cut** is a partition of vertices into two sets S (containing s) and T (containing t). The **capacity of a cut** is the total capacity of edges going from S to T .

Max-flow min-cut theorem: The maximum flow from s to t equals the minimum capacity of any s - t cut.

This is one of the most elegant results in combinatorics. Flow and cuts are dual—the maximum you can push through equals the minimum bottleneck that separates source from sink. The theorem simultaneously gives an algorithm for max flow and a certificate of optimality (finding the min cut proves the flow is maximum).

35.3 Ford-Fulkerson and Augmenting Paths

Ford-Fulkerson algorithm (1956):

```
Initialize all flows to 0
While there exists an augmenting path from  $s$  to  $t$  in the residual graph:
    Find the path
    Push as much flow as the bottleneck edge allows
    Update the residual graph
Return total flow
```

The residual graph is key. For each edge $(u \rightarrow v)$ with capacity c and current flow f :

- Forward residual edge $u \rightarrow v$ with capacity $c - f$ (unused capacity)
- Backward residual edge $v \rightarrow u$ with capacity f (flow that can be "cancelled")

The backward edges are the insight: they allow the algorithm to "undo" previous flow decisions, enabling it to find better routings.

Complexity: $O(E \cdot |\text{max flow}|)$ —the number of augmentations times the

work per augmentation. For integer capacities, this terminates. For irrational capacities, Ford-Fulkerson can actually fail to terminate.

35.4 Edmonds-Karp Algorithm

Use BFS to find augmenting paths (shortest paths in terms of hops). This guarantees at most $O(VE)$ augmentations, giving $O(VE^2)$ total.

Why shortest paths help: BFS augmentation paths have monotonically non-decreasing length, bounding the total number of augmentations.

35.5 Push-Relabel Algorithm

A fundamentally different approach. Instead of augmenting along paths, maintain a "preflow" (conservation may be violated—nodes can have excess flow) and push excess flow toward the sink using height labels.

Complexity: $O(V^2E)$ for basic push-relabel. $O(V^2\sqrt{E})$ with the highest-label variant. Faster in practice than Edmonds-Karp for large networks.

35.6 Applications of Max Flow

The power of max flow comes from **modeling**—translating problems into flow networks.

35.6.1 Bipartite Matching

Problem: Given a bipartite graph (two sets of nodes, edges only between sets), find the maximum matching (maximum set of edges with no shared endpoints).

Reduction to max flow: Add source s connected to all left nodes (capacity 1), all right nodes connected to sink t (capacity 1), all original edges with capacity 1. Max flow = max matching.

Time: $O(E\sqrt{V})$ using Hopcroft-Karp (a specialized BFS-based algorithm for bipartite matching).

Uses: Job assignment. Stable matching (Gale-Shapley algorithm for medical residency matching, Nobel Prize 2012). Network routing. Recommender systems.

35.6.2 Minimum Cut in Graphs

Max-flow min-cut directly gives minimum s-t cuts. Used in image segmentation, network reliability, community detection.

35.6.3 Project Selection / Closure Problem

Problem: Given projects with revenues (possibly negative—costs), and dependencies (project A requires project B), select a subset of projects maximizing profit.

Reduction: Source to each positive-revenue project (capacity = revenue), each negative-revenue project to sink (capacity = |cost|), dependency edges with infinite capacity. Min cut gives optimal selection.

Uses: Open-pit mining (which ore to excavate). Feature selection. Investment planning.

35.6.4 Circulation with Demands

Generalization where edges have both lower bounds (minimum required flow) and upper bounds (capacity). Used in scheduling, traffic engineering, logistics.

35.6.5 Multi-Commodity Flow

Multiple source-sink pairs sharing network capacity. NP-hard in general for integer flows; polynomial for fractional flows (linear programming). Used in traffic routing, network design, telecommunications.

35.7 Matching Beyond Bipartite Graphs

General matching (Edmonds' blossom algorithm, 1965): Maximum matching in non-bipartite graphs. The challenge is "blossoms"—odd-length

cycles that complicate augmenting path search. Edmonds' algorithm handles them with a clever contraction technique.

Complexity: $O(V^3)$ original; $O(E\sqrt{V} \log V)$ with modern implementations.

Weighted matching: Find maximum weight matching. Solved by the Hungarian algorithm (bipartite case, $O(n^3)$) and Galai-Edmonds structure theory (general case).

Applications: Scheduling, pairing problems, graph theory.

Chapter 36

Part II: Linear Programming

36.1 What Is Linear Programming?

Linear programming (LP) solves optimization problems where the objective and all constraints are linear.

Standard form:

Maximize: $c \cdot x$ (dot product---a linear objective)
Subject to: $Ax \leq b$ (linear inequality constraints)
 $x \geq 0$ (non-negativity)

Where x is a vector of n variables, c is the objective coefficient vector, A is an $m \times n$ constraint matrix, and b is the constraint right-hand side.

Geometric interpretation: Each constraint defines a half-space. The feasible region is the intersection—a convex polytope. The objective is a hyperplane we tilt until it barely touches the polytope. The optimal solution is always at a vertex (corner point) of the polytope.

36.2 The Simplex Method

Dantzig's simplex algorithm (1947): Moves from vertex to adjacent vertex of the polytope, always improving the objective, until no improvement is possible.

Complexity: Exponential in the worst case (Klee-Minty examples with 2^n vertices). But in practice, simplex is fast—typically $O(m)$ pivots for m constraints.

Why does it work in practice despite exponential worst case? The polytopes arising from real problems are well-behaved, and randomized pivot rules (Bland's rule, random edge) give good average-case behavior.

36.3 The Ellipsoid Method and Interior Point Methods

Ellipsoid method (Khachiyan, 1979): First polynomial-time LP algorithm. $O(n^4L)$ where L = input bit size. Theoretically important—proved $LP \in P$. Impractical due to large constants.

Interior point methods (Karmarkar, 1984): Move through the *interior* of the polytope rather than along edges. $O(n^{3.5}L)$. Practical for large instances. Modern LP solvers (Gurobi, CPLEX) use interior point methods for large problems and simplex for smaller ones.

36.4 LP Duality

Every LP has a **dual** LP. If the primal is:

Maximize $c \cdot x$ subject to $Ax \leq b, x \geq 0$

The dual is:

Minimize $b \cdot y$ subject to $A^T y \geq c, y \geq 0$

Weak duality: The primal objective value $\leq$ dual objective value for any feasible primal/dual pair. Primal provides lower bounds; dual provides upper bounds.

Strong duality: At optimality, primal = dual. The optimal primal value equals the optimal dual value.

Complementary slackness: A primal-dual pair is optimal if and only if complementary slackness conditions hold—when a primal constraint is slack (not tight), the corresponding dual variable is zero, and vice versa.

Why duality matters:

- Certificates of optimality (dual solution proves primal is optimal)
- Design tool for algorithms (many LP-based approximation algorithms use dual variables to guide choices and prove approximation ratios)
- Economic interpretation (dual variables are shadow prices—the marginal value of relaxing a constraint)
- Connection to max-flow min-cut and other combinatorial duality results

36.5 Integer Linear Programming (ILP)

When variables must be integers, LP becomes ILP—and polynomial becomes NP-hard.

Integrality gap: The LP relaxation (dropping integrality constraints) gives a lower bound. The ratio of ILP optimal to LP optimal is the integrality gap. Small integrality gaps mean the LP relaxation is a useful approximation.

Example: Vertex Cover LP relaxation has integrality gap ≤ 2 . This gives a 2-approximation algorithm: solve the LP, round any variable $\geq 1/2$ up to 1.

36.6 LP in Algorithm Design

LP is not just for optimization—it is a design tool for combinatorial algorithms.

Randomized rounding: Solve the LP relaxation to get fractional solution x . *Round each variable independently (e.g., set $x_i = 1$ with probability x_i).* Analyze the expected cost and feasibility. Used for Set Cover, Facility Location, scheduling.

Primal-dual method: Use complementary slackness as a guide to build a combinatorial algorithm. Start with a feasible dual solution; grow it until primal feasibility is achieved. The algorithm is combinatorial but its correctness and approximation ratio are proved via LP duality.

LP + rounding for Set Cover:

- LP relaxation: minimize $\sum w_i x_i$ subject to for each element e , $\sum_{S \ni e} x_i \geq 1$.
- Round: include set S_i if $x_i \geq 1/f$ where $f = \max$ frequency of any element.
- Achieves f -approximation. For vertex cover ($f=2$), this gives a 2-approximation.

Semidefinite programming (SDP): A generalization of LP where variables are positive semidefinite matrices. More powerful than LP; used for the best known approximation for Max-Cut (Goemans-Williamson, 0.878-approximation using SDP + randomized rounding with hyperplane rounding).

36.7 The Ellipsoid Method in Combinatorics

Even when the LP has exponentially many constraints, the ellipsoid method can solve it in polynomial time as long as there is a polynomial-time **separation oracle**—an algorithm that, given a proposed solution, either confirms it is feasible or finds a violated constraint.

This result unlocks LP-based algorithms for problems where writing down all constraints explicitly is infeasible (e.g., matching polytopes, cut polytopes). It established a deep connection between efficient optimization and efficient separation.

Chapter 37

Part III: Online Algorithms

37.1 The Online Setting

An **online algorithm** must make irrevocable decisions as input arrives, without knowledge of future input. An **offline algorithm** sees the entire input before deciding.

Examples of inherently online problems:

- Paging (which page to evict from cache when a fault occurs)
- Ski rental (rent or buy skis, not knowing how many days you'll ski)
- Online auctions (accept or reject bids in sequence)
- Network packet routing (route each packet as it arrives)
- Job scheduling (assign jobs to machines as they arrive)

The challenge: decisions that seem good locally may look terrible in hindsight.

37.2 Competitive Analysis

Competitive ratio: An online algorithm A has competitive ratio c if for every input sequence σ :

$$A(\sigma) \leq c \cdot \text{OPT}(\sigma) + \text{constant}$$

where $\text{OPT}(\sigma)$ is the cost of the optimal offline algorithm on σ . For maximization problems, $\text{OPT} \leq c \cdot A$.

Interpretation: A c -competitive algorithm is never more than c times worse than the best possible with hindsight. Competitive ratio is the worst-case ratio over all possible inputs.

37.3 The Ski Rental Problem

Problem: You need skis for an unknown number of days. Renting costs $1/\text{day}$; *buying costs* B . On each day, decide rent or buy. You don't know if you'll ski tomorrow.

Deterministic: Rent for $B-1$ days, then buy. If you ski $\geq B$ days, total cost $\leq 2B-1 \leq 2 \cdot \text{OPT}$. If you ski $< B$ days, you rented the whole time, same as OPT . **Competitive ratio:** $2-1/B \approx 2$.

Can we do better? No deterministic algorithm can beat ratio 2 (adversary can always choose the day you buy as the last ski day, or ski forever).

Randomized: Buy on day k with probability $(1-1/B)^{k-1} \cdot (1/B)$ —geometric distribution. Expected competitive ratio: $e/(e-1) \approx 1.58$. This is optimal for randomized algorithms.

Ski rental is a template for the "buy vs. rent" tradeoff that appears in caching, leasing, and investment problems.

37.4 Paging and Caching

Problem: A cache holds k pages. Requests arrive one by one. On a cache miss, a page must be evicted to make room. Minimize cache misses (or faults).

LRU (Least Recently Used): Evict the page not used for the longest time. **Competitive ratio:** k (optimal for deterministic algorithms).

FIFO: Evict the oldest page. Also k -competitive.

Optimal offline (Belady's algorithm): Evict the page whose next use is furthest in the future. This is the optimal offline algorithm, used as the OPT benchmark.

Randomized MARK algorithm: Competitive ratio $O(\log k)$ —exponentially better than deterministic. Optimal for randomized paging.

Lower bound: Any deterministic paging algorithm has competitive ratio $\geq k$. The adversary can always request the one page not in cache.

37.5 The k-Server Problem

A generalization of paging. k servers on a metric space. Requests arrive at points; a server must be moved to each request. Minimize total movement.

k-Server Conjecture: The optimal competitive ratio for k servers on any metric space is k . Proved in 1995 by Koutsoupias and Papadimitriou for some special cases; the full conjecture was proved by Bubeck et al. in 2018 for the randomized version, one of the biggest recent results in online algorithms.

37.6 Secretary Problem

Problem: n candidates interviewed in random order. After each interview, immediately accept or reject. Maximize probability of choosing the best candidate.

Optimal strategy: Reject the first n/e candidates (observe only), then accept the next candidate better than all observed.

Probability of success: $1/e \approx 37\%$. This is optimal.

Generalization: Online matching and auctions. The prophet inequality says: seeing the distribution of n independent random variables, you can achieve expected value at least $1/2$ of the best you'd get knowing all values in advance.

37.7 Online Bipartite Matching

Problem: Left nodes (servers/items) are known. Right nodes (requests/buyers) arrive online. Match each arriving right node to an unmatched left node or reject it. Maximize matching size.

GREEDY: Match each request to any available left node. Competitive ratio $1/2$.

RANKING algorithm (Karp, Vazirani, Vazirani 1990): Assign random ranks to left nodes at start; match each arriving right node to its highest-ranked available neighbor. **Competitive ratio:** $1 - 1/e \approx 0.632$. Proved optimal.

Applications: Online advertising (matching ads to users), ride-sharing (matching drivers to riders), job placement.

37.8 Load Balancing Online

Problem: Jobs arrive one by one. Assign each to a machine immediately. Minimize the makespan (max machine load).

List scheduling (Graham, 1969): Assign each job to the least loaded machine. Competitive ratio: $2 - 1/m$ for m machines.

Offline optimal: $O(n \log n)$ via sorted scheduling. The online vs. offline gap here is well-studied.

37.9 Adversary Models

Online algorithm analysis distinguishes adversary types:

Oblivious adversary: Constructs the input sequence in advance, before seeing the algorithm's random choices. Standard model.

Adaptive online adversary: Sees the algorithm's decisions (but not random choices) and adapts the input accordingly.

Adaptive offline adversary: Sees everything including random choices. Knows the future.

Randomized algorithms only benefit against oblivious adversaries—a randomized algorithm vs. an adaptive offline adversary reduces to the deterministic case.

Chapter 38

Part IV: Approximation Algorithm Design

38.1 The Goal

For NP-hard optimization problems, we seek algorithms that:

1. Run in polynomial time
2. Always produce a feasible solution
3. Come with a provable guarantee that the solution is within factor α of optimal

This is different from heuristics (no guarantee) and from exact algorithms (exponential time).

38.2 Greedy Approximations

38.2.1 Set Cover

Problem: Given a universe U of n elements and sets $S_1 \dots S_m$, find the minimum number of sets whose union is U .

Greedy algorithm: Repeatedly pick the set covering the most uncovered elements.

Analysis: Let $\text{OPT} = k$. After choosing i sets, at most $n(1-1/k)^i$ elements remain uncovered (each set covers at least $1/k$ of remaining). After $k \cdot \ln(n)$ rounds, fewer than 1 element remains uncovered.

Approximation ratio: $O(\log n)$. This is tight—no polynomial algorithm can achieve ratio $(1-\epsilon)\ln(n)$ unless $P = NP$ (by PCP theorem + reduction).

38.2.2 Load Balancing (again)

Sorted-LPT (Longest Processing Time first): Sort jobs by decreasing size, then use list scheduling. Achieves $4/3 - 1/(3m)$ —better than online list scheduling because we can see all jobs first.

38.3 LP Relaxation and Rounding

38.3.1 Weighted Vertex Cover

Problem: Find minimum weight set of vertices covering all edges.

LP relaxation: Minimize $\sum w_i x_i$, subject to $x_i + x_j \geq 1$ for all edges (i,j) , $0 \leq x_i \leq 1$.

Rounding: Set $x_i = 1$ if LP solution has $x_i^* \geq 1/2$.

Why it works: Every edge (i,j) has $x_i + x_j \geq 1$, so at least one of the two is $\geq 1/2$ —the rounding covers every edge. Total weight $\leq 2 \cdot \text{LP optimal} \leq 2 \cdot \text{ILP optimal} = 2 \cdot \text{OPT}$.

Ratio: 2. Tight under UGC—no polynomial algorithm achieves ratio $(2-\epsilon)$ unless UGC fails.

38.3.2 Randomized Rounding for MAX-SAT

Problem: Given a boolean formula, maximize the number of satisfied clauses.

LP relaxation: Variables y_i (truth values), z_j (clause satisfaction). Maximize $\sum z_j$ subject to linear constraints encoding clause satisfaction.

Randomized rounding: Set $x_i = \text{true}$ with probability y_i^* . Each clause of length k is satisfied with probability $\geq 1 - (1-1/k)^k \geq 1 - 1/e$.

Derandomization via method of conditional expectations: Replace random choices with deterministic ones that maintain the probabilistic guarantee.

Combined algorithm (LP + random + simple greedy): Achieves $3/4$ -approximation for MAX-3-SAT.

38.4 Primal-Dual Method for Approximation

A powerful design paradigm that produces combinatorial algorithms with LP-based proofs.

Template:

1. Start with a feasible dual solution (usually all zeros).
2. Raise dual variables uniformly until some constraint becomes tight.
3. Include primal variables corresponding to tight dual constraints.
4. Repeat until the primal solution is feasible.

The approximation ratio comes from bounding how much the primal cost exceeds the dual objective (using complementary slackness conditions).

38.4.1 Primal-Dual for Shortest Path (Dijkstra!)

Dijkstra's algorithm is exactly the primal-dual method applied to shortest path. The distance labels are dual variables; the algorithm greedily raises them.

38.4.2 Primal-Dual for Steiner Tree

Problem: Given a graph with required terminal nodes, find the minimum cost subgraph connecting all terminals.

Goemans-Williamson primal-dual: Simultaneously grow "moats" around unconnected components; add edges when a moat becomes tight. Achieves $2(1 - 1/k)$ -approximation where k = number of terminals.

38.4.3 Primal-Dual for Facility Location

Problem: Open a subset of facilities (each has opening cost) and assign each client to an open facility (assignment cost). Minimize total cost.

JMS algorithm: Raise client dual variables; open facilities when payment covers their cost; assign clients. Achieves 3-approximation. A 1.488 approximation is known using LP + rounding.

38.5 PTAS and FPTAS

PTAS (Polynomial Time Approximation Scheme): For any $\varepsilon > 0$, a polynomial time algorithm achieving $(1+\varepsilon)$ -approximation. The polynomial may depend on $1/\varepsilon$.

Example: Euclidean TSP has a PTAS (Arora's algorithm, 1998): $O(n \log n)$ time for any fixed ε , using a divide-and-conquer on a quadtree decomposition. A deep result showing geometry helps.

FPTAS (Fully Polynomial Time Approximation Scheme): Running time is polynomial in both n and $1/\varepsilon$. Strictly stronger than PTAS.

Example: 0/1 Knapsack has an FPTAS: scale and round item values so they are bounded integers, then apply DP. Running time $O(n^2/\varepsilon)$.

APX-hard: Problems for which no PTAS exists unless $P = NP$ (proved via PCP theorem reductions). MAX-3-SAT and many others are APX-hard—they can be approximated within some constant but no PTAS is possible.

38.6 Semidefinite Programming (SDP) Relaxations

38.6.1 Goemans-Williamson MAX-CUT (1995)

Problem: Partition vertices of a weighted graph into two sets S and $V \setminus S$ to maximize total weight of edges crossing the cut. NP-hard.

SDP relaxation: Associate a unit vector $v_i \in \mathbb{R}^n$ with each vertex. Maximize $(1/2) \sum w_{ij}(1 - v_i \cdot v_j)$. This is a semidefinite program—solvable in polynomial time.

Randomized hyperplane rounding: Choose a random unit vector r ; assign vertex i to S if $v_i \cdot r \geq 0$, else to $V \setminus S$.

Analysis: The probability that edge (i,j) is cut equals $\arccos(v_i \cdot v_j)/\pi$. This gives:

$$E[\text{weight of cut}] \geq 0.878 \cdot \text{OPT(SDP)} \geq 0.878 \cdot \text{OPT}$$

Approximation ratio: 0.878...—the best known for MAX-CUT. Optimal assuming UGC.

This result was transformative. It showed SDP is a strictly more powerful relaxation than LP for combinatorial problems, and established the SDP + randomized rounding paradigm.

38.7 Hardness of Approximation

Proving that certain approximation ratios are impossible (assuming $P \neq NP$ or UGC).

38.7.1 The PCP Theorem's Role

The PCP theorem shows 3-SAT instances can be "gap-amplified": it is NP-hard to distinguish satisfiable instances from instances where at most $(1-\varepsilon)$ fraction of clauses can be satisfied. This gap is the starting point for inapproximability reductions.

Reduction technique (gap-preserving reduction): Transform a gap-3-SAT instance into an instance of your problem, preserving the gap. If a good approximation algorithm existed, it would collapse the gap, solving gap-3-SAT—contradicting NP-hardness.

38.7.2 Key Inapproximability Results

- **Set Cover:** No $(1-\varepsilon)\ln(n)$ -approximation unless $P = NP$.
- **MAX-CLIQUE:** No $n^{1-\varepsilon}$ -approximation unless $P = NP$. (Extreme inapproximability)
- **Chromatic number:** No $n^{1-\varepsilon}$ -approximation unless $ZPP = NP$.
- **Vertex Cover:** No $(2-\varepsilon)$ -approximation unless UGC fails.
- **MAX-3-SAT:** No $(7/8+\varepsilon)$ -approximation unless $P = NP$. ($7/8$ is tight—a simple algorithm achieves it.)

38.8 The Approximation Ratio Landscape

Problems organize themselves by achievable approximation ratio:

Ratio 1 (exact in poly time): Matching, shortest path, MST, LP, 2-SAT, max flow.

Constant ratio:

- Vertex Cover: 2 (tight under UGC)
- TSP with triangle inequality: $3/2$ (Christofides, recently improved)
- Set Cover (fixed universe): $O(\log n)$ is a constant if n is fixed
- MAX-3-SAT: $7/8$

$O(\log n)$:

- Set Cover: $\Theta(\log n)$
- Dominating Set: $O(\log n)$

Polynomial ratio:

- MAX-CLIQUE: $O(n^\varepsilon)$ achievable; $O(n^{1-\varepsilon})$ impossible.

No constant ratio (APX-hard):

- General TSP (without triangle inequality): No constant ratio unless $P = NP$.
- Chromatic number: $\Omega(n^{1-\varepsilon})$ lower bound.

Chapter 39

Part V: Randomized Algorithm Design—Deeper

39.1 The Probabilistic Method

A non-constructive proof technique (due to Erdős): to show an object with certain properties exists, show that a random construction has those properties with positive probability.

Example: A graph on n vertices with no clique or independent set of size $2 \log n$ exists. Proof: a random 2-coloring of edges of K_n has the desired property with probability > 0 .

Derandomization: Convert a probabilistic existence proof into a deterministic algorithm. Techniques: method of conditional expectations, pairwise independence, expander graphs.

39.2 Hashing and Randomness

Universal hashing: A family of hash functions is universal if for any two keys $x \neq y$, $\Pr[h(x) = h(y)] \leq 1/m$ (where m = table size). Guarantees $O(1)$ expected lookup even for adversarial key sets.

k-wise independence: Hash functions where any k keys are independently and uniformly distributed. Enables analysis of algorithms that only need limited independence.

Bloom filters revisited: The false positive rate of a Bloom filter with m bits, n elements, and k hash functions is $(1 - e^{-(kn/m)})^k$. Optimal $k = (m/n) \ln 2$. At $m/n = 10$ bits/element, false positive rate $\approx 0.8\%$.

39.3 Randomized Data Structures

Treaps: Binary search trees where each node has a random priority; maintained as both a BST (by key) and a heap (by priority). Expected $O(\log n)$ height, $O(\log n)$ operations. Simpler to implement than AVL/red-black trees.

Skip lists (revisited): Each element is in level i with probability $1/2^i$. Expected $O(\log n)$ for search, insert, delete. The randomness removes worst-case inputs.

Cuckoo hashing: Two hash tables, two hash functions. Each key can be stored in either of two locations. Insert by placing in a location, evicting the existing key to its alternate location, and repeating (up to $O(\log n)$ steps expected). Achieves $O(1)$ worst-case lookup with high probability.

39.4 Monte Carlo Integration and Sampling

Monte Carlo integration: Estimate $\int f(x)dx$ by sampling random points x and averaging $f(x)$. The error decreases as $O(1/\sqrt{n})$ regardless of dimension—unlike deterministic quadrature rules that suffer the curse of dimensionality.

MCMC (Markov Chain Monte Carlo): Sample from a complex probability distribution by constructing a Markov chain whose stationary distribution is the target. Used in Bayesian statistics, statistical physics, machine learning.

Randomized algorithms for volume computation: Estimating the volume of a convex body in high dimensions is $\#P$ -hard deterministically but has a randomized polynomial-time algorithm using random walks and cooling schedules.

Chapter 40

Part VI: Streaming Algorithms

40.1 The Streaming Model

Data arrives as a sequence of items that can only be read once (or a small number of times). Memory is severely limited—far less than the input size. The goal is to compute approximate answers using $O(\text{polylog } n)$ space.

This models: network traffic monitoring, database query processing on huge datasets, sensor networks.

40.2 Frequency Estimation

Count-Min Sketch: Multiple hash functions map items to counter arrays. To estimate item frequency, return the minimum counter across all arrays. Provides (ϵ, δ) -approximation with $O(1/\epsilon \cdot \log(1/\delta))$ space.

Misra-Gries algorithm: Find all elements with frequency $> n/k$ using $O(k)$ space. Uses a "heavy hitter" detection by maintaining k counters.

40.3 Distinct Element Counting

Flajolet-Martin algorithm (1985): Estimate the number of distinct elements in a stream using hash functions and trailing zeros. A landmark result—the first sublinear space streaming algorithm.

HyperLogLog (2007): The practical algorithm. Uses $O(\log \log n)$ bits per element. Error $\approx 1.04/\sqrt{m}$ where m = registers. Used by Redis, Google, Facebook to count unique visitors, IPs, queries.

40.4 Finding the Median

Finding the exact median requires $\Omega(n)$ space in the streaming model (proven lower bound). Finding an approximate median (within ε rank) requires $O(1/\varepsilon \cdot \log(1/\delta))$ space using the "greenwald-khanna" algorithm or random sampling.

Chapter 41

Part VII: Putting It Together— Advanced Design in Practice

The techniques in this document—flow, LP, online algorithms, approximation—are not isolated. Real algorithm design often combines them.

Network flow + LP: Multi-commodity flow uses LP; integer multi-commodity flow is NP-hard. The LP relaxation guides heuristics and approximations.

Primal-dual + online: The online primal-dual method extends primal-dual design to the online setting, yielding competitive algorithms with LP-certified ratios.

Streaming + sketching + approximation: Heavy hitter detection, frequency estimation, and graph sketching combine streaming with approximation to process data at network speed.

Randomization everywhere: Randomized rounding, random pivot selection, random hash functions, MCMC—randomness is a design tool at every level.

The deepest insight of advanced algorithm design is this: **the right abstraction makes hard problems tractable**. Network flow makes dozens of matching and assignment problems routine. LP makes optimization problems amenable to systematic relaxation and rounding. The probabilistic method makes existence arguments constructive. The key skill is recognizing which framework applies—and that recognition comes from seeing enough problems.

41.1 What Comes Next

Part 5 has covered the four pillars of advanced design:

- Network flow and its remarkable modeling power
- Linear programming as the bridge between combinatorics and continuous optimization
- Online algorithms and competitive analysis
- Approximation algorithm design—greedy, LP rounding, primal-dual, SDP, and inapproximability

Part 6 will shift to **Algorithm Analysis in Depth**—amortized analysis (how to prove data structures are fast on average), probabilistic analysis (average-case complexity), competitive analysis in more depth, smoothed analysis (why simplex works in practice), and the mathematics of recurrences. The goal is to move from "this algorithm seems fast" to "this algorithm is provably fast, and here's the exact argument."

Next: Algorithm Analysis in Depth—Amortized, Probabilistic, Smoothed, and Competitive

Chapter 42

Algorithms: Analysis in Depth

42.0.1 Amortized, Probabilistic, Smoothed, and Competitive Analysis—Proving Algorithms Are Fast

This is Part 6 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures. Part 4: Complexity Theory. Part 5: Advanced Design Techniques.

42.1 Why Analysis Deserves Its Own Document

In Parts 2 and 3, we stated complexities— $O(\log n)$ for binary search, $O(1)$ amortized for dynamic array append, $O(\alpha(n))$ for Union-Find. In Part 4, we proved lower bounds. But we largely took upper bounds on faith, or gave only intuitive arguments.

This document is about rigor. How do you *prove* that an algorithm runs in a certain time? When an operation is sometimes slow and sometimes fast, how do you characterize its long-run cost? When an algorithm performs well on typical inputs but has a bad worst case, what does that mean precisely? When a randomized algorithm is fast "with high probability," what probability, and why?

These are not merely academic questions. Incorrect complexity analysis has caused production systems to fail under load. Misunderstanding amortized vs. worst-case bounds leads to wrong architectural decisions. And the techniques here—aggregate analysis, potential functions, the probabilistic

method, smoothed analysis—are among the most elegant tools in all of computer science.

Chapter 43

Part I: Amortized Analysis

43.1 The Core Idea

Amortized analysis reasons about the *average cost per operation over a sequence of operations*, without making probabilistic assumptions. It gives a worst-case guarantee on the average—not average case in the probabilistic sense, but a bound on the total cost divided by the number of operations.

The key insight: some operations are occasionally expensive, but if expensive operations are rare and make future operations cheap, the average can still be low.

Three main techniques: **aggregate analysis**, **the accounting method**, and **the potential method**.

43.2 Technique 1: Aggregate Analysis

Prove that a sequence of n operations costs $T(n)$ total, so the amortized cost per operation is $T(n)/n$.

43.2.1 Dynamic Array (ArrayList) Appends

A dynamic array doubles its capacity when full. Starting with capacity 1, the capacities are 1, 2, 4, 8, ... when doubling occurs. Copying costs 1, 2, 4, 8, ... elements.

Total cost for n appends:

- n appends: n operations, each $O(1)$ ignoring copies
- Total copying: $1 + 2 + 4 + \dots + n \leq 2n$ (geometric series)

Total cost: $O(n)$. Amortized cost per append: $O(1)$.

The doubling strategy is critical. If we grew by a fixed amount (say, add 10 slots), the total copying cost would be $O(n^2)$ and the amortized cost $O(n)$ —much worse.

43.2.2 Binary Counter Increments

A k -bit binary counter starts at 0. Increment n times. What is the total number of bit flips?

Bit 0 flips every increment: n flips. Bit 1 flips every 2 increments: $n/2$ flips. Bit i flips every 2^i increments: $n/2^i$ flips.

Total flips: $n + n/2 + n/4 + \dots \leq 2n = O(n)$. Amortized cost per increment: $O(1)$.

Even though a single increment can flip k bits (e.g., $0111\dots 1 \rightarrow 1000\dots 0$), this happens rarely enough that the average is constant.

43.3 Technique 2: The Accounting Method

Assign an **amortized cost** $\hat{a}(i)$ to each operation. The actual cost is $a(i)$. The accounting method requires:

For all sequences of n operations:
 $\sum a(i) \leq \sum \hat{a}(i)$

That is, the total actual cost never exceeds the total amortized cost. Operations that cost more than their amortized charge build up "credit" that pays for future operations that cost less than their charge.

43.3.1 Dynamic Array with Accounting

Assign amortized cost 3 to each append (regardless of whether a copy occurs):

- 1 unit pays for the append itself
- 1 unit is saved as credit for future copying of this element
- 1 unit is saved for future copying of an old element that will be displaced

When a doubling occurs on an array of size m , we need to copy m elements. Each of the $m/2$ elements added since the last doubling has saved 2 credits—exactly enough to pay for copying all m elements.

Total amortized cost: $3n$. Actual total cost $\leq 3n = O(n)$. Amortized per operation: $O(1)$. ✓

43.4 Technique 3: The Potential Method

The most powerful and general technique. Define a **potential function** Φ mapping data structure states to non-negative reals. The amortized cost of the i -th operation is:

$$a(i) = \text{actual cost} + \Phi(D_i) - \Phi(D_{i-1})$$

That is: actual cost plus change in potential. The idea: when an operation is expensive, it should *decrease* potential (releasing stored energy). When it's cheap, it may *increase* potential (storing energy for later).

Summing over n operations:

$$\sum a(i) = \sum \text{actual cost} - \Phi(D_n) + \Phi(D_0) \leq \sum a(i) + \Phi(D_0)$$

If $\Phi(D_0) = 0$ and $\Phi(D_n) \geq 0$, then total actual cost $\leq$ total amortized cost.

43.4.1 Dynamic Array with Potential

Define $\Phi = 2 \cdot (\text{number of elements}) - \text{capacity}$.

- After a non-doubling append on an array with m elements and capacity c : Φ changes from $2m - c$ to $2(m+1) - c = \Phi + 2$. Amortized cost = $1 + 2 = 3$.
- After a doubling append ($m = c$): new capacity = $2c = 2m$. New $\Phi = 2(m+1) - 2m = 2$. Old $\Phi = 2m - m = m$. Amortized cost = $(1 + m) + (2 - m) = 3$.

Both cases give amortized cost $3 = O(1)$. ✓

43.4.2 Splay Trees

Splay trees are self-adjusting BSTs. Every access (search, insert, delete) splays the accessed node to the root using rotations. No explicit balance information is maintained.

Claim: Any sequence of m operations on a splay tree with n nodes costs $O((m+n) \log n)$ total— $O(\log n)$ amortized per operation.

Potential function: $\Phi = \sum_v \log(\text{size}(\text{subtree rooted at } v))$

This is the "rank potential." Expensive splays (deep accesses) dramatically reduce the rank potential, and the potential method proves the amortized bound.

Why splay trees matter: They achieve the **working set property**—recently accessed elements are cheap to access again. This matches access patterns in real applications (temporal locality) and gives amortized $O(\log n)$ even for operations that are $\Omega(n)$ in the worst case.

43.5 Amortized Analysis of Union-Find

Recall from Part 3: Union-Find with path compression and union by rank runs in $O(\alpha(n))$ per operation amortized, where α is the inverse Ackermann function.

The potential function for this analysis is complex—it assigns ranks and levels to nodes, tracking how much path compression has occurred. The full proof (due to Tarjan, 1975) is one of the most intricate amortized analyses in computer science.

The key ideas:

- Path compression flattens the tree, making future operations cheaper
- Union by rank ensures trees don't become too tall
- The potential captures the "disorder" in the tree structure
- Each path compression operation decreases potential, paying for its own cost

The result— $O(\alpha(n))$ —is essentially optimal. Union-Find cannot be $O(1)$ per operation (proven lower bound using a cell-probe argument), but $\alpha(n) \leq 4$ for any n that could exist in the universe, so it is functionally constant.

43.6 Lazy Deletion and Amortized Cleanup

Many data structures use **lazy deletion**—mark items as deleted without removing them immediately. Cleanup (physical deletion) happens in bulk when triggered.

Example: Priority queues with decrease-key. Fibonacci heaps use lazy deletion and lazy merging to achieve $O(1)$ amortized insert and decrease-key, with the cleanup work deferred to extract-min ($O(\log n)$ amortized).

The potential method quantifies exactly how much cleanup work gets "charged" to earlier lazy operations.

Chapter 44

Part II: Recurrence Analysis

44.1 Solving Recurrences

Divide-and-conquer algorithms naturally express their running time as recurrences. Solving them precisely is both a practical skill and a theoretical tool.

44.2 Substitution Method

Guess the form, prove by induction.

Example: $T(n) = 2T(n/2) + n$. Guess $T(n) = O(n \log n)$.

Proof: Assume $T(k) \leq ck \log k$ for $k < n$.

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2c(n/2)\log(n/2) + n \\ &= cn(\log n - 1) + n \\ &= cn \log n - cn + n \\ &\leq cn \log n \quad (\text{for } c \geq 1) \end{aligned}$$

Pitfall: Guessing the wrong form leads to circular reasoning. Common trick: subtract a lower-order term to make induction go through.

44.3 Recursion Tree Method

Draw the recursion as a tree. Each node is a subproblem. At each level, compute the total work. Sum over all levels.

Example: $T(n) = 2T(n/2) + n$.

Level 0: n	$= n$
Level 1: $n/2 + n/2$	$= n$
Level 2: $n/4 + n/4 + n/4 + n/4$	$= n$
...	
Level $\log n$: n leaf nodes	$= n$

Total: $n \cdot (\log n + 1) = O(n \log n)$. ✓

Example: $T(n) = T(n/3) + T(2n/3) + n$ (unbalanced splits).

The recursion tree has variable depth. The longest branch has depth $\log_{3/2}(n)$. Work per level is $O(n)$. Total: $O(n \log n)$. The unbalanced split doesn't hurt asymptotically.

44.4 The Master Theorem

For recurrences of the form $T(n) = aT(n/b) + f(n)$:

Let $c^* = \log_b(a)$ (the "critical exponent").

Case 1: $f(n) = O(n^{c^* - \epsilon})$ for some $\epsilon > 0$. The recursion dominates. $\rightarrow T(n) = \Theta(n^{c^*})$

Case 2: $f(n) = \Theta(n^{c^*})$. Work is balanced across levels. $\rightarrow T(n) = \Theta(n^{c^*} \log n)$

Case 3: $f(n) = \Omega(n^{c^* + \epsilon})$ and the regularity condition holds. The top level dominates. $\rightarrow T(n) = \Theta(f(n))$

Examples:

Recurrence	a	b	c*	f(n)	Case	Result
$T(n) = 2T(n/2) + n$	2	2	1	n	2	$O(n \log n)$
$T(n) = 2T(n/2) + 1$	2	2	1	1	1	$O(n)$
$T(n) = 2T(n/2) + n^2$	2	2	1	n^2	3	$O(n^2)$
$T(n) = 4T(n/2) + n$	4	2	2	n	1	$O(n^2)$
$T(n) = T(n/2) + 1$	1	2	0	1	2	$O(\log n)$

44.5 Akra-Bazzi Method

Handles more general recurrences with multiple subproblems of different sizes:

$$T(n) = \sum_i a_i T(b_i n) + f(n)$$

Find p such that $\sum_i a_i b_i^p = 1$. Then:

$$T(n) = \Theta(n^p (1 + \int_1^n f(u)/u^{p+1} du))$$

This handles cases like $T(n) = T(n/3) + T(2n/3) + n$ ($p=1$, gives $O(n \log n)$) that the standard master theorem cannot.

44.6 Linear Recurrences and Fibonacci

Recurrences of the form $T(n) = c_1 T(n-1) + c_2 T(n-2) + \dots$ have solutions via the characteristic polynomial.

Fibonacci: $T(n) = T(n-1) + T(n-2)$. Characteristic roots: $\varphi = (1+\sqrt{5})/2 \approx 1.618$ (golden ratio) and $\psi = (1-\sqrt{5})/2$. Solution: $T(n) = (\varphi^n - \psi^n)/\sqrt{5}$. Fibonacci is $\Theta(\varphi^n)$ —exponential naive recursion, $O(n)$ with memoization, $O(\log n)$ via matrix exponentiation.

Chapter 45

Part III: Probabilistic Analysis

45.1 Average-Case vs. Worst-Case

Worst-case analysis asks: what is the maximum cost over all inputs of size n ? Average-case analysis asks: what is the expected cost over a random input drawn from some distribution?

Average-case analysis is more informative when we have a good model of the input distribution, when adversarial inputs are unlikely in practice, or when the worst case is pathological and rare. It is less informative when inputs are adversarially chosen, when the assumed distribution doesn't match reality, or when guarantees are required.

45.2 Quicksort Average-Case Analysis

Quicksort with a random pivot on a random permutation. The key question: how many comparisons in expectation?

Indicator random variables: Define $X_{ij} = 1$ if element i and element j are compared, 0 otherwise. Total comparisons $X = \sum_i \sum_j X_{ij}$.

Key observation: Elements i and j (where $i < j$ in sorted order) are compared if and only if one of them is chosen as a pivot before any element between them. Since there are $j - i + 1$ elements in the range $i, \dots, j$, the probability that i or j is chosen first is $2/(j - i + 1)$.

$$E[X] = \sum_{i=0}^{n-1} \frac{1}{n-i} = 2(n-1)H_n - 2(n-1) \approx 2n \ln n$$

where $H_n = 1 + 1/2 + 1/3 + \dots + 1/n \approx \ln n$ is the n -th harmonic number.

Expected comparisons: $2n \ln n \approx 1.39n \log_2 n$. Quicksort is $O(n \log n)$ on average.

45.3 Harmonic Numbers and the Coupon Collector

Coupon collector: There are n types of coupons. Each trial gives a random coupon. How many trials until you have all n types?

After collecting k distinct coupons, the expected trials to get the next: $n/(n-k)$. Summing:

$$E[T] = \sum_{k=0}^{n-1} \frac{n}{n-k} = n \cdot H_n \approx n \ln n$$

Expected trials: $n \ln n + O(n)$.

This appears in hash table analysis, randomized algorithms, birthday attacks, DNA sequencing coverage, and web crawling.

45.4 Birthday Paradox and Hash Collisions

Birthday problem: With $n = 365$ days, collisions become likely among $\sim \sqrt{2n \ln 2} \approx 23$ people.

Using $1-x \approx e^{-x}$, the probability of no collision among k people is approximately $e^{-k(k-1)/(2n)}$, which drops below $1/2$ when $k \approx \sqrt{2n \ln 2}$.

In hashing: With a table of size m , collisions become likely after $\sim \sqrt{m}$ insertions. This informs load factor choices and birthday-attack resistance in cryptographic hash functions.

45.5 High-Probability Bounds

Expected value isn't always enough. We often need bounds that hold **with high probability (w.h.p.)**—probability at least $1 - 1/n^c$ for some constant c .

45.5.1 Markov's Inequality

For a non-negative random variable X :

$$\Pr[X \geq t] \leq E[X] / t$$

Weak but requires only the expectation.

45.5.2 Chebyshev's Inequality

$$\Pr[|X - E[X]| \geq t] \leq \text{Var}[X] / t^2$$

Stronger—requires variance. Used when variance is small relative to the mean.

45.5.3 Chernoff Bounds

The most powerful tool for sums of independent 0,1 random variables. If $X = X_1 + \dots + X_n$ where X_i are independent Bernoulli variables with sum of means μ :

Upper tail:

$$\Pr[X \geq (1+\delta)\mu] \leq e^{(-\delta\mu^2/3)} \quad \text{for } 0 < \delta \leq 1$$

Lower tail:

$$\Pr[X \leq (1-\delta)\mu] \leq e^{(-\delta\mu^2/2)} \quad \text{for } 0 < \delta < 1$$

Key feature: Exponentially small tails. This enables "with high probability" arguments.

Example—Load balancing: Throw n balls into n bins randomly. By Chernoff + union bound: the maximum load is $O(\log n / \log \log n)$ w.h.p.

Power of two choices: Each ball hashes to two random bins; place it in the less full one. Max load drops to $\Theta(\log \log n)$ w.h.p.—an exponential improvement from a single extra hash. Used in cloud load balancers, cuckoo hashing, and distributed systems.

45.6 The Union Bound

$$\Pr[A_1 \cup A_2 \cup \dots \cup A_n] \leq \sum \Pr[A_i]$$

Combined with per-event Chernoff bounds, the union bound handles "does any bad event occur?" This is the workhorse of probabilistic algorithm analysis.

Chapter 46

Part IV: Smoothed Analysis

46.1 The Problem with Worst-Case and Average-Case

The simplex method for linear programming has exponential worst-case complexity (Klee-Minty examples) but runs in $O(m^2n)$ in practice—fast enough to solve million-variable problems.

Worst-case analysis says: terrible. Average-case analysis (over random LP instances) says: polynomial. But random LP instances don't represent practice—real inputs come from engineering and economics and are neither random nor adversarial.

Smoothed analysis (Spielman and Teng, 2001, Gödel Prize 2008) bridges the gap.

46.2 Smoothed Complexity

Definition: The smoothed complexity of an algorithm is the maximum, over all inputs, of the expected running time when the input is *slightly perturbed* by random noise.

$$\text{Smoothed complexity} = \max_{\text{input } x} E[\text{runtime}(x + \text{small random noise})]$$

The perturbation model: add Gaussian noise $\sigma \cdot N(0,1)$ to each input coefficient.

Key result: The simplex method has polynomial smoothed complexity $O(n^{2.5}/\sigma)$. For any fixed non-degenerate input distribution ($\sigma > 0$), simplex is fast.

Intuition: Worst-case inputs are "fragile"—they rely on perfect alignment of constraints. Any tiny perturbation destroys the structure that made them hard. In practice, data always has measurement noise or rounding, so worst-case instances essentially never occur.

46.3 Smoothed Analysis of Other Algorithms

k-means clustering: NP-hard in worst case, exponential for some inputs. Smoothed analysis shows polynomial smoothed complexity—explaining why k-means works well in practice.

Local search algorithms: Worst-case exponential, but smoothed polynomial for many problems including TSP local search heuristics.

Beyond smoothed analysis: The field increasingly studies algorithms with more realistic input models—working-set models, access graphs, prediction-augmented algorithms.

Chapter 47

Part V: Competitive Analysis— A Deeper Look

47.1 Potential Functions for Online Algorithms

Just as potential functions prove amortized bounds for offline data structures, they prove competitive ratios for online algorithms.

Online algorithm potential method: Define Φ mapping the current state to a real number. The amortized competitive ratio of each step is:

$$a = \text{actual cost of online} - c \cdot \text{actual cost of OPT} + \Delta\Phi$$

If $\Delta\Phi \leq \text{initial potential}$, then $\text{online cost} \leq c \cdot \text{OPT} + \text{initial potential}$.

47.2 Working-Set Model and Beyond Worst-Case

Worst-case competitive analysis for paging gives ratio k for any deterministic algorithm. But in practice, LRU significantly outperforms FIFO.

Working set model: Programs exhibit **temporal locality**—recently accessed pages are likely to be accessed again. LRU never evicts a page in the current working set (as long as the working set fits in cache). This is why LRU is excellent in practice despite a theoretical competitive ratio of k .

47.3 Algorithms with Predictions (Learning-Augmented Algorithms)

A major new research direction: what if the online algorithm has access to a (possibly incorrect) prediction of future input?

Ski rental with prediction: Given a predicted ski duration p (with error $\eta = |p - \text{actual}|$), an algorithm exists with cost $O(\min(B, p) + \eta)$ —optimal when prediction is correct, gracefully degrades as error grows.

Paging with predictions: Given predicted eviction times for pages, the competitive ratio improves from k to $O(1 + \eta/\text{OPT})$ where η is total prediction error.

This framework—**learning-augmented algorithms**—has become a major research area. It models the practical scenario where machine learning provides imperfect hints to classical algorithms.

The three desiderata:

- **Consistency:** When predictions are perfect, achieve optimal performance.
- **Robustness:** When predictions are garbage, still achieve the best deterministic competitive ratio.
- **Smoothness:** Graceful degradation as prediction error increases.

47.4 Yao's Minimax Principle

Yao's principle (1977): For any randomized online algorithm A and any adversary:

$$\max_{\text{input}} E[\text{cost}_A(\text{input})] \geq \min_{\text{deterministic } B} \max_{\text{distribution}} E[\text{cost}_B(\text{random input})]$$

How to use it: To prove that no randomized algorithm can achieve ratio better than c , exhibit a probability distribution over inputs such that every deterministic algorithm has expected ratio $\geq c$. This lower bound then applies to all randomized algorithms.

Example: For paging with cache size k and $n > k$ pages, a uniform distribution over pages gives the $\Omega(\log k)$ lower bound for randomized paging—proved by designing the right distribution and invoking Yao.

Chapter 48

Part VI: Information-Theoretic Lower Bounds

48.1 Decision Tree Lower Bounds

Any comparison-based algorithm models as a binary decision tree. Lower bound template:

1. Count the number of possible outputs: N
2. The tree must have $\geq N$ leaves
3. Depth $\geq \log_2 N$
4. Therefore $\geq \log_2 N$ comparisons worst case

Sorting: $N = n!$ orderings. Depth $\geq \log_2(n!) = \Omega(n \log n)$. Merge sort and heapsort are optimal.

Finding the minimum: Requires $n-1$ comparisons (every element except the min must lose at least once). Proved by adversary argument: the adversary can always maintain consistency while forcing one more comparison.

Finding both min and max: Can be done in $\lfloor 3n/2 \rfloor - 2$ comparisons—the adversary argument proves this is optimal.

48.2 Communication Complexity

Model: Alice and Bob each hold part of the input. They communicate to compute $f(x, y)$. How many bits in the worst case?

Key result: Equality testing requires $\Omega(n)$ bits deterministically, $O(\log n)$ with randomization (via hashing). This gap has implications for streaming lower bounds.

Disjointness: Do Alice's and Bob's sets intersect? Requires $\Omega(n)$ bits even with randomization. This implies many streaming lower bounds—you need $\Omega(n)$ space to count distinct elements exactly (proving HyperLogLog must be approximate).

48.3 Entropy Lower Bounds

Shannon entropy $H(X) = -\sum p(x) \log_2 p(x)$ measures information content.

Lower bound technique: An algorithm distinguishing among N equally likely cases must read at least $\log_2 N$ bits. More generally, at least $H(X)$ bits where X is the unknown.

Optimal binary search trees: If key k is searched with probability p_k , the expected search depth is minimized by the Huffman tree structure, achieving the entropy lower bound $H(p_1, \dots, p_n)$. This is both a lower bound and a tight algorithm.

Chapter 49

Part VII: Fine-Grained Complexity

49.1 Beyond NP-Hardness

NP-hardness covers problems without polynomial algorithms. But for problems already in P, we want tighter bounds—can we prove $O(n^2)$ is optimal? That matrix multiplication cannot be done significantly faster than $O(n^{2.37})$?

Fine-grained complexity uses conditional lower bounds based on conjectures about specific hard problems.

49.2 The Strong Exponential Time Hypothesis (SETH)

SETH: There is no $O(2^{((1-\varepsilon)n)})$ algorithm for k -SAT for any $\varepsilon > 0$. The brute-force algorithm is essentially optimal.

Consequences of SETH:

- **Edit distance:** Computing the edit distance between two strings of length n requires $\Omega(n^{2-\varepsilon})$ time. The naive $O(n^2)$ DP is essentially optimal.
- **Longest Common Subsequence:** Same $\Omega(n^{2-\varepsilon})$ lower bound.

- **Orthogonal Vectors:** Finding orthogonal vectors among n binary vectors requires $\Omega(n^{2-\epsilon})$ for super-logarithmic dimensions.

These "quadratic lower bounds" explain why decades of effort have failed to find sub-quadratic algorithms for fundamental string problems—under SETH, none exist.

49.3 The 3SUM Conjecture

3SUM: Given n integers, do any three sum to zero? The naive $O(n^2)$ algorithm is believed optimal under the 3SUM conjecture.

3SUM-hard problems: Many computational geometry problems—triangle detection in certain graph families, point containment tests, geom3SUM—are 3SUM-hard, explaining why no sub-quadratic algorithms are known.

49.4 Matrix Multiplication and ω

The matrix multiplication exponent ω is the infimum of all c such that $n \times n$ matrices can be multiplied in $O(n^c)$.

- Naive: $\omega = 3$
- Strassen (1969): $\omega \leq 2.807$
- Current best (Duan et al., 2023): $\omega \leq 2.371552$
- Conjectured: $\omega = 2$

Many problems' complexity is expressed in terms of ω . All-pairs shortest paths, LCS for general alphabets, and graph matching improvements all depend on improving ω .

49.5 APSP Conjecture

APSP conjecture: No $O(n^{3-\epsilon})$ algorithm for All-Pairs Shortest Paths.

Consequences: Diameter, radius, betweenness centrality, minimum weight cycle—all APSP-hard under this conjecture. Fine-grained reductions show these problems are equivalent in difficulty to APSP, forming a cluster of mutually hard problems.

Chapter 50

Part VIII: The Analysis Toolkit— Summary

The full palette of algorithm analysis techniques:

Technique	What It Proves	Key Tool
Big- O / Θ / Ω	Asymptotic complexity	Definition + algebra
Master theorem	Divide-and-conquer recurrences	Pattern matching
Akra-Bazzi	General recurrences	Integration
Aggregate analysis	Amortized cost	Total cost / n
Accounting method	Amortized cost	Credits per operation
Potential method	Amortized cost	Potential function Φ
Indicator variables	Expected cost	Linearity of expectation
Markov's inequality	Tail bounds (weak)	$E[X]$ only
Chebyshev	Tail bounds (medium)	Variance
Chernoff bounds	Tail bounds (strong)	Moment generating function
Union bound	Any-bad-event probability	Sum of individual probabilities
Decision tree	Comparison lower bounds	$\log(\text{number of outputs})$
Adversary argument	Lower bounds	Consistent adversary
Information theory	Lower bounds	Entropy / bits needed
Communication complexity	Algorithm lower bounds	Two-player reduction
SETH / 3SUM / APSP	Fine-grained lower bounds	Conditional polynomial time
Smoothed analysis	Practical complexity	Perturbation + expectation
Yao's minimax	Randomized lower bounds	Min-max theorem

50.1 The Relationship Between Analysis Techniques

These techniques form an interconnected web. Amortized analysis is aggregate analysis when viewed as potential with $\Phi = 0$. Competitive analysis uses potential functions identically to amortized analysis. Chernoff bounds are Markov's inequality applied to the moment generating function e^{tX} . Communication complexity lower bounds imply streaming lower bounds. Fine-grained complexity uses polynomial reductions between problems in P, proving tight polynomial lower bounds rather than separating P from NP.

The unifying theme: **measuring the information content of a problem—how much must be read, communicated, or computed—gives lower bounds. Finding algorithms that match these bounds gives tight complexity.**

50.2 What Comes Next

With six parts complete, the series has built from history through algorithms, data structures, complexity, advanced design, and now analysis. The foundation is thorough.

Part 7 moves to **Systems and Engineering**—where algorithmic theory meets physical reality. Cache-oblivious algorithms, the memory hierarchy, parallel algorithms, distributed algorithms, and the engineering of large-scale systems. The question changes from "what is the asymptotic complexity?" to "how does this actually run on real hardware and at real scale?"

Next: Systems and Engineering—Algorithms in the Real World of Hardware and Scale

Chapter 51

Algorithms: Systems and Engineering

51.0.1 Where Theory Meets Hardware, Scale, and the Messy Real World

This is Part 7 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures. Part 4: Complexity Theory. Part 5: Advanced Design Techniques. Part 6: Analysis in Depth.

51.1 The Gap Between Theory and Practice

The previous six parts have lived largely in the RAM model—an idealized computer where every memory access costs $O(1)$, operations are sequential, and the only resource that matters is the count of primitive steps. This model is clean, mathematically tractable, and surprisingly predictive.

But it is wrong in ways that matter enormously.

A real computer has a hierarchy of memory: registers, L1 cache, L2 cache, L3 cache, RAM, SSD, disk, network—each level 5–1000x slower than the previous. An algorithm that is asymptotically optimal but accesses memory in an unpredictable pattern can be 100x slower in practice than a theoretically inferior algorithm with good locality. A sequential algorithm may leave 95% of a modern CPU’s parallel execution units idle. A single-machine algorithm may be the wrong tool entirely when data doesn’t fit in RAM.

This part is about the gap between algorithmic theory and computational reality. It covers the memory hierarchy and cache-oblivious algorithms, parallel computation, distributed systems, database internals, and the engineering discipline of making algorithms fast in practice. The goal is not to abandon asymptotic analysis—it remains essential—but to layer on top of it the physical constraints that determine real-world performance.

Chapter 52

Part I: The Memory Hierarchy

52.1 Why Memory Hierarchy Exists

Faster memory costs more and consumes more power. The laws of physics impose tradeoffs: SRAM (cache) is fast but expensive; DRAM (RAM) is slower but cheap; SSDs and disks are very cheap but very slow. Modern systems stack these into a hierarchy, betting that programs exhibit **locality**—they access the same data repeatedly and access nearby data together.

Typical latencies (approximate, 2024):

Level	Size	Latency	Bandwidth
L1 cache	32–64 KB	~1 ns / ~4 cycles	~1 TB/s
L2 cache	256 KB – 1 MB	~4 ns / ~12 cycles	~400 GB/s
L3 cache	8–64 MB	~10–40 ns / ~40 cycles	~200 GB/s
RAM (DRAM)	8–512 GB	~60–100 ns / ~200 cycles	~50 GB/s
NVMe SSD	1–8 TB	~100 μ s / ~300,000 cycles	~7 GB/s
Network (LAN)	—	~100 μ s – 1 ms	~10–100 Gb/s
HDD	1–20 TB	~5–10 ms / ~15M cycles	~200 MB/s

The difference between L1 cache and a disk seek is roughly **10 million times**. An algorithm that causes unnecessary disk access doesn't just pay a constant factor penalty—it pays an order-of-magnitude penalty.

52.2 Locality: The Key to Cache Performance

Temporal locality: If a memory location is accessed, it will likely be accessed again soon. Caches exploit this by keeping recently used data.

Spatial locality: If a memory location is accessed, nearby locations will likely be accessed soon. Caches exploit this by loading entire **cache lines** (typically 64 bytes) on each access—even if only one byte was requested.

Implications for algorithm design:

- Sequential access patterns (scanning an array) are excellent—every loaded cache line is fully used.
- Random access patterns (linked list traversal, hash table probing) are poor—each access may load a cache line that is only partially used before being evicted.
- The difference between scanning an array and following pointers in a linked list can be 10–50x in practice, even when the asymptotic complexity is identical.

52.3 Cache-Aware Algorithm Design

Cache-aware algorithms know the cache size and are explicitly designed to fit working sets within it.

52.3.1 Matrix Multiplication—Naive vs. Cache-Friendly

Naive matrix multiplication for $n \times n$ matrices:

```
for i from 0 to n-1:
  for j from 0 to n-1:
    for k from 0 to n-1:
      C[i][j] += A[i][k] * B[k][j]
```

B is accessed column-by-column ($B[k][j]$ for fixed j , varying k)—terrible spatial locality since matrices are stored row-major. Each access to B jumps n elements forward in memory, thrashing the cache.

Tiled (blocked) matrix multiplication:

```

for I in blocks of size B:
    for J in blocks of size B:
        for K in blocks of size B:
            for i in I:
                for j in J:
                    for k in K:
                        C[i][j] += A[i][k] * B[k][j]

```

Choose block size B so that three $B \times B$ blocks fit in L1 or L2 cache. Now each block is loaded once and fully reused. Cache misses drop from $O(n^3)$ to $O(n^3/B)$ —a factor- B improvement.

In practice: Blocked matrix multiplication with $B = 32$ – 64 is typically 5–20x faster than naive, even though both are $\Theta(n^3)$.

52.4 Cache-Oblivious Algorithms

Cache-oblivious algorithms (Frigo, Leiserson, Prokop, Ramachandran, 1999) achieve optimal cache performance for any cache size and line size, *without knowing those parameters*. This is powerful—the same algorithm performs optimally on all hardware.

The cache-oblivious model: Two levels of memory—a fast cache of size M and slow memory. Data moves in blocks of size B . Count the number of block transfers (cache misses).

52.4.1 Cache-Oblivious Matrix Multiplication

Divide each matrix into quadrants and recurse:

```

matmul(A, B, C):
    if matrices are 1x1:
        C[0][0] += A[0][0] * B[0][0]
        return
    partition A, B, C into quadrants
    matmul(A_1_1, B_1_1, C_1_1); matmul(A_1_2, B_2_1, C_1_1)

```

```
matmul(A_1_1, B_1_2, C_1_2); matmul(A_1_2, B_2_2, C_1_2)
matmul(A_2_1, B_1_1, C_2_1); matmul(A_2_2, B_2_1, C_2_1)
matmul(A_2_1, B_1_2, C_2_2); matmul(A_2_2, B_2_2, C_2_2)
```

When the submatrices are small enough to fit in cache (regardless of what size that is), they stay there for the duration of their computation. Cache misses: $O(n^3/(B\sqrt{M}))$ —optimal for any B and M .

52.4.2 Cache-Oblivious Sorting: Funnelsort

Merge sort has poor cache behavior: the merge step accesses two large arrays simultaneously, thrashing cache. Funnelsort (Brodal and Fagerberg, 1999) achieves $O(n/B \cdot \log_M B(n/B))$ cache misses—optimal for any cache.

The key data structure is a **k-merger** (funnel): a binary tree of buffers that lazily merges k sorted sequences. The recursive structure naturally fits subproblems into cache at each level.

Cache-oblivious lower bound: Any comparison-based sorting algorithm requires $\Omega(n/B \cdot \log_M B(n/B))$ cache misses. Funnelsort is optimal.

52.4.3 Cache-Oblivious Search Trees

van Emde Boas layout: Store a binary search tree so that the subtree of height h is always contiguous in memory (regardless of h). The tree is laid out by recursively splitting at the middle level.

For a tree of height h , any root-to-leaf path accesses $O(\log_B h) = O(\log_B \log n)$ cache lines—compared to $O(\log n)$ for a naive BST layout (one node per cache line). This gives $O(\log_B n)$ cache misses per search—optimal.

Used in: BFS-based tree layouts in high-performance databases, spatial indexes.

52.5 External Memory Algorithms

When data doesn't fit in RAM, the bottleneck is disk I/O. The **external memory model** (or I/O model) counts the number of block transfers

between disk and RAM.

Parameters: N = input size, M = RAM size, B = block size (disk sector).

Goal: minimize I/O operations.

52.5.1 External Sorting

Problem: Sort N records that don't fit in RAM.

Algorithm (external merge sort):

1. Load M/B blocks at a time, sort in RAM, write sorted runs to disk. Produces N/M sorted runs.
2. Merge runs in passes. Load one block from each run, merge, write output in blocks.
3. With k -way merge ($k = M/B - 1$), the number of passes is $\log_{M/B}(N/M)$.

Total I/Os: $O((N/B) \cdot \log_{M/B}(N/B))$ —same form as cache-oblivious sorting, now exact.

For context: $N = 1$ TB, $B = 4$ MB, $M = 32$ GB: only 2 merge passes needed. The logarithm base M/B is large.

52.5.2 B-Trees as Optimal External Search Structures

B-trees with branching factor B are optimal for external search: $O(\log_B N)$ I/Os per operation, matching the information-theoretic lower bound. This is why databases universally use B+ trees for disk-resident indexes.

Chapter 53

Part II: Parallel Algorithms

53.1 Why Parallelism Now

For four decades after 1970, single-core performance doubled roughly every 18 months (Moore's Law + Dennard scaling). Around 2005, Dennard scaling broke down—higher clock speeds caused unacceptable heat. The industry pivoted: instead of faster cores, more cores. Modern CPUs have 8–128 cores; GPUs have thousands.

Sequential algorithms leave most of this hardware idle. Exploiting parallelism is no longer optional for performance-critical code.

53.2 Models of Parallel Computation

PRAM (Parallel RAM): p processors sharing a single memory. Operations are synchronous. Variants: EREW (exclusive read/write), CREW (concurrent read), CRCW (concurrent read/write). Theoretical but useful for algorithm design.

Work-span model (the practical one):

- **Work** $W(n)$: total operations across all processors (equivalent to sequential time)
- **Span** (critical path) $S(n)$: time on an unlimited number of processors (the longest chain of dependent operations)

- **Time on p processors:** $T(p) \geq \max(W/p, S)$ —Brent’s theorem
- **Parallelism:** W/S —maximum useful number of processors

Goal: minimize span while keeping work optimal (or near-optimal).

53.3 Parallel Sorting

53.3.1 Parallel Merge Sort

Divide array, sort halves in parallel, merge in parallel.

The merge step is the challenge—naive merging is sequential. **Parallel merge** uses binary search: to merge arrays A and B , for each element of A , binary search in B to find its rank in the merged output. Each element is placed independently.

Work: $O(n \log n)$. **Span:** $O(\log^2 n)$. **Parallelism:** $O(n / \log n)$ —nearly linear.

53.3.2 Bitonic Sort

A sorting network—a fixed pattern of compare-and-swap operations that sorts any input. Fully data-independent: the sequence of comparisons doesn’t depend on the data values.

Work: $O(n \log^2 n)$. **Span:** $O(\log^2 n)$. Excellent for hardware implementation and GPU sorting.

53.3.3 Sample Sort (Parallel Quicksort)

Choose $\sqrt{p}$ splitter elements (by sampling), partition data into p buckets, sort each bucket independently in parallel.

Work: $O(n \log n)$. **Span:** $O(\log^2 n)$. Widely used in practice—MPI parallel sorting, Hadoop, Spark.

53.4 Parallel Graph Algorithms

53.4.1 Parallel BFS

Standard BFS is sequential—each frontier depends on the previous. Parallel BFS processes all frontier nodes simultaneously.

Direction-optimized BFS: When the frontier is large, switch from top-down (push from frontier) to bottom-down (pull from unvisited nodes checking if any neighbor is in frontier). Reduces work from $O(E)$ to $O(V)$ in dense graphs. Used in graph500 benchmark.

Span: $O(D \cdot \log n)$ where D = graph diameter. Good parallelism for small-diameter graphs (social networks, web graph).

53.4.2 Parallel Minimum Spanning Tree

Boruvka's algorithm: Each component finds its cheapest outgoing edge; contract those edges. Each round halves the number of components.

Work: $O(E \log V)$. **Span:** $O(\log^2 V)$. Naturally parallel—each component's step is independent.

Modern parallel MST algorithms achieve $O(\log \log V)$ span with near-linear work.

53.4.3 Parallel Prefix Sum (Scan)

Problem: Given array $[a_1, a_2, \dots, a_n]$, compute $[a_1, a_1+a_2, a_1+a_2+a_3, \dots]$.

Sequential: $O(n)$. Can we parallelize? Yes—with a classic up-sweep / down-sweep tree algorithm.

Work: $O(n)$. **Span:** $O(\log n)$. This is optimal.

Why it matters: Prefix sum is a primitive that enables parallel versions of many algorithms—parallel sorting, parallel graph algorithms, parallel string processing. If you can express an algorithm as a scan, it parallelizes efficiently.

53.4.4 List Ranking and Euler Tour

List ranking: Given a linked list, compute the rank (position) of each node. Naively sequential. Using pointer jumping (each node points to its grandparent), solvable in $O(\log n)$ span.

Euler tour technique: Flattens a tree into a sequence using list ranking. Enables parallel tree algorithms: computing subtree sizes, depths, LCA, centroid decomposition—all in $O(\log n)$ span.

53.5 GPU Computing

GPUs have thousands of cores optimized for throughput (many simple parallel operations) rather than latency (fast sequential operations). NVIDIA's A100 has 6912 CUDA cores and ~312 TFLOPS of FP16 performance.

SIMD (Single Instruction, Multiple Data): All cores in a warp execute the same instruction simultaneously on different data. Divergent branches (if statements with different outcomes per thread) kill performance—warps serialize.

Memory bandwidth is the bottleneck: GPU memory bandwidth (~2 TB/s for A100) is much higher than CPU RAM bandwidth (~50 GB/s), but arithmetic throughput is even higher. Most GPU algorithms are memory-bound, not compute-bound.

Key GPU algorithm patterns:

- **Reduction:** Sum, max, min across millions of elements. $O(\log n)$ parallel steps.
- **Prefix scan:** Used in GPU sorting (radix sort is the dominant GPU sorting algorithm).
- **Stencil computation:** Each output element depends on its neighbors. Matrix multiply, convolution, PDE solving.
- **Gather/scatter:** Irregular memory access—the performance killer. Algorithmic design aims to replace gather/scatter with sequential or strided access.

Deep learning on GPUs: Matrix multiplication (the dominant operation in neural network training) maps perfectly to GPU parallelism. cuBLAS and cuDNN are hand-optimized libraries that achieve near-peak GPU performance for these operations. The entire deep learning revolution was enabled by realizing that GPU matrix multiply is exactly the operation neural networks need.

53.6 Concurrent Data Structures

Parallel algorithms need data structures safe for concurrent access.

Locks (mutex): Serialize access. Correct but can create bottlenecks. Lock granularity is critical: too coarse wastes parallelism, too fine creates overhead.

Lock-free structures: Use atomic compare-and-swap (CAS) operations. No thread ever blocks—if a CAS fails, retry. Examples: lock-free stack, Michael-Scott queue.

Wait-free structures: Every operation completes in a bounded number of steps regardless of other threads. Stronger than lock-free. Harder to implement.

Transactional memory: Mark code regions as transactions; hardware or software detects conflicts and rolls back. Simplifies concurrent programming at a performance cost.

Concurrent BSTs: Lock-free concurrent red-black trees and skip lists exist but are extremely complex to implement correctly. In practice, many systems use coarse-grained locking or copy-on-write structures.

Chapter 54

Part III: Distributed Algorithms

54.1 The Distributed Setting

A distributed system consists of multiple machines communicating over a network. Unlike parallel computing (shared memory, fast communication), distributed systems have:

- **High latency:** Network round-trips take milliseconds
- **Partial failures:** Individual machines can crash or become unreachable
- **No shared state:** Each machine sees only its local data
- **Unreliable communication:** Messages can be delayed, lost, or reordered
- **No global clock:** Machines don't share a synchronized clock

These constraints fundamentally change what algorithms can do.

54.2 The CAP Theorem

CAP theorem (Brewer, 2000; proved by Gilbert and Lynch, 2002): A distributed system can guarantee at most two of three properties:

- **Consistency (C):** Every read receives the most recent write (or an error)

- **Availability (A):** Every request receives a response (not an error)
- **Partition tolerance (P):** The system continues operating when network partitions occur (some machines can't reach others)

Since network partitions *will* occur in any real distributed system, the practical choice is between **CP** (consistent but possibly unavailable during partitions) and **AP** (available but possibly returning stale data during partitions).

CP systems: HBase, Zookeeper, etcd, Consul. Used for coordination, configuration, distributed locking.

AP systems: Cassandra, DynamoDB (in some modes), CouchDB. Used for high-availability user-facing data.

PACELC extension: Even without partitions, there's a tradeoff between latency and consistency. PACELC captures the full spectrum.

54.3 Consensus—The Core Problem

Consensus: Multiple nodes must agree on a single value, even in the presence of failures.

This is the fundamental problem of distributed computing. Leader election, distributed transactions, replicated state machines—all reduce to consensus.

54.3.1 FLP Impossibility (Fischer, Lynch, Paterson, 1985)

Result: In an asynchronous distributed system where even one process may fail (crash), no deterministic algorithm can solve consensus.

Why: In an asynchronous system, you cannot distinguish a crashed process from a very slow one. Any algorithm that terminates must eventually "give up" waiting—but that creates a window where a slow process appears crashed, leading to incorrect decisions.

Practical response: Relax the model. Use timeouts (making the system partially synchronous), randomization (circumventing the determinism

requirement), or accept that consensus is possible only when fewer than a threshold of nodes fail simultaneously.

54.3.2 Paxos (Lamport, 1989/1998)

The classic consensus algorithm. Ensures that a value is chosen only if a majority of nodes agree.

Three roles: Proposers (suggest values), acceptors (vote on proposals), learners (learn the chosen value).

Two phases:

1. **Prepare:** Proposer sends a prepare request with a proposal number n . Acceptors promise not to accept proposals with lower numbers and report the highest-numbered proposal they've already accepted.
2. **Accept:** If a majority responds, proposer sends an accept request. If a value was already accepted by any acceptor, use that value; otherwise use the proposer's own value.

A value is chosen when a majority of acceptors accept a proposal with the same value.

Multi-Paxos extends this for a sequence of decisions (a replicated log) by establishing a stable leader. This is the basis for Chubby (Google), Zookeeper, and etcd.

The critique: Paxos is notoriously hard to understand and implement correctly. Lamport's original paper sat unpublished for nine years because reviewers found it confusing.

54.3.3 Raft (Ongaro and Ousterhout, 2014)

Designed explicitly for understandability. Equivalent to Paxos in fault tolerance, but structured to be easier to reason about.

Key ideas:

- Strong leader: all writes go through the leader
- Leader election via randomized timeouts

- Log replication: leader replicates its log to followers; entry committed when majority acknowledges

Used in: etcd (the coordination layer for Kubernetes), CockroachDB, TiKV, Consul.

54.3.4 Byzantine Fault Tolerance (BFT)

Paxos and Raft handle *crash failures* (nodes stop responding). **Byzantine faults** are worse: nodes can behave arbitrarily—sending incorrect or conflicting messages, acting maliciously.

Byzantine Generals Problem (Lamport, Shostak, Pease, 1982): Generals must agree on an attack/retreat plan despite traitors. Proved: consensus requires at least $3f+1$ nodes to tolerate f Byzantine faults. Majority is not enough—you need a supermajority.

PBFT (Practical Byzantine Fault Tolerance, Castro and Liskov, 1999): First practical BFT protocol. $O(n^2)$ message complexity per consensus round.

Applications: Blockchain consensus (Bitcoin’s proof-of-work, Ethereum’s proof-of-stake, Tendermint, HotStuff are all BFT variants), safety-critical systems.

54.4 Distributed Data Structures

54.4.1 Consistent Hashing

From Part 5: distributes keys across servers so that only K/n keys must remapped when a server is added/removed. The standard approach for distributed caches (Memcached, Redis Cluster) and distributed databases (Cassandra, DynamoDB).

Virtual nodes: Each physical server is represented by multiple virtual nodes on the hash ring. This improves load balance when servers have heterogeneous capacity.

54.4.2 Distributed Hash Tables (DHTs)

A fully decentralized key-value store where no single node has the complete mapping.

Chord (Stoica et al., 2001): Each node is responsible for keys between itself and its predecessor. Routing tables (finger tables) allow $O(\log n)$ hop lookups in a ring of n nodes.

Kademlia: DHT using XOR metric for distance. Used in BitTorrent, Ethereum, IPFS. $O(\log n)$ lookups with parallelism.

54.4.3 CRDTs—Conflict-Free Replicated Data Types

Problem: In an AP distributed system, multiple replicas accept writes concurrently. When they synchronize, conflicts arise.

CRDT: A data type with merge operations that are commutative, associative, and idempotent. Any two replicas can be merged in any order and the result is the same.

Examples:

- **G-Counter:** Grow-only counter. Each node maintains its own count; total = sum. Merge = max per node.
- **LWW-Register:** Last-Write-Wins register using timestamps.
- **OR-Set:** Observed-Remove Set. Add operations are tagged with unique IDs; remove operations remove specific tagged additions.

Used in: Redis (replication), Riak (key-value store), collaborative editing (Google Docs uses OT, a related technique; some newer systems use CRDTs).

54.5 MapReduce and Large-Scale Data Processing

MapReduce (Dean and Ghemawat, Google, 2004): A programming model for processing large datasets on commodity clusters.

Map phase: Apply a function to each input record, emitting (key, value) pairs. **Shuffle phase:** Group all values by key, distributing to workers. **Reduce phase:** For each key, apply a reducing function to all values.

Key insight: This simple model covers a huge fraction of large-scale data processing needs, and the framework handles all the distributed machinery (fault tolerance, data distribution, progress monitoring) automatically.

Fault tolerance: If a worker fails, re-run its tasks. This is possible because Map and Reduce are deterministic and side-effect-free.

Hadoop: Open-source MapReduce implementation. Dominated large-scale data processing 2008–2014.

Spark: Extends MapReduce with in-memory computation and richer operations (DAG execution, iterative algorithms, streaming). Replaced Hadoop for most use cases.

Modern successors: Google’s Dataflow (Millwheel), Apache Flink, Apache Beam—support streaming as well as batch, with more flexible execution models.

54.6 Distributed Databases and Transactions

54.6.1 ACID vs. BASE

ACID (Atomicity, Consistency, Isolation, Durability): Traditional relational database guarantees. Hard to achieve across multiple machines.

BASE (Basically Available, Soft state, Eventually consistent): The alternative for highly distributed systems. Accept temporary inconsistency in exchange for availability and performance.

54.6.2 Two-Phase Commit (2PC)

The classic protocol for distributed transactions.

Phase 1 (Prepare): Coordinator asks all participants if they can commit. Participants lock their resources and respond yes/no. **Phase 2 (Com-**

mit/Abort): If all said yes, coordinator sends commit. Otherwise sends abort.

Problem: If the coordinator crashes after Phase 1, participants are stuck holding locks indefinitely—the **blocking problem**. 2PC is not fault-tolerant by itself.

54.6.3 Three-Phase Commit (3PC)

Adds a pre-commit phase to avoid the blocking problem. Theoretically solves it but introduces other complications. Not widely used in practice.

54.6.4 Distributed Transactions with Paxos / Raft

Modern distributed databases (Google Spanner, CockroachDB, TiDB) implement distributed transactions by combining 2PC with Paxos/Raft for each participant's commitment decision. Each participant is a Paxos group, not a single machine—eliminating the single-point-of-failure problem.

54.6.5 Google Spanner: TrueTime

Spanner uses GPS and atomic clocks to provide **TrueTime**—a globally synchronized clock with bounded uncertainty ($\pm 7\text{ms}$). This enables **external consistency** (a stronger form of consistency equivalent to linearizability) without expensive coordination for read operations.

The insight: if you can bound how far clocks can diverge, you can use timestamps to order events without requiring communication—as long as you wait out the uncertainty window before committing.

54.7 The Logical Clock Perspective

Without synchronized clocks, distributed systems use **logical clocks** to impose ordering.

Lamport timestamps (1978): Each event has a timestamp. The rule: if event A causes event B (A "happens before" B), then $\text{timestamp}(A) <$

timestamp(B). Implemented by incrementing a counter on each event and taking the max on message receipt.

Vector clocks: Each node maintains a vector of counters, one per node. Captures causality precisely: two events are concurrent if neither's vector dominates the other's. Used in Dynamo, Riak, and version control systems.

Version vectors: Used in distributed storage to detect and resolve conflicts. Each replica's version is a vector; you can tell which versions are ancestors, descendants, or concurrent.

Chapter 55

Part IV: Database Internals

55.1 The Storage Layer

55.1.1 Row vs. Column Storage

Row-oriented storage (OLTP): Records stored together. Fast for writes and single-record reads. Used in PostgreSQL, MySQL, Oracle. Good for transactional workloads.

Column-oriented storage (OLAP): All values for one column stored together. Fast for analytical queries that aggregate one column over millions of rows. Excellent compression (similar values together compress well). Used in Redshift, BigQuery, Snowflake, ClickHouse, Parquet files.

Why column storage is faster for analytics: A query like "average revenue by country" only needs two columns out of 50. Row storage reads all 50 columns; column storage reads only 2—a 25x reduction in I/O.

55.1.2 LSM Trees (Log-Structured Merge Trees)

B+ trees are read-optimized. For write-heavy workloads, **LSM trees** are better.

Core idea:

1. Write to an in-memory sorted structure (memtable)
2. When memtable is full, flush to disk as an immutable sorted file (SSTable)

3. Background compaction merges SSTables, removing duplicates and deleted records

Reads: Must check memtable and all SSTables. Bloom filters help—each SSTable has a bloom filter so keys known not to be there are skipped quickly.

Write amplification: Compaction rewrites data multiple times. Read amplification: may need to check multiple SSTables. There is a fundamental tradeoff between write and read amplification.

Used in: LevelDB, RocksDB (used by Facebook, LinkedIn, MySQL, CockroachDB, TiKV), Cassandra, HBase, ScyllaDB.

55.1.3 Write-Ahead Logging (WAL)

Before any data page is modified, the change is recorded in a sequential log. On crash, replay the log from the last checkpoint to recover.

Sequential writes to the WAL are fast (no random I/O). This is why databases can be both durable (log is persisted) and fast (actual data pages are modified lazily).

55.2 Query Execution

55.2.1 The Query Planner

SQL queries are high-level declarations: "give me all customers in Stockholm who bought X." The query planner transforms this into an execution plan—a tree of physical operators.

Logical plan: Relational algebra (select, project, join, aggregate). **Physical plan:** Specific algorithms for each operation (hash join vs. merge join vs. nested loop join, index scan vs. table scan).

Cost estimation: The planner estimates costs using statistics (cardinality estimates—how many rows each operation produces). Wrong estimates lead to catastrophically bad query plans. PostgreSQL's planner uses histograms, correlation statistics, and sampling.

55.2.2 Join Algorithms

Nested loop join: For each row in table A, scan table B. $O(|A| \cdot |B|)$. Only acceptable for small inner tables.

Hash join: Build a hash table on the smaller table, probe with the larger. $O(|A| + |B|)$ expected. Best for large equi-joins when the smaller table fits in memory.

Sort-merge join: Sort both tables on the join key, merge. $O((|A| + |B|) \log(|A| + |B|))$. Good when both tables are large, data is already sorted, or a sort is needed anyway.

Grace hash join (external memory): When the hash table doesn't fit in memory, partition both tables by hash, then hash-join each partition pair. $O((|A| + |B|)/B \cdot \log_{M/B}((|A| + |B|)/M))$ I/Os—optimal for external memory joins.

55.2.3 Query Compilation

Modern databases (HyPer, DuckDB, Umbra, LLVM-backed systems) compile SQL queries to machine code at runtime instead of interpreting them. This eliminates interpreter overhead and enables SIMD vectorization of query operators.

Vectorized execution (MonetDB, DuckDB): Process data in columns of fixed-size batches (1024 rows), applying SIMD instructions across the batch. 4–8x faster than row-at-a-time execution for analytical queries.

55.3 Indexes Beyond B+ Trees

Hash indexes: $O(1)$ equality lookup, no range queries. Used in many NoSQL stores.

Bitmap indexes: For low-cardinality columns (e.g., gender, country). Each distinct value has a bit vector. AND/OR operations efficiently answer compound queries. Excellent for OLAP, bad for high-cardinality columns.

Inverted indexes: Map terms to document lists. The core data structure of

search engines. Posting lists are compressed (delta encoding + variable-byte encoding or PForDelta) for space efficiency and I/O speed.

Spatial indexes: R-trees (Part 3) for 2D/3D bounding box queries. PostGIS, spatial databases.

Full-text search indexes: Trie + inverted index + BM25 ranking. Elastic-search, Lucene, Solr.

Chapter 56

Part V: Compiler Algorithms

56.1 Why Compilers Matter Here

Compilers are the translators between the high-level abstractions in which algorithms are expressed and the low-level machine instructions that execute them. Many of the most interesting algorithmic techniques in systems appear inside compilers.

56.2 Lexing and Parsing

Lexing (tokenization): Convert source characters into tokens. Implemented as a deterministic finite automaton (DFA), typically generated from regular expressions by the Thompson construction (NFA from regex) followed by subset construction (NFA to DFA). $O(n)$ time.

Parsing: Convert token stream into a parse tree. Two main classes:

LL parsers (top-down): Recursive descent. Build the parse tree from the root down, predicting which production rule to apply based on lookahead. LL(k) parsers use k tokens of lookahead. Used in hand-written parsers (GCC, Clang's C frontend).

LR parsers (bottom-up): Shift-reduce. Push tokens onto a stack, reduce by grammar rules when a rule's right-hand side appears on the stack. LALR(1) parsers (Look-Ahead LR with 1 token) can parse most programming language grammars. Generated by tools like yacc/bison.

Earley parser: Handles any context-free grammar in $O(n^3)$ worst case, $O(n^2)$ for unambiguous grammars, $O(n)$ for most practical grammars. Used in NLP parsing.

56.3 Intermediate Representation and SSA

Modern compilers transform source code into an **Intermediate Representation (IR)**—a lower-level representation suitable for optimization. LLVM IR and GCC's GIMPLE are well-known examples.

Static Single Assignment (SSA): Each variable is assigned exactly once. Phi (φ) functions at control flow merge points select between values from different branches.

```
# Normal form:
x = 1
if cond: x = 2
use(x)

# SSA form:
x_1 = 1
if cond: x_2 = 2
x_3 = phi(x_1, x_2)  <- phi function
use(x_3)
```

SSA simplifies most compiler analyses: use-def chains are explicit, data flow is clear, and many optimizations become straightforward.

56.4 Key Compiler Optimizations

Constant folding: Replace compile-time-constant expressions with their values. $2 * 3 + 1$? 7.

Dead code elimination: Remove code whose results are never used. Straightforward in SSA—variables with no uses can be removed.

Common subexpression elimination (CSE): Compute repeated expressions only once. $a*b + a*b$? $t = a*b$; $t + t$.

Loop invariant code motion: Move computations that don't change across iterations outside the loop.

Inlining: Replace function calls with the function body. Enables other optimizations and eliminates call overhead.

Vectorization (auto-vectorization): Replace scalar loops with SIMD instructions. The compiler analyzes loop data dependencies and replaces scalar operations with vector operations when safe.

Register allocation: Map program variables to physical CPU registers. Modeled as graph coloring—build an interference graph (two variables interfere if both are live at the same time), color it with k colors (k = number of registers). Graph k -coloring is NP-hard in general, but practical approximations (Chaitin's algorithm, linear scan) work well.

56.5 Dataflow Analysis

Compiler optimizations require reasoning about program properties at each point. **Dataflow analysis** computes these properties by solving fixpoint equations on the control flow graph.

Reaching definitions: At each program point, which assignments might have reached here without being overwritten? Used for CSE, constant propagation.

Live variable analysis: At each point, which variables might be used in the future? Used for register allocation, dead code elimination.

Available expressions: Which expressions have been computed and not invalidated? Used for CSE.

General framework: A dataflow problem is a lattice of facts + transfer functions per statement + a meet operator at control flow joins. Fixed-point iteration (repeatedly apply transfer functions until no change) converges because the lattice has finite height.

Chapter 57

Part VI: Making Algorithms Fast in Practice

57.1 The Optimization Hierarchy

When optimizing real code, improvements come from multiple levels, in order of leverage:

1. **Algorithm and data structure choice**— $O(n \log n)$ vs $O(n^2)$ dominates everything else for large n .
2. **Avoid unnecessary work**—caching, memoization, early termination, lazy evaluation.
3. **Memory access patterns**—cache locality, prefetching, avoiding pointer chasing.
4. **Instruction-level parallelism**—SIMD, loop unrolling, branch elimination.
5. **Compiler optimizations**—inlining, vectorization, O2/O3 flags.
6. **Hardware-specific tuning**—cache line alignment, NUMA awareness, huge pages.

Too often, engineers jump to level 5 or 6 without addressing levels 1–3. A well-designed algorithm with good cache behavior will outperform a poorly-designed algorithm at maximum compiler optimization.

57.2 Benchmarking and Profiling

Profiling identifies where a program spends its time. Tools: gprof, perf (Linux), Instruments (macOS), VTune (Intel), py-spy (Python).

Common profiling insights:

- 90% of time is often in 10% of the code (Pareto principle)
- Cache misses are invisible in algorithmic analysis but dominate real profiles
- Branch mispredictions cost ~15 cycles each—worth eliminating hot branches
- Function call overhead matters in tight loops

Benchmarking pitfalls:

- **JIT warmup:** Languages like Java and Python JIT warm up after initial runs—benchmark the steady state.
- **Branch prediction warmup:** First runs are slower as the branch predictor learns.
- **Memory effects:** Benchmarks that fit in cache may not represent production where working sets are larger.
- **Measurement overhead:** Time measurement itself takes nanoseconds—for very fast operations, the measurement dominates.

57.3 SIMD and Bit Manipulation

SIMD (Single Instruction, Multiple Data): Modern CPUs can perform the same operation on 4, 8, or 16 values simultaneously using 128/256/512-bit registers (SSE, AVX, AVX-512).

Effective SIMD patterns:

- Sum of arrays: 8x float addition in one instruction
- String search: Compare 32 characters simultaneously

- **Sorting networks:** SIMD-friendly comparison networks sort 8 elements in ~10 instructions
- **Bitmap operations:** Count set bits, find first set bit, AND/OR 64 values simultaneously

Popcount (counting set bits): A single hardware instruction on x86. Used in: Hamming distance, Bloom filters, cardinality estimation, chess engines (bitboard representation of piece positions).

Bit tricks: Many algorithms have bit-manipulation fast paths. Finding the lowest set bit, isolating or clearing bits, checking power of two ($n \& (n-1) == 0$), etc. Brian Kernighan's bit-counting trick, De Bruijn sequences for bit scanning.

57.4 Memory Allocators

The standard `malloc/free` is a general-purpose allocator. For performance-critical code, specialized allocators can be dramatically faster.

Arena allocator (bump allocator): Allocate by incrementing a pointer. Free all at once by resetting the pointer. $O(1)$ allocation, $O(1)$ free. Used in compilers, query engines, game frames.

Pool allocator: Maintain a free list of fixed-size objects. $O(1)$ allocation and free. Excellent for data structures with many same-sized nodes (linked lists, tree nodes).

Slab allocator: Linux kernel allocator. Caches objects of each type; recently freed objects are immediately reused, keeping them warm in cache.

tcmalloc (Google) and jemalloc (Facebook): High-performance general allocators using per-thread caches to eliminate contention. Dramatically faster than glibc malloc in multithreaded applications.

57.5 String Processing Optimizations

Strings are everywhere. Optimized string algorithms matter enormously in practice.

Short string optimization (SSO): Store strings shorter than 15–22 characters inline in the string object (no heap allocation). Used in C++ `std::string`, Rust’s `SmallString`, many language runtimes.

String interning: Store each unique string once; represent strings as integer IDs. $O(1)$ equality comparison, reduced memory. Used in compilers (symbol tables), Python (identifier interning), JVM.

SIMD string search: Compare 32 characters simultaneously using AVX2. Searching for a byte in a string at ~32 GB/s—approaching memory bandwidth limits.

57.6 Real-World Case Studies

`std::sort` in C++: Uses introsort—quicksort with heapsort fallback when recursion depth exceeds $2 \log n$ (avoiding quicksort’s $O(n^2)$ worst case) and insertion sort for small subarrays. Carefully tuned to achieve both theoretical guarantees and practical performance.

Python’s Timsort: Detects and exploits existing runs in the data (natural mergesort). A sorted or reverse-sorted array is handled in $O(n)$. Used by every Python program that sorts anything.

Google’s B-Tree implementation: Open-sourced as `absl::btree_map`. Uses cache-line-aligned nodes with 4–16 children (vs. 2 for a standard BST). Dramatically better cache performance for in-memory ordered maps.

RocksDB’s compaction: Tiered compaction (level-based) vs. size-tiered compaction—different tradeoffs between write amplification, read amplification, and space amplification. Tuned per workload.

LLVM’s register allocator: Uses a linear scan algorithm ($O(n)$ vs. $O(n^3)$ for optimal graph coloring) with special handling of split intervals and spill code generation. Produces near-optimal register allocation fast enough to not dominate compile time.

Chapter 58

Part VII: The Systems Perspective—Pulling It Together

58.1 The Stack of Abstractions

A running program sits at the intersection of multiple algorithmic layers, each with its own design choices:

```
Application algorithm (merge sort, Dijkstra, neural network training)
      v
Data structure layer (arrays, B+ trees, hash tables)
      v
Memory allocator (tcmalloc, arena allocator)
      v
Operating system (virtual memory, page cache, scheduler)
      v
Hardware (CPU pipeline, cache hierarchy, DRAM, SSD)
```

Each layer makes assumptions about the ones below and provides guarantees to the ones above. Performance problems can occur at any layer, and a problem at a lower layer can make even a perfectly designed upper-layer algorithm perform poorly.

58.2 The Lessons

Asymptotic complexity is necessary but not sufficient. An $O(n \log n)$ algorithm with poor cache behavior can be slower than an $O(n^2)$ algorithm with excellent cache behavior for n up to millions.

The memory hierarchy dominates at scale. Cache misses, page faults, and network round-trips dwarf arithmetic cost for most real workloads.

Parallelism requires algorithmic rethinking. Not every algorithm parallelizes well. The span of an algorithm (critical path) is as important as its total work.

Distribution changes the problem. When data spans machines, consistency, fault tolerance, and latency become first-class algorithmic concerns—not afterthoughts.

Measure before optimizing. Profile-guided optimization focuses effort where it matters. Intuitions about bottlenecks are wrong more often than right.

The best systems codesign algorithm and implementation. The fastest systems—Google’s Spanner, RocksDB, DuckDB, LLVM—succeed because algorithmic insight and systems engineering are applied together, not sequentially.

58.3 What Comes Next

Part 7 has grounded the series in physical reality. The remaining parts turn outward—from general principles to specific domains.

Part 8: Domain Deep Dives—algorithms as they are actually applied in bioinformatics, cryptography, computer graphics, compilers, and databases. Each domain has developed its own vocabulary of algorithmic patterns, its own set of canonical problems, and its own tradeoffs driven by the specific nature of its data and constraints.

Part 9: Machine Learning Algorithms in Depth—the full training pipeline as an algorithmic system: optimization theory, gradient descent

variants, generalization and regularization, architecture design, and the theoretical underpinnings of why any of it works.

Part 10: Problem Solving and Pattern Recognition—the capstone. How to look at an unknown problem and recognize which algorithmic tools apply. Canonical patterns, reduction strategies, the art of simplification, and the discipline of algorithmic thinking as a practice.

Next: Domain Deep Dives—Algorithms in Bioinformatics, Cryptography, Graphics, Compilers, and Databases

Chapter 59

Algorithms: Domain Deep Dives

59.0.1 How Algorithmic Thinking Transforms Five Fields

This is Part 8 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures. Part 4: Complexity Theory. Part 5: Advanced Design Techniques. Part 6: Analysis in Depth. Part 7: Systems and Engineering.

59.1 Why Domains Matter

The previous seven parts built a general-purpose toolkit: sorting, graphs, dynamic programming, complexity classes, amortized analysis, cache-oblivious design, distributed consensus. These are the universal laws of algorithmic thinking.

But science and engineering are not universal. Each domain has its own data, its own constraints, its own definition of "solved." A biologist's sequence of 3 billion base pairs is nothing like an economist's flow network. A cryptographer's notion of "hard" is fundamentally different from a computer graphics artist's notion of "real-time."

What this document shows is that the general toolkit—applied with domain knowledge—produces something richer than either alone. Each of the five domains here has developed its own vocabulary of canonical problems, its own set of key algorithms, and its own sense of what makes a solution elegant. Together they illustrate the scope and unity of algorithmic thinking.

The five domains: **Bioinformatics**, **Cryptography**, **Computer Graphics**, **Compilers** (extended from Part 7), and **Search Engines and Information Retrieval**.

Chapter 60

Part I: Bioinformatics

60.1 The Data of Life

Biology generates sequence data on a scale that dwarfs almost every other field. The human genome is 3.2 billion base pairs—3.2 GB of A, T, C, G characters. A single long-read sequencing run produces terabytes. The 1000 Genomes Project, gnomAD, UK Biobank—these datasets collectively represent petabytes of genomic data.

The fundamental question is deceptively simple: what does this sequence mean? Answering it requires solving string problems, graph problems, and statistical inference problems, all at enormous scale, often with noisy data.

60.2 Sequence Alignment

Pairwise alignment: Given two sequences, find the best way to line them up, allowing gaps (insertions/deletions) and mismatches.

This is the foundational problem of bioinformatics. It tells us how similar two genes are, whether a patient's mutation matches a known disease variant, and where a sequenced read belongs in a genome.

60.2.1 Needleman-Wunsch (Global Alignment)

Align two sequences in their entirety. Dynamic programming on an $(m+1) \times (n+1)$ table.

```
dp[i][j] = best alignment of first i chars of A and first j chars of B

dp[i][j] = max(
    dp[i-1][j-1] + score(A[i], B[j]), <- match or mismatch
    dp[i-1][j] + gap_penalty,          <- gap in B
    dp[i][j-1] + gap_penalty          <- gap in A
)
```

Trace back through the table to recover the alignment. **$O(mn)$ time and space.**

60.2.2 Smith-Waterman (Local Alignment)

Find the best-aligning subsequences—useful when a short gene appears within a long genome.

Same DP, but also allow starting fresh (add 0 to the max), and report the highest-scoring cell.

```
dp[i][j] = max(0, dp[i-1][j-1] + score(...), dp[i-1][j] + gap, dp[i][j-1] + gap)
```

$O(mn)$ time. The gold standard for local alignment accuracy. Too slow for genome-scale search—see BLAST below.

60.2.3 Affine Gap Penalties

Real biology punishes opening a gap more than extending one (a single insertion event creates a run of gaps). Affine gap penalties model this: $\text{gap cost} = \text{gap_open} + (\text{gap_length} - 1) \times \text{gap_extend}$. Implemented with three DP tables (match, gap-in-A, gap-in-B) at the same $O(mn)$ complexity.

60.2.4 BLAST—Practical Heuristic Alignment

Smith-Waterman is $O(mn)$ per query against a database. NCBI's nucleotide database has trillions of bases. Searching it exhaustively is intractable.

BLAST (Basic Local Alignment Search Tool): A heuristic that finds high-scoring local alignments in seconds rather than hours.

Key ideas:

1. **Seed-and-extend:** Find short exact matches (seeds) between query and database using a k-mer index. Only extend seeds that score well.
2. **Neighborhood seeds:** Include not just exact k-mer matches but near-matches (within a score threshold), balancing sensitivity and speed.
3. **Ungapped extension first, then gapped:** Two-phase process—quick ungapped extension to find promising regions, then the full gapped alignment only where justified.

BLAST is one of the most cited scientific tools ever. It underlies essentially all sequence database searching.

60.3 Genome Assembly

Problem: Sequencing machines don't read a whole genome. They produce millions of short reads (50–300 bp for short reads; 10,000–50,000 bp for long reads). Reconstruct the original genome from overlapping reads.

This is a massive string assembly problem. The genome is unknown; you only have fragments; fragments may have sequencing errors; the genome contains repetitive regions (some repeats are millions of bp long).

60.3.1 de Bruijn Graph Assembly

Key insight: Represent the assembly problem as a graph.

Build a de Bruijn graph:

- **Nodes:** All (k-1)-mers present in the reads
- **Edges:** A k-mer from the reads creates an edge from its (k-1)-mer prefix to its (k-1)-mer suffix

Assembling the genome becomes finding an **Eulerian path** through this graph—a path that visits every edge exactly once. Eulerian paths exist and are findable in $O(V+E)$ when every node has in-degree = out-degree (or exactly two exceptions for a path).

Why Eulerian, not Hamiltonian? Hamiltonian path (visit every node once) is NP-hard. Eulerian path is linear. By encoding reads as edges (not nodes), we transform an NP-hard problem into a tractable one.

Complications: Sequencing errors create spurious edges. Repeats create ambiguous paths. Real assemblers (SPAdes, Velvet, MEGAHIT) handle these with graph simplification (tip removal, bubble popping, repeat resolution).

60.3.2 Overlap-Layout-Consensus (OLC)

For long reads (PacBio, Oxford Nanopore), the de Bruijn approach is less effective. OLC takes a different approach:

1. **Overlap:** Find all pairs of reads that overlap (using suffix arrays or BLAST-like methods)
2. **Layout:** Build an overlap graph; find a Hamiltonian path (using greedy heuristics and graph simplification)
3. **Consensus:** Align all reads to the layout, call consensus at each position

Miniasm, Canu, Flye use OLC or hybrid approaches. Long reads span most repeats, enabling more complete assemblies.

60.4 Read Mapping

Once a reference genome exists, map new sequencing reads to it: given a short string (read) and a long string (reference genome), find all approximate matches.

This must be done for billions of reads against a 3 GB reference—efficiency is critical.

60.4.1 Suffix Array + BWT Alignment (BWA, Bowtie2)

Burrows-Wheeler Transform (BWT): A reversible transformation of a string that clusters similar characters together, enabling extremely compact indexing.

The BWT of a string T is constructed by:

1. Generating all rotations of T
2. Sorting them lexicographically
3. Taking the last character of each sorted rotation

The resulting string can be indexed with an FM-index (Ferragina-Manzini index)—a compressed suffix array that enables $O(m)$ pattern matching for a pattern of length m , using only $O(n)$ space (often compressed to $O(n \log \sigma / \log n)$ bits where σ = alphabet size).

Used in: BWA-MEM (Burrows-Wheeler Aligner), Bowtie2—the two most widely used short-read aligners. A 3 GB genome can be indexed in ~3 GB of memory, and a read can be aligned in microseconds.

60.5 Variant Calling

After mapping reads to a reference, identify differences: SNPs (single nucleotide polymorphisms), indels (insertions/deletions), structural variants (large rearrangements).

Hidden Markov Models (HMMs) for variant calling: The HMM models the true sequence as a hidden state; the observed bases (with sequencing errors) are emissions. The Viterbi algorithm finds the most likely true sequence given the observations.

Bayesian variant calling (GATK): For each position, compute the posterior probability of each possible genotype given the observed reads and quality scores. Calls variants where the posterior favors a non-reference genotype.

60.6 Protein Structure Prediction—AlphaFold

Problem: Given a protein's amino acid sequence, predict its 3D structure. This is the "protein folding problem"—one of biology's grandest challenges, open for 50 years.

AlphaFold2 (DeepMind, 2021): Achieved near-experimental accuracy on the CASP14 benchmark, essentially solving the problem for most proteins.

Algorithmic key ideas:

- **Multiple sequence alignment (MSA):** Use evolutionary information—if two positions in a protein family always mutate together, they're likely in contact in 3D space. Computed using Hidden Markov Models and covariance statistics.
- **Attention-based neural network:** A transformer architecture processes the MSA and pair representations jointly. Self-attention over both the sequence and pair dimensions.
- **Iterative refinement:** The structure representation is refined over multiple cycles—the predicted structure feeds back into the attention computation.
- **Equivariant structure module:** Final structure prediction is equivariant to rotations and translations (a structure's meaning doesn't change if you rotate it).

AlphaFold has predicted structures for ~200 million proteins—essentially all known proteins. This is among the most impactful applications of deep learning algorithms to science.

60.7 Phylogenetics

Problem: Given a set of species (or genes), reconstruct the evolutionary tree (phylogeny) that best explains their relationships.

Neighbor joining: A greedy algorithm that iteratively merges the two taxa with the smallest corrected distance. $O(n^3)$. Fast but not statistically optimal.

Maximum likelihood: Find the tree topology and branch lengths maximizing the probability of the observed sequences under an evolutionary model. NP-hard in general—heuristic tree search (SPR moves, NNI moves) is used in practice.

Bayesian phylogenetics (MrBayes, BEAST): MCMC over tree space, sampling from the posterior distribution of phylogenies given the data. Computationally intensive but produces uncertainty estimates.

Chapter 61

Part II: Cryptography

61.1 The Algorithmic Foundation of Trust

Cryptography is the science of securing information. Every HTTPS connection, every digital signature, every encrypted message relies on algorithms whose security is grounded in computational hardness assumptions—problems that can be verified in polynomial time but are believed (and sometimes proved) to require exponential time to solve.

Modern cryptography is a branch of algorithm design where the goal is not efficiency but a precise, mathematical security guarantee.

61.2 Symmetric Cryptography

61.2.1 AES—The Advanced Encryption Standard

AES is a **substitution-permutation network (SPN)**: a sequence of rounds, each performing substitution (S-box lookup), permutation (ShiftRows, MixColumns), and key addition (AddRoundKey).

Parameters: 128/192/256-bit keys, 10/12/14 rounds respectively, 128-bit block size.

S-box (SubBytes): A fixed 16×16 lookup table derived from the multiplicative inverse in $\text{GF}(2^8)$ followed by an affine transformation. Chosen to have no algebraic simplicity that would enable algebraic attacks.

MixColumns: Treats each column of the state as a polynomial over $\text{GF}(2^8)$ and multiplies by a fixed polynomial. This provides **diffusion**—each output byte depends on all input bytes.

Security: No practical attack on full AES is known. The best theoretical attacks require 2^{126} operations (for AES-128)—infeasible. AES is considered secure for the foreseeable future (including against quantum computers, since Grover’s algorithm only provides a quadratic speedup, reducing 128-bit security to 64-bit—still infeasible).

Speed: AES-NI hardware instructions on modern x86 CPUs execute one AES round in a single clock cycle. AES-128 achieves ~1 byte/cycle encryption throughput in hardware—fast enough for disk encryption, network encryption, and TLS at full line rate.

61.2.2 Stream Ciphers and ChaCha20

Block ciphers (like AES) encrypt fixed-size blocks. Stream ciphers generate a pseudorandom keystream that is XORed with plaintext.

ChaCha20 (Bernstein, 2008): Based on ARX operations (Add, Rotate, XOR)—no lookup tables, excellent software performance (especially on systems without AES-NI), and strong security proofs.

Used in TLS 1.3 (as ChaCha20-Poly1305), SSH, WireGuard. Preferred over AES-GCM on mobile devices where AES-NI is absent.

61.3 Asymmetric (Public Key) Cryptography

61.3.1 RSA—Number Theory as Security

Setup: Choose large primes p, q . Compute $n = pq$. Choose e (public exponent) coprime to $\varphi(n) = (p-1)(q-1)$. Compute $d = e^{-1} \bmod \varphi(n)$.

Public key: (n, e) . **Private key:** d .

Encrypt: $C = M^e \bmod n$. **Decrypt:** $M = C^d \bmod n$. Works because $M^{ed} = M^{(1 + k\varphi(n))} = M$ (by Euler’s theorem).

Security: Breaking RSA requires factoring n to recover p and q , then computing $\varphi(n)$ and d . No polynomial-time factoring algorithm is known classically.

RSA in practice: Never encrypt data directly with RSA (many pitfalls). Use RSA to encrypt a symmetric key or sign a hash. OAEP (Optimal Asymmetric Encryption Padding) or PSS (Probabilistic Signature Scheme) provide provable security.

Key sizes: 2048-bit RSA provides ~ 112 bits of security. 3072-bit provides ~ 128 bits. Both will be broken by sufficiently large quantum computers (Shor's algorithm).

61.3.2 Elliptic Curve Cryptography (ECC)

An elliptic curve over a finite field is the set of points satisfying:

$$y^2 = x^3 + ax + b \pmod{p}$$

Points on the curve form a group under a geometric addition rule. The **discrete logarithm problem on elliptic curves (ECDLP)**: Given points P and $Q = kP$, find k . No sub-exponential algorithm is known (unlike integer factoring or discrete log over multiplicative groups).

Consequence: A 256-bit ECC key provides security equivalent to a 3072-bit RSA key. Smaller keys, faster operations, lower bandwidth.

Curve25519: A specific curve designed by Bernstein for high performance and resistance to implementation errors. Used in Signal, WhatsApp, SSH, TLS 1.3.

ECDSA: Elliptic curve digital signature algorithm. Used in Bitcoin (secp256k1 curve), TLS, code signing.

61.3.3 Diffie-Hellman Key Exchange

Two parties agree on a shared secret without ever transmitting it.

Alice chooses secret a , computes $A = g^a \bmod p$, sends A to Bob
Bob chooses secret b , computes $B = g^b \bmod p$, sends B to Alice
Alice computes shared $= B^a \bmod p = g^{a^b} \bmod p$
Bob computes shared $= A^b \bmod p = g^{a^b} \bmod p$

The shared secret g^{ab} is the same for both; an eavesdropper sees only A , B , g , p and must solve the discrete logarithm to recover a or b .

ECDH (Elliptic Curve Diffie-Hellman): The same protocol using elliptic curve point multiplication instead of modular exponentiation.

61.4 Cryptographic Hash Functions

A cryptographic hash function H maps arbitrary-length input to a fixed-size output (digest) with three properties:

1. **Pre-image resistance:** Given $H(x)$, infeasible to find x .
2. **Second pre-image resistance:** Given x , infeasible to find $x' \neq x$ with $H(x) = H(x')$.
3. **Collision resistance:** Infeasible to find any x, x' with $H(x) = H(x')$.

61.4.1 SHA-256 and SHA-3

SHA-256 (Merkle-Damgård construction): Process input in 512-bit blocks, each updating a 256-bit state through 64 rounds of mixing. Widely used, proven in practice.

SHA-3 (Keccak, sponge construction): A completely different design—a sponge function that absorbs input and squeezes output. Resistant to length extension attacks that affect SHA-256. Standardized as a backup in case SHA-2 is broken.

61.4.2 HMAC—Hash-Based Message Authentication

Combining a hash function with a secret key to produce a message authentication code (MAC). Proves both integrity (message wasn't modified) and

authenticity (only someone with the key could have created it).

$$\text{HMAC}(\text{key}, \text{message}) = \text{H}((\text{key} \text{ xor opad}) \parallel \text{H}((\text{key} \text{ xor ipad}) \parallel \text{message}))$$

Provably secure under the assumption that the underlying hash function is a pseudorandom function.

61.5 Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) allow one party (the prover) to convince another (the verifier) that a statement is true without revealing any information beyond the truth of the statement.

Classic example—graph 3-coloring: Peggy wants to prove to Victor she has a valid 3-coloring of a graph without revealing the coloring.

Protocol (repeated many times):

1. Peggy randomly permutes her coloring and commits to each node's color (using a cryptographic commitment scheme).
2. Victor picks a random edge.
3. Peggy reveals the colors of that edge's two endpoints.
4. Victor verifies they are different.

If Peggy is lying (doesn't have a valid coloring), she'll be caught with probability $\geq 1/|\text{edges}|$ per round. After k rounds, probability of undetected lying is $\leq (1-1/|\text{edges}|)^k \rightarrow 0$. Victor learns nothing beyond "the coloring is valid."

Modern ZKPs—zk-SNARKs and zk-STARKs:

zk-SNARKs (Succinct Non-interactive Argument of Knowledge): A prover generates a short proof (~200 bytes) that can be verified in milliseconds, proving knowledge of a secret satisfying some computation, without revealing the secret. Based on elliptic curve pairings.

Used in: Zcash (private cryptocurrency), Ethereum (rollup scaling solutions), identity verification.

zk-STARKs: Transparent (no trusted setup), post-quantum secure, but larger proof sizes. Used in StarkWare's Ethereum scaling.

61.6 Post-Quantum Cryptography

Shor's algorithm runs in polynomial time on a quantum computer and breaks RSA, ECC, and Diffie-Hellman. NIST standardized post-quantum algorithms in 2024:

CRYSTALS-Kyber (key encapsulation): Based on the hardness of the **Module Learning with Errors (MLWE)** problem—given a matrix A and vector $b = As + e$ (where s is the secret and e is small noise), recovering s is believed hard even for quantum computers.

CRYSTALS-Dilithium (digital signatures): Also lattice-based (MLDSA variant). Efficient, small signatures.

SPHINCS+ (signatures): Hash-based signatures. Security reduces to the security of the hash function—very conservative and well-understood, but larger signatures.

61.7 Side-Channel Attacks—Algorithms Must Be Constant-Time

A cryptographic algorithm that is mathematically secure can be broken by observing its physical execution: timing variations, power consumption, electromagnetic emissions.

Timing attack on RSA: A naive RSA implementation's execution time depends on the private key (through the sequence of multiplications in modular exponentiation). Measuring response times across many queries can reveal the private key.

Countermeasure: constant-time programming. The execution time and memory access pattern must not depend on secret data. This means:

- No secret-dependent branches
- No secret-dependent memory accesses (to avoid cache-timing attacks)
- No early exit from loops

Writing constant-time code is notoriously difficult. Even C compilers may "optimize away" constant-time patterns. Libraries like libsodium are specifically designed for constant-time operation.

Chapter 62

Part III: Computer Graphics

62.1 The Rendering Pipeline

Computer graphics asks: given a 3D scene, generate a 2D image. This requires solving problems in geometry, physics simulation, signal processing, and linear algebra—all in real time (>30 frames/second for interactive graphics) or with photographic accuracy (offline rendering).

62.2 Rasterization—The Real-Time Pipeline

The dominant approach for real-time graphics (games, VR, UI): transform 3D geometry into 2D screen pixels by projecting triangles and filling them.

The pipeline:

1. **Vertex processing:** Transform 3D vertex positions through model → world → camera → clip space. Apply lighting calculations. Executed by vertex shaders on the GPU.
2. **Clipping and rasterization:** Clip triangles to the view frustum. Rasterize each triangle—enumerate all pixels it covers using edge equations.
3. **Fragment processing:** For each covered pixel, compute its color. Apply textures, lighting models, shadow maps. Executed by fragment shaders.

4. **Output merger:** Depth testing (z-buffer: keep the nearest fragment), blending (transparency).

Z-buffer algorithm: Maintain a depth value per pixel. When rasterizing a fragment, if its depth < current z-buffer value, update z-buffer and write color. $O(\text{pixels} \times \text{average triangle coverage})$. Simple and hardware-accelerated.

62.2.1 Rasterization—The Triangle Scan-Line Algorithm

A triangle defined by three vertices V_0, V_1, V_2 . For each scanline y :

- Find x-intercepts with the two triangle edges crossing that scanline
- Fill all pixels between intercepts
- Interpolate attributes (texture coordinates, normals) using barycentric coordinates

Barycentric coordinates: Any point P inside a triangle can be expressed as $P = \alpha V_0 + \beta V_1 + \gamma V_2$ where $\alpha + \beta + \gamma = 1$ and all ≥ 0 . Used to interpolate any per-vertex attribute across the triangle surface.

62.2.2 Texture Mapping and Mipmaps

Texture mapping: Map a 2D image (texture) onto a 3D surface by associating (u,v) texture coordinates with each vertex and interpolating.

Aliasing problem: When a textured surface is viewed from far away, many texture pixels map to one screen pixel—causing shimmer and aliasing.

Mipmaps: Precomputed, progressively downsampled versions of a texture ($1\times, 1/2\times, 1/4\times, \dots$). At runtime, select the mipmap level where one texture pixel $\approx$ one screen pixel. Trilinear filtering blends between levels. $O(4/3)$ extra storage per texture.

Anisotropic filtering: Textures at oblique angles have non-square footprints. Anisotropic filtering samples along the elongated footprint direction, eliminating blurriness. $4\text{--}16\times$ anisotropic filtering is standard in modern games.

62.3 Ray Tracing—Physically Accurate Rendering

Rasterization is fast but physically inaccurate—global illumination effects (reflections, refractions, caustics, ambient occlusion) require expensive tricks. Ray tracing is physically correct.

Basic ray tracing: For each pixel, cast a ray from the eye through the pixel into the scene. Find the nearest intersection with any object. Compute lighting at that point. Recursively cast reflection/refraction rays.

Ray-scene intersection is the bottleneck. Naively test every ray against every object: $O(\text{rays} \times \text{objects})$. For a scene with millions of triangles and millions of rays, this is impractical.

62.3.1 Bounding Volume Hierarchies (BVH)

Organize scene geometry in a tree of bounding boxes. Each node has a bounding box containing all its children’s geometry.

Ray-BVH traversal: Test the ray against the root bounding box. If it misses, skip the entire subtree. Recurse into children that the ray intersects.

Cost: $O(\log n)$ for most rays in a well-built BVH. Building the BVH takes $O(n \log n)$ using Surface Area Heuristic (SAH)—split at the position minimizing expected ray intersection cost.

Hardware BVH traversal: NVIDIA’s RT Core units in Turing/Ampere/Ada GPUs are dedicated hardware for BVH traversal, executing ray-box and ray-triangle tests independently of shader execution. Enabled real-time ray tracing at 30-60 fps.

62.3.2 Monte Carlo Path Tracing

Physically correct rendering requires integrating the rendering equation—an integral over all incoming light directions at each surface point.

Monte Carlo integration: Estimate the integral by sampling random incoming directions, weighting by the BRDF (Bidirectional Reflectance Distribution Function) and following the light path.

Each "path" traces the full journey of a photon from camera through multiple bounces to a light source. Average many paths per pixel.

The problem: Converges slowly—variance decreases as $O(1/\sqrt{\text{samples}})$. Noise requires thousands of samples per pixel.

Importance sampling: Sample directions proportional to the BRDF and/or light intensity, not uniformly. Dramatically reduces variance by concentrating samples where they matter.

Next event estimation: At each intersection, directly sample the light source rather than hoping a random ray hits it. Combined with path continuation, this gives both direct and indirect illumination.

Denoising: Neural denoising (DLSS, OptiX Denoiser, OIDN) trains a CNN to remove Monte Carlo noise from low-sample renders. Renders at 4 samples/pixel and denoises to quality equivalent to 1000 samples—a $250\times$ speedup.

62.4 Global Illumination Techniques

62.4.1 Radiosity

For diffuse surfaces, model light transport as a system of linear equations—each surface patch's radiosity depends on all other patches (via view factors). Solving this gives fully accurate diffuse illumination.

$O(n^2)$ to compute view factors, $O(n^3)$ to solve the system. Excellent for static scenes; impractical for dynamic ones.

62.4.2 Photon Mapping (Jensen, 1996)

Two-pass algorithm:

1. **Forward pass:** Trace photons from light sources, accumulating them in a photon map (a k-d tree of photon impacts).
2. **Backward pass:** Trace rays from the camera. At each intersection, gather nearby photons from the map (using k-nearest-neighbor search in the k-d tree) to estimate radiance.

Excellent for caustics (focused light through glass, underwater patterns). $O(n \log n)$ photon map construction and $O(\log n)$ per photon gather.

62.4.3 Spherical Harmonics for Irradiance

Precompute the low-frequency irradiance at each surface point and represent it using the first few spherical harmonic basis functions (9 coefficients for 2nd-order SH). Achieves soft, low-frequency ambient lighting in real time. Used in game engines for baked global illumination.

62.5 Geometry Processing

62.5.1 Mesh Simplification

3D models from scanning or CAD tools often have millions of triangles—too many for real-time rendering of distant objects.

Quadric Error Metrics (Garland-Heckbert, 1997): Iteratively collapse the edge (merge two vertices) that minimizes geometric error, measured by a quadric form. Each edge collapse uses a priority queue; the highest-priority (lowest-error) collapse is applied first.

Reduces a million-triangle mesh to 10,000 triangles with visually minimal degradation. $O(n \log n)$.

62.5.2 Subdivision Surfaces

Represent smooth surfaces as a coarse mesh that is refined by subdivision rules. Catmull-Clark subdivision (used in Pixar's RenderMan): each quad face is subdivided into four quads, with vertex positions averaged according to specific weights. Converges to a C^2 continuous surface.

62.5.3 Signed Distance Fields (SDFs)

Represent geometry implicitly: each point in space stores its signed distance to the nearest surface (positive = outside, negative = inside).

Advantages: Smooth blending of shapes (min operation = union), clean ray marching intersection (sphere tracing), GPU-friendly.

Uses: Font rendering (anti-aliased text at any scale), soft body physics, cloud/smoke rendering, game terrain, 3D printing.

62.6 Physics Simulation

62.6.1 Rigid Body Dynamics

Simulate the motion of rigid objects under forces and constraints.

Integration: Euler integration (first order, simple, unstable for stiff systems), Verlet integration (second order, more stable, used in cloth simulation), Runge-Kutta (fourth order, accurate).

Collision detection:

- **Broad phase:** Use BVH or spatial hashing to quickly find potentially colliding pairs.
- **Narrow phase:** GJK algorithm (Gilbert-Johnson-Keerthi, 1988) for convex shape intersection. $O(1)$ per pair for typical cases.

Constraint solving: Rigid body constraints (joints, hinges) are solved as a linear system. Iterative solvers (Projected Gauss-Seidel) are standard in physics engines (PhysX, Bullet, Havok).

62.6.2 Fluid Simulation

Eulerian approach (grid-based): Represent the fluid on a fixed grid; advect quantities (velocity, density) through the grid.

Navier-Stokes equations govern fluid motion. Solved numerically by splitting into advection (move quantities along velocity field) + pressure projection (enforce incompressibility by solving a Poisson equation—a sparse linear system).

Lagrangian approach (particle-based, SPH): Represent fluid as particles. Each particle's properties are smoothed over nearby particles using a kernel function. Used in real-time water effects (games), computational fluid dynamics.

Chapter 63

Part IV: Information Retrieval and Search Engines

63.1 The Scale of Search

Google indexes hundreds of billions of web pages. A query like "machine learning algorithms" is matched against this index in under 100 milliseconds, ranked, and personalized. This is arguably the most demanding algorithmic system ever built in terms of scale $\times$ latency $\times$ relevance requirements.

63.2 Indexing—The Inverted Index

Inverted index: A mapping from each word (term) to a posting list—the list of documents containing that word, often with position and frequency information.

```
"algorithm" -> [doc3: [pos 12, 47], doc8: [pos 3], doc15: [pos 200, 201], ...]  
"sorting"   -> [doc3: [pos 13], doc15: [pos 199], ...]
```

Construction: Process each document, tokenize, apply stemming (reduce words to root form), stop word removal (remove "the", "a", etc.), build in-memory index, merge into disk-based index using external sort.

Compression: Posting lists are sorted by document ID. Store differences (delta encoding) instead of absolute IDs. Variable-byte encoding or PForDelta

compress these deltas efficiently. The entire Google index is in compressed form.

63.2.1 Intersection and Union—Boolean Retrieval

For a query "algorithm AND sorting": intersect the two posting lists.

With sorted lists, intersection takes $O(|P_1| + |P_2|)$ using two pointers. This is $O(n)$ —the same as scanning—but with early termination when one list is exhausted.

DAAT (Document-At-A-Time): Process all posting lists simultaneously, advancing the pointer of the lowest document ID. Efficient when many lists must be intersected.

TAAT (Term-At-A-Time): Process one term's posting list fully, then the next. Uses an accumulator for each document. Better when result set is small relative to index size.

63.3 Ranking—From Retrieval to Relevance

Returning all matching documents is not enough. 100 million documents match "machine learning." Ranking determines what the user actually sees.

63.3.1 TF-IDF

Term Frequency (TF): How often does the query term appear in this document? More occurrences → more relevant (with diminishing returns).

Inverse Document Frequency (IDF): How rare is this term across all documents? $\log(N/df)$ where N = total docs, df = docs containing this term. Rare terms are more discriminative.

TF-IDF score: $TF \times IDF$. Balance term frequency with term rarity. Simple, effective, still widely used.

63.3.2 BM25 (Best Match 25)

The standard ranking function in information retrieval, outperforming TF-IDF consistently.

$$\text{BM25}(d, q) = \text{Sigma IDF}(t) \times (\text{tf}(t, d) \times (k_1 + 1)) / (\text{tf}(t, d) + k_1 \times (1 - b))$$

Parameters k_1 (term frequency saturation, typically 1.2–2.0) and b (length normalization, typically 0.75) are tuned per collection. $|d|$ is document length; avgdl is average document length.

Key improvements over TF-IDF: Nonlinear TF saturation (doubling occurrences doesn't double score), explicit document length normalization.

Used in: Elasticsearch, Solr, Lucene, most production search systems.

63.3.3 Learning to Rank (LTR)

Modern search engines use machine learning to combine hundreds of features (BM25 score, PageRank, user click-through rate, freshness, anchor text, semantic similarity) into a single ranking function.

Pointwise: Train a regression model predicting relevance score per (query, document) pair.

Pairwise: Train to correctly order pairs of documents—RankSVM, Rank-Boost.

Listwise: Optimize directly for ranking metrics (NDCG, MAP)—LambdaMART (gradient boosted trees over pairwise preferences). Used in Bing and most modern web search engines.

63.3.4 PageRank—The Structural Signal

PageRank (Page, Brin, 1998): Model a random web surfer following links uniformly at random (with occasional teleportation). The steady-state probability of visiting each page is its PageRank.

$$\text{PR}(A) = (1-d)/N + d \times \text{Sigma}_{\{B \rightarrow A\}} \text{PR}(B) / \text{OutLinks}(B)$$

Where $d \approx 0.85$ is the damping factor. Computed by power iteration—repeatedly apply the transition matrix until convergence.

Convergence: Power iteration converges in $O(\log(1/\varepsilon) / \log(1/\lambda_2))$ iterations where λ_2 is the second eigenvalue of the transition matrix. For the web graph, typically 30–100 iterations.

Personalized PageRank: Teleport preferentially to a seed set (the user’s interest profile or a set of trusted pages). Used for search personalization, spam detection, recommendation systems.

63.4 Approximate Nearest Neighbor Search

Problem: Given a large corpus of high-dimensional vectors (document embeddings, image features, product representations), find the k nearest neighbors to a query vector. Exact search is $O(n \cdot d)$ per query—too slow for millions of queries per second.

63.4.1 Locality Sensitive Hashing (LSH)

Hash vectors such that similar vectors are more likely to hash to the same bucket. For cosine similarity, random hyperplane projections work: $\text{hash}(v) = \text{sign}(r \cdot v)$ for a random unit vector r .

Multiple hash tables + multiple hash functions give high recall with $O(1)$ query time (at some cost in accuracy).

63.4.2 HNSW (Hierarchical Navigable Small World)

A graph-based ANN algorithm. Build a multi-layer graph where upper layers are sparse (long-range connections) and lower layers are dense (local connections). Navigate from coarse to fine.

Query: Start at the top layer’s entry point. Greedily follow edges to neighbors closer to the query. Drop to the next layer. Repeat until the bottom layer. Return k nearest found.

Performance: Sub-linear query time in practice. State-of-the-art accuracy-speed tradeoff for dense vector search.

Used in: FAISS (Facebook AI Similarity Search), Weaviate, Milvus, Qdrant, pgvector—the infrastructure of modern semantic search and RAG (retrieval-augmented generation) systems.

63.4.3 Product Quantization (PQ)

Compress vectors into short codes for approximate distance computation. Split each vector into M subvectors; quantize each independently using k -means (256 centroids $\rightarrow$ 8-bit code per subvector). Compute approximate distances using precomputed lookup tables.

Used in FAISS for billion-scale vector search: 1 billion 128-dimensional vectors in ~128 GB instead of 512 GB, with approximate nearest neighbor search in milliseconds.

63.5 Document Representation and Neural Retrieval

63.5.1 TF-IDF Vectors and Sparse Retrieval

The classical approach: represent documents as sparse TF-IDF vectors in a vocabulary-sized space. Similarity = cosine similarity or BM25. Efficient via inverted index. Misses synonyms and paraphrases.

63.5.2 Dense Retrieval (Neural Embeddings)

Encode documents and queries as dense vectors (e.g., 768-dimensional) using a transformer model (BERT, DPR). Similarity = dot product or cosine similarity.

Captures semantic similarity: "automobile" and "car" map to nearby vectors. **Challenge:** No inverted index applies—must use ANN search. Encoding millions of documents requires significant compute.

Used in: Google's MUM and Gemini, Microsoft's Bing (with OpenAI models), Elasticsearch's kNN search, all modern enterprise search.

63.5.3 Hybrid Retrieval

Combine BM25 (exact term matching) with dense retrieval (semantic matching) via reciprocal rank fusion or learned fusion. Captures both exact keyword matches and semantic similarity—consistently outperforms either alone.

63.6 Query Understanding and Spelling Correction

Spelling correction: Given a potentially misspelled query, find the most likely intended query.

Noisy channel model: $P(\text{intended} \mid \text{observed}) \propto P(\text{observed} \mid \text{intended}) \times P(\text{intended})$. Edit distance for $P(\text{observed} \mid \text{intended})$, language model for $P(\text{intended})$.

Peter Norvig's spell corrector: Generate all words within edit distance 1 or 2, filter to valid words, rank by frequency. Elegant, $O(1)$ per query for common cases.

Neural spell correction: Sequence-to-sequence models (encoder-decoder with attention) learn correction patterns from data. Used in modern search engines for complex misspellings and phonetic errors.

Chapter 64

Part V: Cross-Domain Themes

64.1 Probabilistic Models Appear Everywhere

Hidden Markov Models: gene finding in bioinformatics, speech recognition in NLP, protein family classification.

Bayesian inference: variant calling, spam filtering, document classification, recommendation systems.

Monte Carlo methods: path tracing in graphics, MCMC in phylogenetics and statistics, simulated annealing in optimization.

The mathematics of probability and statistics is not just one domain's tool—it is a layer beneath all of these fields.

64.2 Graph Algorithms Underlie Everything

Genome assembly is an Eulerian path problem. The web is a graph to be PageRanked. Dependency analysis in compilers is a DAG problem. Scene hierarchy in graphics is a tree traversal. Network routing in distributed systems is shortest path.

The graph abstraction is universal. Problems that look nothing like graphs turn out, on inspection, to reduce to fundamental graph problems.

64.3 Dimensionality is the Universal Enemy

The curse of dimensionality affects all five domains: high-dimensional genomic feature spaces, high-dimensional neural network weight spaces, high-dimensional image and video representations, high-dimensional document embeddings, high-dimensional molecular conformations. Every domain has developed its own dimensionality reduction techniques—PCA in bioinformatics and genomics, polynomial basis reduction in cryptography, level-of-detail in graphics, word embeddings and attention mechanisms in search—but the enemy is always the same.

64.4 Approximation and Exactness in Tension

In bioinformatics, exact sequence alignment is too slow at genome scale—BLAST uses heuristics. In cryptography, approximate solutions are unacceptable—security requires exactness. In graphics, approximate rendering is essential for real-time—ray tracing approximates the full physics. In search, exact Boolean retrieval is too rigid—BM25 and neural ranking are approximations.

Each domain has developed a precise sense of where approximation is acceptable and where it is not. That judgment is inseparable from domain knowledge—and it is one of the deepest skills an algorithm designer can develop.

64.5 What Comes Next

Part 8 has shown the general toolkit applied to five specific domains, each with its own richness and constraints.

Part 9: Machine Learning Algorithms in Depth zooms into one of the most important algorithmic developments of the last two decades. Machine learning is not just another application domain—it represents a fundamental shift in how algorithms are designed, moving from explicit programming to learning from data. The optimization theory, architecture design principles,

generalization theory, and practical engineering of training large models form a complete algorithmic system in their own right.

Part 10: Problem Solving and Pattern Recognition will be the capstone—how to look at any new problem and find the algorithmic handle.

Next: Machine Learning Algorithms in Depth—Optimization, Architecture, and the Theory of Learning

Chapter 65

Algorithms: Machine Learning in Depth

65.0.1 Optimization, Architecture, Generalization, and the Theory of Learning

This is Part 9 of the Algorithms series. Part 1: History & Theory. Part 2: Algorithm Categories. Part 3: Data Structures. Part 4: Complexity Theory. Part 5: Advanced Design Techniques. Part 6: Analysis in Depth. Part 7: Systems & Engineering. Part 8: Domain Deep Dives.

65.1 A Different Kind of Algorithm

Every algorithm in this series so far has been **explicitly designed**—a human specified the steps, proved correctness, and analysed performance. Merge sort doesn't discover sorting; it executes a sorting procedure we wrote. Dijkstra doesn't infer shortest paths; it computes them by a fixed rule.

Machine learning turns this around. The algorithm is the **training procedure**—the mechanism by which a model discovers its own behaviour from data. We specify the structure (architecture), the objective (loss function), and the update rule (optimiser). The specific computation the trained model performs—the weights, the representations, the decisions—is learned.

This is a profound shift. It means that many problems we cannot program explicitly (recognising faces, translating language, predicting protein structure, playing Go) become tractable if we have enough data and the right training machinery. It also means that understanding ML requires understanding not just the trained model but the optimisation landscape it navigates, the statistical guarantees on what it learns, and the systems engineering required to train at scale.

This document covers all four dimensions: **optimisation** (how models are trained), **architecture** (what structures are learned), **generalisation** (why trained models work on new data), and **engineering** (how this is done at scale).

Chapter 66

Part I: The Learning Problem—Foundations

66.1 What We're Trying to Do

Supervised learning: Given a training set $(x_1, y_1), \dots, (x_n, y_n)$ of input-output pairs, learn a function $f: X \rightarrow Y$ that generalizes to new inputs.

Unsupervised learning: Given inputs $x_1, \dots, x_n$ without labels, discover structure—clusters, representations, generative models.

Reinforcement learning: An agent interacts with an environment, receiving rewards. Learn a policy $\pi(a|s)$ that maximizes cumulative reward.

The formal framework for supervised learning is **statistical learning theory**.

66.2 Statistical Learning Theory

The setup: Data is drawn i.i.d. from an unknown distribution $P(x, y)$. A hypothesis class $\mathcal{H}$ is a set of functions $f: X \rightarrow Y$ (e.g., all linear classifiers, all depth-5 decision trees, all neural networks with a given architecture). A loss function $\ell(f(x), y)$ measures prediction error.

True risk: $R(f) = E_{(x,y) \sim P}[\ell(f(x), y)]$ —what we care about. **Empirical risk:** $R(f) = (1/n) \sum \ell(f(x_i), y_i)$ —what we can compute.

Empirical Risk Minimisation (ERM): Choose $f^* = \operatorname{argmin}_{f \in \mathcal{H}} R(f)$.

The fundamental question: When does minimising empirical risk guarantee low true risk?

66.3 The Bias-Variance Tradeoff

Bias: How wrong is the best hypothesis in $\mathcal{H}$? A restricted class (low-degree polynomials, linear models) may be unable to represent the true function—high bias.

Variance: How much does our learned hypothesis vary with the training data? A very flexible class may fit training data perfectly but change wildly across different training sets—high variance.

Bias-variance decomposition:

$$\text{Expected test error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

Increasing model capacity decreases bias but increases variance. The classical view says there is an optimal capacity—a sweet spot minimising total error.

The modern twist: Deep neural networks have enormous capacity—enough to memorise training data. Classical theory predicts high variance and poor generalisation. In practice, large neural networks generalise well. This is the **double descent** phenomenon—error first increases as capacity grows (classical overfitting regime), then decreases again for very large models (interpolation regime where the model is overparameterized). Understanding why is an active research area.

66.4 VC Dimension and Sample Complexity

VC dimension (Vapnik-Chervonenkis dimension): A measure of a hypothesis class's capacity. $VC(\mathcal{H})$ is the largest set of points that $\mathcal{H}$ can **shatter**—classify correctly in all 2^n possible labelings.

- Linear classifiers in $\mathbb{R}^d$: VC dimension = $d+1$
- Neural networks: VC dimension grows with the number of parameters

Fundamental theorem of learning theory (PAC learning):

With n training examples, probability delta of failure:
 $|R(f) - R(f)| \leq O(\sqrt{VC(H)/n} + \log(1/\delta)/n)$

For generalisation, you need $n = \Omega(VC(H)/\varepsilon^2)$ samples. The VC dimension controls the sample complexity.

The modern paradox: Large neural networks have VC dimension in the billions, yet generalise well with millions of training examples—far fewer than VC theory predicts is sufficient. Classical VC-dimension bounds are too loose by orders of magnitude for deep learning.

Better explanations: Implicit regularisation (gradient descent prefers simpler solutions), norm-based bounds (generalisation depends on weight norms, not parameter count), algorithmic stability (small perturbations to training data lead to small changes in the learned model).

66.5 PAC Learning

Probably Approximately Correct (PAC) learning (Valiant, 1984):

A concept class C is PAC-learnable if there exists an algorithm A and polynomial $p(1/\varepsilon, 1/\delta, n, \text{size})$ such that for any $\varepsilon, \delta > 0$ and any target concept $c \in C$, after seeing $p(\cdot)$ samples drawn from any distribution, A outputs h with $\Pr[R(h) > \varepsilon] < \delta$.

Efficiently PAC-learnable: There also exists a polynomial-time learning algorithm.

Results:

- Linear threshold functions: PAC-learnable in polynomial time
- Boolean conjunctions: efficiently PAC-learnable

- 3-term DNF: PAC-learnable but not known to be efficiently so
- General neural networks: not known to be efficiently PAC-learnable

PAC learning captures the theoretical sample and computational efficiency of learning. But the focus on worst-case distributions makes it pessimistic for practical deep learning.

Chapter 67

Part II: Optimisation—How Models Are Trained

67.1 The Loss Landscape

Training a neural network means minimising a loss function $L(\theta)$ where $\theta \in \mathbb{R}^d$ is the parameter vector (potentially billions of dimensions for large models).

The loss landscape is a high-dimensional surface. The optimisation challenge:

- **Non-convex:** Many local minima and saddle points
- **High-dimensional:** Intuitions from low dimensions break down
- **Stochastic:** We only see mini-batches, not the full gradient
- **Computationally expensive:** Each gradient evaluation requires a forward and backward pass through the full model

67.2 Gradient Descent and Its Variants

Gradient descent: Move parameters in the direction of steepest descent.

```
theta <- theta - ?nablaL(theta)
```

Where η is the learning rate. For convex functions, converges to the global minimum. For non-convex, converges to a stationary point ($\nabla L = 0$).

Batch gradient descent: Compute ∇L over all training data. Expensive per step but accurate gradient estimate.

Stochastic gradient descent (SGD): Estimate gradient from a single random sample. Cheap but noisy.

Mini-batch SGD: Estimate gradient from a small batch (32–1024 examples). The standard in practice—balances cost and accuracy.

67.2.1 The Learning Rate—The Critical Hyperparameter

Too high: Diverges—loss increases, parameters explode. **Too low:** Converges slowly; may get stuck in poor local minima.

Learning rate schedules:

- **Step decay:** Reduce by a factor every k epochs
- **Cosine annealing:** Learning rate follows a cosine curve from max to near-zero
- **Linear warmup + decay:** Start low, ramp up, then decay—standard for transformer training
- **Cyclical learning rates (CLR):** Cycle between min and max, helping escape local minima

67.3 Momentum and Acceleration

Momentum: Accumulate a velocity vector that dampens oscillations and accelerates in consistent directions.

```
v <- betav - ?nablaL(theta)
theta <- theta + v
```

With $\beta = 0.9$, the effective learning rate in a consistent direction is $\eta/(1-\beta) = 10\eta$.

Nesterov momentum: Compute gradient at $\theta + \beta v$ (where we'll be) rather than θ (where we are). Improves convergence for convex problems by a constant factor, and often better in practice for non-convex.

Why momentum helps intuitively: In a long narrow valley (common in deep learning loss landscapes), gradient descent oscillates across the narrow dimension while making slow progress along the valley. Momentum dampens oscillations and accelerates along the valley direction.

67.4 Adaptive Learning Rate Methods

Different parameters need different learning rates. Early weights may need large updates; later weights may be nearly converged.

67.4.1 AdaGrad (Duchi et al., 2011)

```
g_t = nablaL(theta_t)
G_t = G_{t-1} + g_t^2          <- accumulated squared gradients
theta_{t+1} = theta_t - (? / sqrt(G_t + eps)) x g_t
```

Parameters with large historical gradients get smaller effective learning rates. Good for sparse gradients (NLP tasks with rare words). Problem: G_t grows monotonically—learning rate decreases to zero, stopping learning.

67.4.2 RMSProp (Hinton, 2012)

```
G_t = beta G_{t-1} + (1-beta) g_t^2  <- exponential moving average, not sum
theta_{t+1} = theta_t - (? / sqrt(G_t + eps)) x g_t
```

Fixes AdaGrad's vanishing learning rate by using an exponential moving average.

67.4.3 Adam (Kingma and Ba, 2014)

The dominant optimiser for deep learning.

```

m_t = beta_1m_{t-1} + (1-beta_1)g_t      <- 1st moment (mean)
v_t = beta_2v_{t-1} + (1-beta_2)g_t^2    <- 2nd moment (variance)
m_t = m_t / (1 - beta_1^t)               <- bias correction
v_t = v_t / (1 - beta_2^t)               <- bias correction
theta_{t+1} = theta_t - ? x m_t / (sqrt(v_t) + eps)

```

Combines momentum (1st moment) with adaptive learning rates (2nd moment). Bias correction is critical at the start when moments are poorly estimated.

Default hyperparameters $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$ work well across a huge range of tasks.

AdamW: Adam + decoupled weight decay (L2 regularisation applied to weights directly, not through the gradient). Better generalisation for large language models. The standard optimiser for transformers.

67.4.4 Convergence Theory of Adam

Adam has complicated convergence properties. For non-convex stochastic optimisation, it converges to a stationary point at rate $O(1/\sqrt{T})$. But in practice it often generalises *worse* than SGD with momentum for image classification—a known phenomenon. For language models, Adam is consistently better.

The gap between theory and practice is wide here—Adam’s empirical success is only partially explained by theory.

67.5 Second-Order Methods

Gradient descent uses only first derivatives. Second-order methods use the Hessian (matrix of second derivatives) to account for curvature.

Newton’s method:

```
theta <- theta - H^-1nablaL(theta)
```

Where H is the $n \times n$ Hessian. Converges quadratically near a minimum. Problem: computing and inverting H is $O(d^3)$ —completely infeasible for $d = \text{billions}$.

Quasi-Newton (L-BFGS): Approximate H^{-1} using gradient history. $O(d)$ per step (with limited memory). Used for smaller models and convex problems. L-BFGS is the standard optimiser for logistic regression and SVMs.

Fisher Information Matrix (Natural Gradient): Use the Fisher information matrix (the expected outer product of gradients) as a curvature matrix. This gives "natural gradient descent"—invariant to reparameterization of the model. K-FAC (Kronecker-Factored Approximate Curvature) makes this practical for neural networks by factoring the Fisher matrix.

Shampoo (Gupta et al., 2018): A full-matrix preconditioner for each layer's gradient. Used by Google for large model training—faster convergence in steps, though more expensive per step.

67.6 Backpropagation—The Engine of Deep Learning

The fundamental challenge: Computing $\nabla L(\theta)$ for a neural network with millions of parameters.

Naïve approach: Perturb each parameter by ϵ , observe change in loss, estimate partial derivative. $O(d)$ forward passes—infeasible.

Backpropagation (Rumelhart, Hinton, Williams, 1986): Compute all partial derivatives in a single backward pass through the network using the chain rule.

The chain rule:

$$dL/d\theta_i = \sum_j (dL/dy_j)(dy_j/d\theta_i)$$

For a neural network, this decomposes into local operations at each layer. The backward pass mirrors the forward pass in computational cost—one

backward pass costs roughly $2\times$ a forward pass.

Automatic differentiation (autograd): Modern deep learning frameworks (PyTorch, JAX, TensorFlow) implement **reverse-mode automatic differentiation**—a generalisation of backprop that works for any differentiable computation graph.

Forward-mode vs. reverse-mode AD:

- Forward-mode: Propagate derivatives forward. Efficient when $|\text{inputs}| \ll |\text{outputs}|$ (rare in ML).
- Reverse-mode (backprop): Propagate gradients backward. Efficient when $|\text{outputs}| \ll |\text{inputs}|$ —exactly the ML case (one scalar loss, millions of parameters).

Higher-order derivatives: Taking gradients of gradients. Used in meta-learning (MAML), Hessian-vector products for second-order methods, and hyperparameter optimisation.

67.7 The Loss Landscape of Deep Networks

Saddle points, not local minima: In high dimensions, most critical points ($\nabla L = 0$) are saddle points—some directions curve up, some curve down. Gradient descent escapes saddle points slowly, but SGD's noise helps it escape. True local minima (all directions curve up) are rare in high-dimensional non-convex landscapes.

The loss surface is connected: Empirically, most local minima found by SGD are connected by paths of approximately equal loss (the "loss plateau" or "linear mode connectivity"). This explains why ensembling models found by different random seeds gives consistent improvements.

Sharp vs. flat minima: Flat minima (where the loss is low in a large neighbourhood) generalise better than sharp minima (where the loss is low in a narrow valley). SGD's noise drives toward flat minima; large-batch training finds sharper minima and generalises worse—the "batch size generalisation gap."

Overparameterized interpolation: When the model has more parameters than training points, it can fit the training data exactly (zero training loss). In this regime, gradient descent finds the minimum-norm solution—the solution closest to initialisation. This implicit bias toward low-norm solutions explains why overparameterized networks generalise.

Chapter 68

Part III: Neural Network Architectures

68.1 The Universal Approximation Theorem

Universal approximation (Cybenko, 1989; Hornik et al., 1989): A neural network with one hidden layer and a sufficient number of neurons can approximate any continuous function on a compact domain to arbitrary precision.

What it doesn't say: It doesn't bound the required width, doesn't guarantee that gradient descent finds the approximation, and doesn't address generalisation. It is an existence theorem, not a constructive algorithm.

Depth vs. width: Deeper networks can represent some functions exponentially more efficiently than shallow networks. A function that requires exponentially many neurons in one layer may be representable with polynomially many neurons in logarithmically many layers. This is the formal justification for "deep" learning.

68.2 Fully Connected Networks

The original architecture: neurons organised in layers, each fully connected to the next.

$$\text{Layer 1: } a^1 = \sigma(W^1 a^{l-1} + b^1)$$

Where W^l is the weight matrix, b^l the bias vector, σ an activation function.

Activation functions:

- **Sigmoid:** $\sigma(x) = 1/(1+e^{-x})$. Range (0,1). Saturates and kills gradients for large $|x|$. Rarely used in hidden layers today.
- **Tanh:** Range (-1,1). Better than sigmoid but still saturates.
- **ReLU (Rectified Linear Unit):** $\max(0, x)$. Solves vanishing gradient problem for $x > 0$. Dead neurons for $x < 0$. The standard for decades.
- **Leaky ReLU:** $\max(\alpha x, x)$ for small α . Prevents dead neurons.
- **GELU (Gaussian Error Linear Unit):** $x \cdot \Phi(x)$. Smooth approximation to ReLU. Used in GPT, BERT, and most modern transformers.
- **SwiGLU:** A gated variant used in LLaMA and many modern LLMs.

68.3 Convolutional Neural Networks (CNNs)

Motivation: Images have spatial structure. A cat's ear could appear anywhere in the image. Translation invariance and spatial locality suggest a different architecture than fully connected layers.

Convolution layer: Apply a small filter (kernel) at every spatial position, sharing weights across positions.

$$(f * x)[i, j] = \sum_{m, n} f[m, n] \cdot x[i+m, j+n]$$

Key properties:

- **Weight sharing:** The same filter is applied everywhere—dramatically fewer parameters than a fully connected layer
- **Translation equivariance:** Shifting the input shifts the feature map by the same amount

- **Local connectivity:** Each output unit sees only a small receptive field

Pooling: Reduce spatial dimensions, building translation invariance and hierarchy. Max pooling takes the maximum over a local region.

68.3.1 Classic CNN Architectures

LeNet-5 (LeCun, 1989): The original CNN for digit recognition. Conv → Pool → Conv → Pool → FC.

AlexNet (Krizhevsky, Sutskever, Hinton, 2012): Won ImageNet by a large margin. Deeper, larger, ReLU activation, dropout regularisation, GPU training. Sparked the deep learning revolution.

VGG (Simonyan, Zisserman, 2014): Very deep (16-19 layers) using only 3×3 convolutions. Showed depth matters.

ResNet (He et al., 2015): Introduced residual connections—skip connections that add the input directly to the output of a block.

$$\text{output} = F(x) + x \quad \leftarrow \text{residual connection}$$

This solves the **vanishing gradient problem** for very deep networks: gradients can flow directly through the skip connections. ResNets with 100+ layers became feasible. The residual block is now a universal primitive.

EfficientNet: Systematically scales network width, depth, and resolution using a compound coefficient. Achieves better accuracy-compute tradeoffs than any previous architecture through principled scaling.

68.4 Recurrent Neural Networks (RNNs)

For sequential data—text, speech, time series—process inputs one step at a time, maintaining a hidden state.

```
h_t = sigma(W_hh_{t-1} + W_xx_t + b)
y_t = W_yh_t
```

The vanishing gradient problem in RNNs: Backpropagation through time (BPTT) requires multiplying gradients across many time steps. If $|W_h| < 1$, gradients vanish exponentially. If $|W_h| > 1$, gradients explode. Long-range dependencies cannot be learned.

68.4.1 LSTM (Long Short-Term Memory, Hochreiter and Schmidhuber, 1997)

The solution: gated memory cells that can store information for hundreds of timesteps.

```
f_t = sigma(Wf.[h_{t-1}, x_t] + bf)      <- forget gate
i_t = sigma(Wi.[h_{t-1}, x_t] + bi)      <- input gate
g_t = tanh(Wg.[h_{t-1}, x_t] + bg)      <- candidate cell
C_t = f_t * C_{t-1} + i_t * g_t          <- cell state update
o_t = sigma(Wo.[h_{t-1}, x_t] + bo)      <- output gate
h_t = o_t * tanh(C_t)                   <- hidden state
```

The cell state C_t provides a "highway" for gradients to flow without passing through nonlinearities—solving the vanishing gradient problem.

GRU (Gated Recurrent Unit): A simpler variant with fewer gates. Similar performance to LSTM, faster to train. Widely used before transformers.

The demise of RNNs: LSTMs and GRUs are sequential—step t cannot be computed until step $t-1$ finishes. This prevents parallelisation over sequence length. Transformers, which process all positions simultaneously, replaced RNNs for most sequence tasks.

68.5 The Transformer Architecture

The Transformer (Vaswani et al., 2017—"Attention Is All You Need") replaced RNNs for sequence modelling and is now the dominant architec-

ture in natural language processing, vision, speech, and increasingly, other domains.

68.5.1 Self-Attention

The core mechanism. For each position in a sequence, compute a weighted average of all positions' values, where weights are determined by how relevant each position is.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \text{sqrtd_k}) \times V$$

Where:

- **Q (Query):** What is each position looking for?
- **K (Key):** What does each position offer?
- **V (Value):** What information does each position contribute?

The dot product QK^T measures compatibility between queries and keys. Dividing by $\sqrt{d_k}$ stabilises gradients (without this, dot products grow large in magnitude for large d_k , pushing softmax into saturation).

Complexity: $O(n^2d)$ for a sequence of length n and dimension d . Quadratic in sequence length—the primary scaling bottleneck.

68.5.2 Multi-Head Attention

Run h attention mechanisms in parallel with different weight matrices, then concatenate and project.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \times W^O \\ \text{head}_i &= \text{Attention}(QW_iQ, KW_iK, VW_iV) \end{aligned}$$

Each head learns to attend to different types of relationships (syntactic structure, semantic similarity, coreference, etc.).

68.5.3 Positional Encoding

Self-attention is permutation-invariant—it doesn't know position order. Positional encodings add position information.

Sinusoidal encoding (original transformer):

$$\begin{aligned}\text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d)}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d)})\end{aligned}$$

Rotary Position Embedding (RoPE): Encodes position by rotating the query and key vectors. Relative positions are captured naturally. Used in LLaMA, GPT-NeoX, Mistral.

ALiBi: Add a linear bias to attention scores based on distance. Generalises better to longer sequences than seen during training.

68.5.4 The Full Transformer Block

$x \rightarrow \text{LayerNorm} \rightarrow \text{MultiHeadAttention} \rightarrow \text{Residual} \rightarrow \text{LayerNorm} \rightarrow \text{FFN} \rightarrow \text{Residual}$

Feed-Forward Network (FFN): Two linear layers with a nonlinearity—typically a $4\times$ expansion followed by a GELU or SwiGLU activation. Provides per-position computation beyond attention.

Layer Normalisation: Normalise activations within each example (not across the batch). Essential for stable training of deep transformers.

Pre-LN vs. Post-LN: Original transformer uses post-LN (normalise after residual). Modern practice uses pre-LN (normalise before each sub-layer)—more stable for very deep networks.

68.5.5 Encoder, Decoder, Encoder-Decoder

Encoder (BERT): Bidirectional—each position attends to all other positions. Used for classification, entity recognition, semantic understanding.

Decoder (GPT): Causal/autoregressive—each position attends only to previous positions (masked attention). Used for generation.

Encoder-decoder (original transformer, T5): Encoder processes the input, decoder generates output attending to the encoded representation via cross-attention. Used for translation, summarisation, question answering.

68.6 Scaling Laws

Neural scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022): For language models, loss follows power laws in model size N , dataset size D , and compute C :

$$\begin{aligned} L(N) &\sim N^{-0.076} && (\text{loss vs. model size}) \\ L(D) &\sim D^{-0.095} && (\text{loss vs. data size}) \end{aligned}$$

Chinchilla scaling (Hoffmann et al.): For a fixed compute budget, the optimal model size and training tokens satisfy $N \propto C^{0.5}$ and $D \propto C^{0.5}$. The optimal ratio is ~ 20 tokens per parameter. GPT-3 was undertrained—a GPT-3-sized model trained on $20\times$ more data would be better.

Implications: Scaling laws allow predicting model performance before training. They guide decisions about model size vs. data size tradeoffs. They suggest that data is as important as parameters.

Emergent capabilities: Certain abilities appear discontinuously above a threshold scale—few-shot learning, chain-of-thought reasoning, arithmetic. These are not predicted by smooth scaling laws and are poorly understood.

68.7 Attention Variants—Scaling to Longer Sequences

The $O(n^2)$ complexity of self-attention limits sequence length. Many variants address this.

Sparse attention (Longformer, BigBird): Each token attends to a local window plus a few global tokens. Reduces complexity to $O(n)$.

Linear attention: Replace the softmax with a kernel approximation, allowing the attention computation to be reformulated as a linear operation. $O(n)$ complexity but reduced expressivity.

Flash Attention (Dao et al., 2022): Not an approximation—computes exact attention but reorganises the computation to be IO-efficient (tiling to avoid materialising the $n \times n$ attention matrix). 2–4× faster and much more memory-efficient. Now standard in all major frameworks.

Multi-Query Attention (MQA): Share K and V heads across Q heads—much faster inference, slight quality loss. Used in PaLM.

Grouped Query Attention (GQA): A middle ground between MHA and MQA. Used in LLaMA 2 and 3, Mistral.

Mamba (State Space Models): A recurrent architecture with $O(n)$ complexity that achieves transformer-level performance on many tasks by learning to selectively remember information. Challenges transformers for long-sequence tasks.

68.8 Vision Transformers and Beyond

ViT (Vision Transformer, Dosovitskiy et al., 2020): Apply the transformer directly to images by splitting into 16×16 patches, projecting each patch linearly, and treating the sequence of patches as a token sequence. Achieves state-of-the-art image classification with enough data.

CLIP (Radford et al., 2021): Train a vision encoder and text encoder together to align image and text representations. Enables zero-shot classification ("Is this a cat or a dog?") and image-text retrieval.

Diffusion Models (DALL-E 2, Stable Diffusion, Sora): Generate images by learning to reverse a noise process.

Forward process: add Gaussian noise to an image over T steps until it's pure noise. Backward process: a neural network learns to predict and remove the noise at each step.

```
q(x_t|x_{t-1}) = N(x_t; sqrt(1-beta_t)x_{t-1}, beta_t)    <- forward (fixed)
p_theta(x_{t-1}|x_t) = N(x_{t-1}; mu_theta(x_t,t), Sigma_theta(x_t,t)) <- back
```

Training objective: predict the noise added at each step (ϵ -prediction) or the

original image (x-prediction). Sample by starting from noise and running the backward process.

Score matching and DDPM (Ho et al., 2020): The modern formulation. Score = $\nabla_x \log p(x)$. Learning the score function is equivalent to denoising.

Chapter 69

Part IV: Regularisation and Generalisation

69.1 The Generalisation Problem

A model that memorises training data achieves zero training loss but may fail on new data. Regularisation constrains the hypothesis class to prevent overfitting.

69.2 Classical Regularisation

L2 regularisation (weight decay): Add $\lambda \|\theta\|^2$ to the loss. Penalises large weights, shrinks them toward zero. Equivalent to placing a Gaussian prior on weights (MAP estimation).

L1 regularisation (Lasso): Add $\lambda \|\theta\|_1$. Produces sparse solutions—many weights become exactly zero. Used for feature selection.

Elastic net: L1 + L2 combined. Retains L1's sparsity with L2's stability.

69.3 Dropout (Srivastava et al., 2014)

Randomly zero out each neuron with probability p during training. At inference, scale activations by $(1-p)$.

Why it works:

- Forces the network to learn redundant representations—no single neuron can be relied upon
- Approximately equivalent to training an ensemble of 2^n networks (for n neurons) and averaging them at inference
- Acts as an adaptive regulariser that depends on the network's learned representations

Dropout rates of 0.1–0.5 are common. Higher rates for larger models. Less used in modern transformers (replaced by weight decay and careful architecture design).

69.4 Batch Normalisation (Ioffe and Szegedy, 2015)

Normalise activations within each mini-batch to have zero mean and unit variance, then apply learnable scale and shift.

$$\text{BN}(x) = \gamma \times (x - \mu_{\text{batch}}) / \sigma_{\text{batch}} + \beta$$

Effects:

- Reduces internal covariate shift—each layer sees a more stable input distribution
- Acts as regularisation (the batch statistics add noise)
- Allows higher learning rates
- Reduces sensitivity to weight initialisation

Layer Normalisation: Normalise across features within each example rather than across the batch. Better for variable-length sequences and small batches. Standard in transformers.

Root Mean Square Normalisation (RMSNorm): Simplified LayerNorm without entering (only scale). Used in LLaMA and many modern LLMs—empirically equivalent to LayerNorm, slightly faster.

69.5 Data Augmentation

Artificially expand the training set by applying transformations that preserve the label.

Images: Random crops, horizontal flips, color jitter, rotation, cutout (masking random patches), mixup (interpolating between examples and labels), CutMix (replacing patches with patches from another image).

Text: Synonym replacement, back-translation, random deletion, random swap, EDA (Easy Data Augmentation).

The key insight: Data augmentation encodes domain knowledge about what invariances the model should have. A cat remains a cat whether flipped horizontally—so training on flipped cats teaches horizontal flip invariance.

AutoAugment (Cubuk et al., 2018): Learn augmentation policies using reinforcement learning. Found policies that outperformed hand-designed ones on ImageNet.

69.6 Transfer Learning and Fine-Tuning

Pre-training: Train a large model on a large, general dataset. **Fine-tuning:** Continue training on a smaller, task-specific dataset.

The pre-trained model’s representations capture general structure (edges, textures, and shapes for vision; syntax, semantics, and world knowledge for language). Fine-tuning adapts these to the specific task with limited data.

Why it works: The learned representations are compositional and transferable. Features learned for ImageNet (textures, shapes, object parts) are useful for medical imaging, satellite imagery, etc.

BERT fine-tuning: Pre-train on masked language modelling + next sentence prediction. Fine-tune by adding a task-specific head and training end-to-end on labeled data. Achieves state-of-the-art on most NLP benchmarks with minimal task-specific data.

69.6.1 Parameter-Efficient Fine-Tuning (PEFT)

Fine-tuning all parameters of a 70B-parameter model requires enormous compute and storage. PEFT methods adapt pre-trained models with minimal new parameters.

Adapter layers: Insert small bottleneck modules (down-project $\rightarrow$ nonlinearity $\rightarrow$ up-project) into each transformer block. Only adapter parameters are trained.

LoRA (Low-Rank Adaptation, Hu et al., 2021): Represent the weight update ΔW as a low-rank factorisation $\Delta W = AB$ where $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{r \times n}$ with $r \ll \min(m, n)$.

$$W' = W + \Delta W = W + AB$$

Only A and B are trained ($r/\min(m, n) \times$ fewer parameters). At inference, ΔW can be merged into W —zero additional inference cost.

Used in: Fine-tuning LLMs for specific domains (legal, medical, code), adapting image generation models to specific styles, personalisation.

Chapter 70

Part V: Self-Supervised and Un-supervised Learning

70.1 Self-Supervised Learning

The insight: Labels are expensive; raw data is cheap. Can we design pretext tasks that force the model to learn useful representations without human labels?

BERT's masked language modelling: Mask 15% of tokens; predict them. The model must understand context to fill in masks.

GPT's autoregressive language modelling: Predict the next token. The model must capture everything relevant about the sequence context.

SimCLR / contrastive learning: Apply two different augmentations to an image; train the model so the two views are close in embedding space and far from other examples.

$$L = -\log(\exp(\text{sim}(z_i, z_j)/\tau) / \sum_k \exp(\text{sim}(z_i, z_k)/\tau))$$

Where sim = cosine similarity and τ = temperature. **InfoNCE loss**—an instance of mutual information maximisation.

BYOL (Bootstrap Your Own Latent): No negative pairs—uses two networks (online and target), with the target network being an exponential

moving average of the online network. Avoids representation collapse without explicit negatives.

DINO (Self-Distillation with No Labels): Vision transformer trained with self-distillation learns patch-level semantic features. The attention maps of DINO-trained ViTs naturally segment foreground objects—completely unsupervised.

70.2 Representation Learning and Embeddings

Word2Vec (Mikolov et al., 2013): Learn word embeddings by predicting context words (skip-gram) or predicting a word from context (CBOW). Words with similar meanings end up close in embedding space.

Properties of word embeddings:

- Semantic similarity: similar words have close vectors
- Analogical reasoning: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ (vector arithmetic captures semantic relationships)
- Compositionality: phrase meanings can be approximately composed from word meanings

Sentence and document embeddings: Dense vectors representing the meaning of a sentence or document. Trained with contrastive objectives (similar texts should be close; dissimilar texts should be far).

Used in: Semantic search, recommendation, clustering, few-shot classification via nearest neighbour in embedding space.

70.3 Generative Models

Variational Autoencoders (VAEs, Kingma and Welling, 2013): Encode inputs to a distribution over latent vectors; decode samples from this distribution back to data.

Training objective: maximise the evidence lower bound (ELBO):

$$\text{ELBO} = \mathbb{E}_{z \sim q(z|x)} [\log p(x|z)] - \text{KL}(q(z|x) \parallel p(z))$$

Reconstruction term + regularisation toward the prior. VAEs produce smooth, continuous latent spaces—useful for interpolation and controlled generation.

GANs (Generative Adversarial Networks, Goodfellow et al., 2014):

A generator G creates fake samples; a discriminator D tries to distinguish real from fake. Trained adversarially.

Generator loss: fool the discriminator. Discriminator loss: correctly classify real vs. fake.

Training challenges: Mode collapse (generator learns to produce one high-quality example), training instability (oscillating between generator and discriminator domination). Many stabilisation techniques: Wasserstein GAN (WGAN), spectral normalisation, progressive growing.

Diffusion models (covered in the architecture section) have largely superseded GANs for image generation due to more stable training and better sample diversity.

Chapter 71

Part VI: Reinforcement Learning Algorithms

71.1 The RL Framework

An agent interacts with an environment. At each step:

- Observe state s_t
- Take action $a_t \sim \pi(\cdot|s_t)$ according to policy π
- Receive reward r_t
- Transition to state s_{t+1}

Goal: maximise expected cumulative discounted reward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Where $\gamma \in [0,1]$ is the discount factor.

71.2 Value-Based Methods

Value function $V^\pi(s)$: Expected return starting from state s following policy π .

Q-function $Q^\pi(s,a)$: Expected return starting from state s , taking action a , then following policy π .

Bellman equation:

$$Q^{\pi}(s,a) = r + \gamma \sum_{s'} P(s'|s,a) \times \sum_{a'} \pi(a'|s') Q^{\pi}(s',a')$$

Q-Learning (Watkins, 1989): Model-free, off-policy. Update Q-values using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Convergence: Q-learning converges to Q^* (the optimal Q-function) for tabular MDPs under mild conditions.

71.2.1 Deep Q-Networks (DQN, Mnih et al., 2013)

Approximate $Q(s,a)$ with a neural network. Directly applying Q-learning to neural networks is unstable—two innovations made it work:

Experience replay: Store transitions (s, a, r, s') in a replay buffer. Sample randomly to break correlations between consecutive updates.

Target network: Maintain a separate "frozen" target network for computing TD targets. Update it periodically (not at every step). Prevents a moving target problem.

DQN achieved human-level performance on 49 Atari games from raw pixels—the first demonstration of deep RL on a diverse benchmark.

Double DQN: Use the online network to select actions, the target network to evaluate them. Reduces overestimation of Q-values.

Prioritized Experience Replay: Sample transitions with probability proportional to their TD error—prioritise surprising experiences.

Dueling DQN: Decompose $Q(s,a) = V(s) + A(s,a)$ (value + advantage). Shares representation between value and advantage estimates—more efficient learning.

71.3 Policy Gradient Methods

Instead of learning Q-functions and deriving a policy, directly optimise the policy parameterised by θ .

Policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\{\tau \sim \pi_{\theta}\}} [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$$

The gradient of expected return is the expected sum of log-policy gradients weighted by returns. Estimating this by sampling trajectories (REINFORCE) has high variance.

Baseline subtraction: Replace G_t with $(G_t - b)$ for any baseline b that doesn't depend on the action. The gradient is unchanged in expectation but variance is reduced. Using the value function $V(s_t)$ as a baseline gives the **advantage** $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$.

71.3.1 Actor-Critic Methods

Maintain both a policy (actor) and a value function (critic). The critic provides low-variance value estimates; the actor updates the policy using them.

A3C (Asynchronous Advantage Actor-Critic): Multiple parallel workers each maintain a copy of the model and interact with their own environment instance, asynchronously updating shared parameters.

PPO (Proximal Policy Optimisation, Schulman et al., 2017): The dominant RL algorithm for practical applications. Clips the policy update to prevent too-large changes:

$$L(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

Where $r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the probability ratio.

PPO achieves near-TRPO (Trust Region Policy Optimisation) performance with simpler implementation and without computing second derivatives. Used in: OpenAI Five (Dota 2), robotics, ChatGPT's RLHF training.

71.3.2 SAC (Soft Actor-Critic)

Off-policy actor-critic with entropy regularisation—maximise reward plus entropy of the policy. Encourages exploration while maintaining stability. State-of-the-art for continuous control (robotics manipulation, locomotion).

71.4 Model-Based RL

Model-free RL: Learn from raw interaction. Sample-efficient in parameter updates but data-inefficient (needs many environment interactions).

Model-based RL: Learn a model of the environment (transition dynamics, reward function). Use the model for planning or to generate synthetic experience.

Dyna (Sutton, 1990): Learn a model while interacting; use the model to generate synthetic transitions for additional Q-learning updates. Dramatically improves sample efficiency.

World models (Ha and Schmidhuber, 2018): Learn a compressed world model (RNN + VAE). Plan entirely in the latent space of the world model.

MuZero (Schrittwieser et al., 2019): AlphaGo's successor. Learns an implicit model—not the true environment dynamics but a representation useful for planning with MCTS. Achieves superhuman performance in chess, shogi, Go, and Atari from pixels, without knowing the rules.

71.5 RLHF—Reinforcement Learning from Human Feedback

The problem: We want AI systems to follow human intentions, but it's difficult to specify a reward function for complex tasks like "write a helpful, harmless, honest response."

RLHF (Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022):

1. **Supervised fine-tuning (SFT):** Fine-tune a pre-trained LLM on demonstration data.

2. **Reward model training:** Collect human preferences (which response is better?) and train a reward model to predict preferences.
3. **RL fine-tuning:** Use PPO to optimise the LLM against the reward model, with a KL penalty toward the SFT model (preventing the model from drifting too far).

Used in: ChatGPT, GPT-4, Claude, Gemini. RLHF is the technique that turned raw language model pre-training into conversational AI.

Direct Preference Optimisation (DPO): A simpler alternative to RLHF that bypasses the explicit reward model. Directly fine-tunes the LLM to maximize a re-expressed preference objective. Often comparable to or better than RLHF with less complexity.

71.6 AlphaGo / AlphaZero—Deep RL at the Frontier

AlphaGo (Silver et al., 2016): First superhuman Go program.

Architecture:

1. **Policy network:** CNN predicting the probability distribution over moves
2. **Value network:** CNN predicting win probability from a position
3. **Monte Carlo Tree Search (MCTS):** Uses policy and value networks to guide tree search

Training:

1. Supervised learning from human expert games (initialise policy network)
2. Self-play RL: play against previous versions of itself, update policy by policy gradient
3. Train value network on outcomes of self-play games

AlphaZero (Silver et al., 2017): Generalisation that learns from scratch (no human games) for Go, chess, and shogi. Achieves superhuman performance in all three. The policy and value networks are trained purely from self-play using MCTS to generate training targets.

MuZero (2019): Removes the requirement for known game rules—learns an implicit model. Extends to Atari.

The AlphaGo family represents the state of the art in combining deep learning with classical planning—using neural networks to guide exponential search spaces that would otherwise be intractable.

Chapter 72

Part VII: Large Language Models— The Current Frontier

72.1 The Pre-Training Paradigm

Modern LLMs are trained in two phases:

Pre-training: Train a large transformer on a massive corpus (hundreds of billions to trillions of tokens) with next-token prediction. The model learns grammar, facts, reasoning patterns, code, mathematics—everything in the data.

Post-training: Fine-tune for specific behaviours: instruction following (SFT), human preference alignment (RLHF/DPO), safety (Constitutional AI, rejection sampling).

The pre-training phase dominates compute—GPT-4 reportedly required ~\$100M in compute. Post-training is orders of magnitude cheaper.

72.2 Tokenization

Text must be converted to discrete tokens (integers) before processing.

Byte Pair Encoding (BPE): Start with individual characters. Iteratively merge the most frequent adjacent pair. Continue until vocabulary size is reached. Common words become single tokens; rare words are split into subword pieces.

GPT-4 uses ~100,000 BPE tokens. LLaMA 3 uses 128,000. Larger vocabularies reduce average sequence length (fewer tokens per character) and improve efficiency.

72.3 Context Length and Attention Efficiency

Modern LLMs support increasingly long contexts: GPT-4 (128K tokens), Gemini 1.5 (1M+ tokens), Claude 3 (200K tokens).

Challenges:

- $O(n^2)$ attention memory and compute
- Position generalisation (models trained on short contexts may not generalise to long ones)
- The "lost in the middle" problem—models recall information near the beginning and end of long contexts but miss information in the middle

Solutions: Flash Attention (IO-efficient exact attention), sliding window attention (Mistral), RoPE with extended context fine-tuning, YaRN (context length extrapolation).

72.4 Inference—Making Generation Fast

Training uses teacher forcing (feed ground truth tokens as input). Inference is autoregressive: generate one token at a time, feeding each generated token back as input.

Key-Value (KV) Cache: At each attention layer, the K and V matrices for previously generated tokens don't change. Cache them and only compute new rows for the latest token. Reduces inference from $O(n^2)$ to $O(n)$ per new token.

Speculative decoding: Use a small "draft" model to generate k candidate tokens, then verify them in parallel with the large "verifier" model. If the large model agrees, accept k tokens in one pass. Achieves $2\text{--}3\times$ speedup for the largest models.

Quantisation: Reduce weight precision from 32-bit float to 16-bit, 8-bit, or 4-bit. 4-bit quantisation (GPTQ, AWQ, bitsandbytes) reduces memory footprint by $8\times$ with minimal quality loss. Makes 70B-parameter models runnable on consumer GPUs.

Mixture of Experts (MoE): Only activate a subset of the model's parameters for each token. Mistral MoE, GPT-4 (reportedly), and Gemini use MoE to achieve high capacity with lower inference cost.

72.5 Emergent Capabilities and Chain-of-Thought

Large models exhibit behaviours not present at smaller scales:

In-context learning (few-shot prompting): A model learns a new task from examples in the prompt without any parameter updates. Simply showing 5 examples of "input $\rightarrow$ output" teaches the task.

Chain-of-thought reasoning (Wei et al., 2022): Prompting the model to "think step by step" dramatically improves performance on arithmetic, commonsense, and symbolic reasoning tasks. Breaking reasoning into explicit steps allows the model to use more computation for complex problems.

Tool use and function calling: Models trained to use external tools (calculators, search engines, code interpreters) greatly extend their capabilities beyond what can be represented in weights alone.

Chapter 73

Part VIII: The Theory Behind Practice

73.1 Why Does Any of This Work?

Deep learning’s empirical success is far ahead of its theoretical understanding. Here are the best current explanations:

The lottery ticket hypothesis (Frankle and Carlin, 2018): Large neural networks contain smaller “winning ticket” sub-networks that could be trained from scratch to the same performance. Large networks succeed partly because they contain many sub-networks, increasing the probability that one of them is “good.”

Neural tangent kernel (NTK) theory: In the infinite-width limit, gradient descent on a neural network is equivalent to kernel regression with the Neural Tangent Kernel. This provides a theoretical handle on convergence but applies mainly to overparameterised lazy-training regimes, not the feature learning that makes deep networks powerful.

Feature learning vs. lazy learning: Neural networks in the NTK regime (very large learning rates or specific architectures) don’t learn meaningful features—they behave like kernel methods. Feature learning (the regime where the network’s representations change meaningfully during training) is better in practice but harder to analyse.

Grokking: Models trained on small datasets sometimes suddenly “gener-

alise” after a long period of overfitting, during which the loss on training data has been constant at zero. The model is still changing internally even when the training loss is flat—learning a more compressed, generalising representation. Suggests that generalisation requires more than just fitting the training data.

Mechanistic interpretability: Reverse-engineer the algorithms implemented by trained neural networks. Discovered that some circuits in language models implement recognisable algorithms—induction heads for in-context learning, lookup circuits for factual recall, attention heads that track syntactic dependencies. This is an early but growing science.

73.2 What Comes Next

Part 9 has traced the full arc of machine learning as an algorithmic system: from the formal learning problem through optimisation, architectures, regularisation, self-supervised learning, reinforcement learning, and large language models.

Part 10: Problem Solving and Pattern Recognition brings the series to its capstone—the meta-skill of algorithmic thinking. How do you look at a new problem and identify which tools apply? What patterns recur across problems? How do you simplify, reduce, and find the structural insight that makes a problem tractable? After nine parts of techniques and theory, the final part is about practice.

Next: Problem Solving and Pattern Recognition—The Art and Science of Algorithmic Thinking

Chapter 74

Algorithms: Problem Solving and Pattern Recognition

74.0.1 The Art and Science of Algorithmic Thinking

This is Part 10 of the Algorithms series—the capstone. Parts 1–9 built the complete toolkit: history, algorithm categories, data structures, complexity theory, advanced design, analysis, systems engineering, domain deep dives, and machine learning. This final part is about how to use all of it.

74.1 What This Part Is About

Nine parts of techniques, theory, and domain knowledge. The question this final part answers is: **so what do you do when you see a new problem?**

Knowing merge sort doesn't help you if you don't recognise when a problem reduces to sorting. Knowing dynamic programming doesn't help if you don't see that a problem has optimal substructure. The entire toolkit is only as useful as your ability to look at something unfamiliar and find the handle—the insight, the reduction, the structure that makes a hard problem tractable.

This is not a skill that comes from reading. It comes from practice, pattern matching, and—most importantly—a disciplined way of thinking about what a problem actually is before trying to solve it. This document makes that discipline explicit.

It has three sections. The first is a taxonomy of the patterns that recur across problems—the recognisable shapes that, once seen, point you toward the right tool. The second is a problem-solving framework—a sequence of questions to ask when facing an unknown problem. The third is a collection of canonical reductions and transformations—the bridges between problem domains that let you apply hard-won knowledge in new places.

Chapter 75

Part I: The Pattern Taxonomy

75.1 Pattern 1: The Optimisation Problem

Signature: "Find the best ____" or "maximise/minimise ____."

Most problems in practice are optimisation problems in disguise. The first question: **what is the structure of the solution space?**

Continuous and convex: If the objective is convex and constraints are linear or convex, use gradient descent, LP, or convex programming. Global optimum is guaranteed. The hard part is often recognising convexity.

Discrete with optimal substructure: If the best solution to the whole problem contains best solutions to subproblems—use dynamic programming. The key question: what is the natural decomposition into subproblems? What is the state?

Discrete with greedy choice property: If a locally optimal choice leads to a globally optimal solution—use a greedy algorithm. The key question: what exchange argument proves greedy is optimal? Can you always improve a non-greedy solution by making it more greedy?

NP-hard: If the problem is NP-complete, you have four options (from Part 4): approximation, parameterised algorithms (if a natural parameter is small), exact exponential (if n is small), or heuristics with no guarantee. Choosing among these depends on the application's tolerance for approximation error and the typical size of n .

Non-convex with structure: Many ML optimisation problems are non-convex but have enough structure (overparameterisation, gradient flow toward flat minima) that local search (gradient descent) finds good solutions in practice. Knowing when to trust local search is a judgment call.

75.2 Pattern 2: The Counting Problem

Signature: "How many ___ are there?" or "what is the probability of ___?"

Counting problems are often harder than decision problems (Part 4: #P vs. NP). But structure frequently makes them tractable.

Combinatorial identities: Does the count simplify via inclusion-exclusion, binomial coefficients, stars-and-bars, the principle of double counting? Often a direct formula exists.

DP counting: If the count decomposes across subproblems—count the number of paths, sequences, or configurations satisfying a constraint—DP computes it efficiently.

Generating functions: Encode a counting problem as the coefficients of a polynomial or power series. Multiplication = convolution = combining independent choices. The FFT computes polynomial multiplication in $O(n \log n)$, enabling fast counting over many classes of combinatorial objects.

Approximate counting: When exact counting is #P-hard, randomised approximate counting algorithms (FPRAS—Fully Polynomial Randomised Approximation Scheme) may exist. Canonical example: approximately counting satisfying assignments to a DNF formula in polynomial time via sampling.

75.3 Pattern 3: The Search Problem

Signature: "Does there exist a ___ satisfying ___?" or "find an example of ___."

Search problems reduce to decision problems (if decision is easy, binary search over the "amount" to find the optimum). The important cases:

Structured search: Search in a space with order, hierarchy, or metric structure. Binary search (ordered), BFS/DFS (graphs), branch-and-bound (constrained), A* (metric spaces with heuristics).

Constraint satisfaction (CSP): Assign values to variables satisfying constraints. Backtracking + constraint propagation (arc consistency). The key optimisation: choose the most constrained variable next (minimum remaining values heuristic) and propagate constraints aggressively (forward checking, AC-3).

Satisfiability: If the problem encodes as SAT or Max-SAT, industrial SAT solvers (CDCL with clause learning) handle millions of variables in practice. Many verification, planning, and scheduling problems reduce naturally to SAT.

Search in a tree of possibilities: Branch-and-bound explores the space systematically, pruning branches that cannot improve on the current best. LP relaxation at each node provides an upper bound for pruning in integer programming. This is how commercial MIP (mixed integer programming) solvers work.

75.4 Pattern 4: The Graph Problem

Signature: Entities with relationships. Almost everything is a graph if you squint.

The first move: **draw the graph**. Nodes are what? Edges are what? Directed or undirected? Weighted or unweighted?

Reachability and connectivity: BFS/DFS. Is there a path? Are all nodes connected? Find connected components.

Shortest paths: Is there a single source? Use Dijkstra (non-negative weights) or Bellman-Ford (negative weights). All-pairs? Floyd-Warshall. Special structure (DAG)? DP on topological order.

Cycles and ordering: Does the graph have a cycle? DFS can detect cycles. Is it a DAG? Topological sort gives a valid ordering of dependencies.

Flow and matching: Is the problem about routing, assignment, or matching? Model it as a flow network. Most assignment and matching problems reduce to max flow or bipartite matching.

Tree problems: Are you working on a tree (or can you reduce to a tree: spanning tree, BFS tree, tree decomposition)? Trees are algorithmically much easier than general graphs. Most graph problems that are NP-hard generally become polynomial on trees.

Spectral properties: Eigenvalues of the graph Laplacian encode connectivity, expansion, clustering, and random walk behaviour. Spectral graph theory underpins graph neural networks, graph partitioning, and manifold learning.

75.5 Pattern 5: The String or Sequence Problem

Signature: Ordered sequences of characters, tokens, events, or values.

Pattern matching: Does a pattern appear in a text? Use KMP ($O(n+m)$), Boyer-Moore (fast in practice), or Rabin-Karp (rolling hash, multiple patterns). For approximate matching: dynamic programming for edit distance.

Sequence alignment and similarity: Two sequences; find their relationship. LCS (DP, $O(mn)$), edit distance (DP, $O(mn)$), sequence alignment (Needleman-Wunsch, Smith-Waterman). For genome scale: BWT index.

Parsing and structure: Does the sequence have hierarchical structure? Context-free grammars capture most structured sequence languages. CYK/Earley parsing in $O(n^3)$.

Subsequence problems: Longest increasing subsequence (LIS): $O(n \log n)$ using patience sorting / binary search. Longest common subsequence: $O(mn)$ DP. Many variants reduce to LCS or LIS.

Suffix structures: For multiple queries on the same text—build a suffix array ($O(n \log n)$) or suffix tree ($O(n)$). $O(m)$ per pattern query thereafter.

75.6 Pattern 6: The Interval or Geometric Problem

Signature: Objects in space—points, line segments, intervals, rectangles, polygons.

Sorting by endpoint: Most interval problems become tractable after sorting. Interval scheduling: sort by end time, greedy. Interval covering: sort by start time. Meeting rooms: sort by both start and end times.

Sweep line: Move a vertical or horizontal line across the space. Maintain an active data structure as events occur (objects enter/exit the sweep line). Enables $O(n \log n)$ algorithms for intersection detection, closest pair, convex hull, and many other problems.

Divide and conquer in geometry: Closest pair of points: divide by x -coordinate, recurse, check the strip. $O(n \log n)$. Many geometric algorithms use this paradigm.

Coordinate compression: Geometric problems often have large coordinate ranges but few distinct values. Compress coordinates to $1, \dots, n$, then use data structures designed for small integer ranges.

Voronoi and Delaunay: Spatial partitioning and triangulation. For nearest-neighbour, point location, and triangulation problems, these structures are the right tool ($O(n \log n)$ construction, $O(\log n)$ queries).

75.7 Pattern 7: The Resource Allocation Problem

Signature: Limited resources, competing demands, maximise utilisation or minimise cost.

Scheduling: Assign jobs to machines or time slots subject to constraints. Single-machine scheduling problems have elegant greedy algorithms. Multi-machine scheduling is often NP-hard but has good approximations (list scheduling, LPT rule).

Knapsack-type problems: A capacity constraint; items have weight and value; maximise value subject to weight. 0/1 knapsack: DP in $O(nW)$. Fractional knapsack: greedy. Multiple knapsacks: harder.

Bin packing: Pack items of varying sizes into bins of fixed capacity. NP-hard; greedy approximations (first fit decreasing: $11/9 \text{ OPT} + 6/9$). Bin packing and knapsack are closely related.

Flow-based allocation: If allocation has network structure (routes, capacities), model as flow. If items must be assigned to agents optimally, model as bipartite matching or assignment problem.

LP formulation: Most resource allocation problems can be formulated as LP or ILP. LP gives a lower bound and fractional solution; rounding + LP duality give approximation algorithms.

75.8 Pattern 8: The Recursive or Fractal Problem

Signature: A problem that contains smaller instances of itself.

Divide and conquer: Can you split into independent subproblems, solve them, and combine efficiently? Master theorem predicts the complexity. Merge sort, FFT, closest pair, Karatsuba.

Dynamic programming: Can you split into overlapping subproblems? Memoize to avoid recomputation. Fibonacci, LCS, matrix chain, optimal BST, interval DP.

Inductive structure: Is there a clean base case and a way to extend a solution for $n-1$ to n ? Insertion sort, incremental algorithms, online algorithms.

Self-similar structure: Does the problem on a data structure (tree, graph) reduce to the same problem on subtrees/subgraphs? Tree DP: solve the problem on leaves first, then combine up the tree. This handles most tree optimisation problems.

75.9 Pattern 9: The Probability or Randomness Problem

Signature: Uncertainty, distributions, expected values, "what happens on average."

Expected value by linearity: Break complex expected values into sums of indicator random variables. Quicksort analysis, hashing analysis, birthday paradox. Indicator variables + linearity of expectation handle surprisingly complex expectations.

Tail bounds: Chernoff, Chebyshev, Markov for concentration of measure. Used to convert expected-value results into "with high probability" results.

Probabilistic existence proofs (probabilistic method): Show that a random object has the desired property with positive probability—therefore the object exists. Often gives non-constructive proofs that are then made constructive by derandomisation.

Markov chains and mixing: Random walks on graphs. Does the random walk mix quickly (reach stationarity fast)? Spectral gap governs mixing time. Used in MCMC, randomised algorithms, network analysis.

Streaming and sketching: Data arrives online; space is limited. Approximate answers using hashing and random projections. Count-Min Sketch, HyperLogLog, JL Lemma (random projections approximately preserve distances).

75.10 Pattern 10: The Data Structure Selection Problem

Signature: Recurring operations on a collection of data—what structure supports them efficiently?

This isn't a problem type so much as a meta-problem that appears inside almost every other problem. The questions:

- What operations are needed? (Insert, delete, search, range query, min/max, rank, predecessor, successor, merge, split)
- Are operations static (data fixed) or dynamic (data changes)?
- Is ordered access needed, or only membership/lookup?
- Does the data fit in memory, or must it reside on disk?

- Is approximate access acceptable?

From the answers, the data structure follows (Part 3 gives the complete map). The key insight: **data structure choice determines algorithm shape**. Choosing the wrong structure forces awkward algorithms; choosing the right one makes the algorithm obvious.

Chapter 76

Part II: The Problem-Solving Framework

76.1 Step 1: Understand the Problem—Really

Before writing a single line of code or pseudocode, understand the problem deeply.

Restate it in your own words. If you can't explain the problem to someone unfamiliar with it, you don't understand it.

Clarify the constraints. Input size? Data types? Exact or approximate answer? Online (data arrives in sequence) or offline (all data available)? Static or dynamic data? What are the time and space constraints?

Work through small examples by hand. For $n=3$ or $n=4$, what is the answer? This builds intuition and often reveals the structure.

Identify the degrees of freedom. What are you choosing? What is fixed? What is the space of possible solutions?

Ask: what makes this hard? What would the brute-force solution look like, and why is it too slow? The answer to this question usually points toward the right tool.

76.2 Step 2: Classify the Problem

Apply the pattern taxonomy from Part I. Ask:

- Is this an optimisation, decision, or counting problem?
- Does it involve graphs, strings, geometry, sequences, or intervals?
- Is there a natural decomposition into subproblems?
- Is there a greedy structure?

Often a problem falls into multiple categories—this is a signal, not confusion. A shortest path is both a graph problem and an optimisation problem; the intersection of these two categories points directly at Dijkstra.

Look for the simplest version. Remove constraints one at a time. If the graph were a tree, how would you solve it? If all weights were equal? If there were only two items? The simplest version often has an obvious solution; adding constraints back reveals the algorithm.

76.3 Step 3: Reduce to a Known Problem

The most powerful move: show that your problem is an instance of one you already know how to solve.

Canonical reductions (the most useful ones):

Your problem looks like...	Reduce to...
Assignment of items to agents (min cost)	Bipartite matching / min-cost flow
Choosing a subset satisfying constraints	Integer LP / ILP
Ordering with dependencies	Topological sort
Routing with capacity limits	Max flow
Covering all elements with fewest sets	Set cover (greedy, $O(\log n)$ approximation)
Finding the optimal split point	Binary search on the answer
Combining independent choices	Convolution / generating functions
Counting paths in a DAG	DP on topological order
Clustering by similarity	k-means / spectral clustering
Searching in high-dimensional space	ANN search / dimensionality reduction
Boolean constraint satisfaction	SAT / SMT

When you successfully reduce to a known problem, you inherit all the theory: the algorithm, the lower bounds, the approximability results, and the practical implementations.

76.4 Step 4: Design the Algorithm

If reduction doesn't work directly, design. The design paradigms (Part 2) as a checklist:

Brute force first. What is the obviously correct but slow solution? This is your baseline and sometimes reveals structure.

Try divide and conquer. Is there a natural split? What is the cost of combining? Apply the master theorem to check if the resulting recurrence is fast enough.

Try dynamic programming. Define the subproblem precisely. Write the recurrence. Check that subproblems overlap (otherwise, divide and conquer is better). What is the table size? What order do you fill it in?

Try greedy. What is the locally optimal choice? Can you prove by exchange argument that it leads to the global optimum? If not, can you prove a bound on how far it can be from optimal (approximation ratio)?

Try graph reduction. Model entities as nodes, relationships as edges, and see if a standard graph algorithm applies.

Try randomisation. Is there a random process that, in expectation or with high probability, produces the right answer efficiently?

Try relaxation. Relax the problem (drop constraints, allow fractional solutions, allow approximate answers). Is the relaxed problem easy? How much do you lose by rounding back?

76.5 Step 5: Prove Correctness

An algorithm without a correctness argument is a guess. The standard tools:

Loop invariant: For iterative algorithms, identify a property that is true before the loop begins and maintained after each iteration. If it implies the correct answer when the loop terminates, the algorithm is correct.

Induction: For recursive algorithms, assume the algorithm is correct for smaller inputs; prove it correct for the current input using the assumption.

Exchange argument (for greedy): Take any optimal solution. Show that you can transform it into the greedy solution step by step without decreasing its quality. Therefore the greedy solution is at least as good as any optimal solution.

Invariant on the data structure: Many data structure algorithms maintain a representation invariant (the BST property, the heap property). Prove that each operation preserves the invariant.

Adversarial argument (for lower bounds): Show that an adversary can force any algorithm to make at least $\Omega(f(n))$ operations. Therefore no algorithm can do better.

76.6 Step 6: Analyse Complexity

Time complexity: Count the dominant operations. Apply the master theorem if recursive. Use amortised analysis if some operations are occasionally expensive.

Space complexity: Does the algorithm require $O(n)$ auxiliary space? $O(\log n)$ for recursion stack? $O(1)$ for in-place algorithms?

Is this tight? Does the complexity match a known lower bound? If not, is there a better algorithm or a better lower bound?

Practical performance: Beyond big- O , consider cache behaviour, constant factors, and how well the algorithm parallelises. An $O(n \log n)$ algorithm with poor cache locality may be slower in practice than an $O(n^2)$ algorithm with excellent cache performance for small n .

76.7 Step 7: Implement and Test

Implementation considerations:

- What is the simplest correct implementation?
- Are there edge cases (empty input, single element, all elements equal, adversarial input)?
- Does the implementation have integer overflow, off-by-one errors, or incorrect index bounds?
- Is the implementation correct for both the base case and the inductive case?

Testing strategy:

- Test on the examples you worked by hand in Step 1
- Test on edge cases
- Test on large random inputs (check output is plausible, check against brute force on small inputs)
- For competitive programming: test on the maximum input size to verify time bounds

Debugging by invariant: If the output is wrong, find which invariant is violated and where. This is almost always faster than print-debugging.

Chapter 77

Part III: Canonical Reductions and Transformations

77.1 The Reduction Library

A library of known reductions is the most valuable asset an algorithm designer accumulates over time. Here are the most powerful ones.

77.1.1 Binary Search on the Answer

Template: The problem asks "find the minimum X such that some condition holds." If the condition is monotone (once X satisfies it, all larger X do too), binary search over X .

Examples:

- "Find the minimum time to complete all jobs on k machines." Binary search on time T ; for each T , check feasibility (can we schedule all jobs in T time on k machines?). The feasibility check is usually simpler than the original problem.
- "Find the k th smallest element in a sorted matrix." Binary search on the value; count elements $\leq v$.
- "Find the maximum minimum distance between k points." Binary search on the distance; check if k points can be placed with pairwise distance $\geq d$ (greedy).

When to use: When you can check a proposed answer more easily than finding it directly. Converts search problems into decision problems.

77.1.2 Reduce to Shortest Path

Template: If you can model the problem as finding an optimal path through a state space, use Dijkstra or Bellman-Ford.

State space graph construction:

- Nodes: states of the problem (a configuration, a position, a (row, col, fuel) tuple)
- Edges: legal transitions between states
- Edge weights: cost of each transition
- Source: initial state
- Target: goal state or any state satisfying the goal condition

Examples:

- Word ladder (transform one word to another by changing one letter at a time, minimum steps): nodes = words, edges = words differing by one letter, unit weight.
- Minimum-cost navigation with obstacles and terrain: nodes = grid cells, edges = moves, weights = terrain cost.
- Shortest transformation sequence with constraints: define the state to include the constraint (e.g., "arrived here with k fuel remaining").

Key insight: When the state space is large but has structure, model it implicitly—generate neighbours on the fly rather than materialising the full graph.

77.1.3 Reduce to Max Flow

Template: Problems involving routing, matching, covering, and assignment often reduce to max flow.

Bipartite matching: Add source $\rightarrow$ left nodes (cap 1), right nodes $\rightarrow$ sink (cap 1), original edges (cap 1). Max flow = max matching.

Project selection / closure problem: Positive-profit projects go from source with capacity = profit. Negative-cost projects go to sink with capacity = $|\text{cost}|$. Dependencies get infinite-capacity edges. Min cut gives max profit subset.

Vertex connectivity: How many vertices must be removed to disconnect s from t ? Split each vertex v into v_{in} and v_{out} with edge $(v_{\text{in}}, v_{\text{out}}, \text{capacity } 1)$. Max flow = min vertex cut.

Baseball elimination: Can team i still win the division? Construct a flow network from remaining games between teams and capacity constraints. If max flow saturates all game edges, team i can win; otherwise, it is eliminated.

Nurse scheduling, task assignment, resource allocation—almost all assignment problems where items must be assigned to agents with capacity constraints reduce to flow.

77.1.4 Reduce to Dynamic Programming

The DP question: "What is the minimum information I need to know about the past to make an optimal decision about the future?"

The answer defines the **DP state**. Once you have the state, the recurrence usually follows.

State design patterns:

- **Suffix/prefix:** $\text{dp}[i]$ = answer for the suffix/prefix ending at i . (LIS, LCS, edit distance)
- **Interval:** $\text{dp}[i][j]$ = answer for the subarray from i to j . (Matrix chain, optimal BST, palindrome partitioning)
- **Subset:** $\text{dp}[S]$ = answer for the subset S . (TSP bitmask DP, set cover DP)

- **Tuple of decisions:** $dp[i][j][k]$ = answer after making i, j, k specific choices. (DP with multiple dimensions)
- **On a graph/tree:** $dp[v][...]$ = answer for the subtree rooted at v . (Tree DP)

The overlapping subproblem check: If the recursion tree has the same subproblem computed multiple times, DP helps. If each subproblem appears only once, divide and conquer is better.

Bottom-up vs. top-down: Bottom-up fills the table in a fixed order; top-down uses recursion + memoization. Bottom-up avoids recursion overhead and is often faster in practice. Top-down only computes needed subproblems (useful when the state space is sparse).

77.1.5 Reduce to Minimum Spanning Tree

Template: Problems about connecting components at minimum cost often reduce to MST.

Minimum cost network: Build a graph where edge weights represent the cost of connecting components. MST gives the minimum cost connection.

Cluster by similarity: Build a complete graph where edge weights are dissimilarity. The MST connects the most similar items first—equivalent to single-linkage hierarchical clustering.

Bottleneck MST: Find the spanning tree minimising the maximum edge weight. Any MST minimises the bottleneck—the bottleneck MST is a standard MST.

Point cluster selection: Given n points, find the subset of k points maximising the minimum pairwise distance. Build a complete graph, sort edges by weight, add edges in order until the graph has $< k$ connected components—the $(k-1)$ th MST edge gives the answer.

77.1.6 Reduce to Sorting

Template: Problems that look quadratic often become linear after sorting.

Two-sum: Given an array, are there two elements summing to target? Brute force $O(n^2)$. Sort + two pointers: $O(n \log n)$. Hash table: $O(n)$.

Closest pair: Sort by x-coordinate, use divide and conquer. $O(n \log n)$ instead of $O(n^2)$.

Inversion count: How many pairs (i, j) with $i < j$ but $a[i] > a[j]$? Use merge sort—count inversions during the merge step. $O(n \log n)$.

Activity selection, interval scheduling: Sort by end time, greedy in $O(n \log n)$.

Majority element: Sort; the majority element must occupy the middle position. $O(n \log n)$ (or $O(n)$ by Boyer-Moore voting).

The pattern: Sorting imposes an order that reduces a 2D comparison problem (compare every pair) to a 1D sweep (scan in order). This changes the structure from quadratic to linear.

77.1.7 Reduce to Convex Hull

Template: Geometric optimisation problems often reduce to finding the convex hull of a point set.

Maximum dot product: Given points P and a query vector d , find $p \in P$ maximising $d \cdot p$. This is the point on the convex hull in direction d . After building the convex hull, $O(\log n)$ per query.

Half-plane intersection: Intersection of n half-planes is the dual of a convex hull problem.

Minimum enclosing circle / ellipse: Related to convex hull—smallest enclosing circle passes through at most 3 points on the convex hull.

The dual transform (point-line duality): A point (a, b) maps to the line $y = ax - b$; a line $y = mx + c$ maps to the point $(m, -c)$. Problems about lines in one space become problems about points in the dual space. Many hard-looking line arrangement problems become convex hull problems after dualising.

77.1.8 Reduce to String Matching

Template: Problems about detecting patterns, motifs, or recurring structures in sequences reduce to string matching.

Circular string containment: Is string A a rotation of string B? Concatenate B with itself; search for A in BB. $O(n)$ using KMP.

Longest repeated substring: Suffix array + LCP array. $O(n \log n)$ construction; longest repeated substring is the maximum LCP value.

Pattern with wildcards: KMP generalisation or bit-parallel algorithms.

Sequence alignment in bioinformatics: All reduce to string DP variants (edit distance, LCS, Needleman-Wunsch).

77.1.9 Reduce to Matrix Operations

Template: Problems with linear recurrences or state transitions reduce to matrix exponentiation.

Linear recurrence in $O(\log n)$: Fibonacci in $O(\log n)$: represent the recurrence as a matrix equation.

$$\begin{bmatrix} F(n+1) \\ F(n) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \times \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}$$

Matrix exponentiation by repeated squaring computes M^n in $O(d^3 \log n)$ for a $d \times d$ matrix.

State transition systems: Any system with a finite number of states and linear transition rules can be solved in $O(\text{states}^3 \log n)$ for n steps.

Counting paths of length k : The (i,j) entry of A^k gives the number of paths of length k from i to j in a graph with adjacency matrix A .

Gaussian elimination: Solving systems of linear equations. Used in network flow (LP), scientific computing, cryptography (linear algebra over finite fields in coding theory).

Chapter 78

Part IV: The Deeper Patterns

78.1 Duality—The Same Problem, Two Views

Many of the deepest results in algorithms are duality results: the same problem has two equivalent formulations that illuminate different aspects.

Max-flow min-cut: The maximum flow equals the minimum cut. The flow formulation finds the solution; the cut formulation proves optimality.

LP duality: Every LP has a dual LP with the same optimal value. Primal gives the solution; dual gives the certificate of optimality.

Minimax theorems: In two-player zero-sum games, the maximum guaranteed payoff for the minimising player equals the minimum guaranteed payoff for the maximising player (von Neumann's minimax theorem). This underlies game theory and competitive analysis.

Boolean duality: De Morgan's laws, duality between CNF and DNF, between SAT and UNSAT. Problems that are hard in one form may be easy in the other.

Geometric duality (point-line): Problems about points become problems about lines and vice versa. This transforms hard geometric intersection problems into easier convex hull problems.

The lesson: When a problem is hard, look for its dual. The dual may be easier to solve or may provide a proof of optimality for the primal solution.

78.2 Relaxation—The Art of Letting Go

Almost every sophisticated algorithm involves relaxing the problem, solving the relaxed version, and recovering a solution to the original.

LP relaxation of ILP: Drop integrality constraints. The fractional solution gives a lower bound and a starting point for rounding.

Lagrangian relaxation: Move difficult constraints into the objective with a multiplier (Lagrange multiplier). The resulting problem is easier; the multiplier's value is tuned to respect the constraint in expectation.

Continuous relaxation of combinatorial problems: Replace discrete choices with probabilities. Solve the continuous optimisation. Round to get a combinatorial solution.

Approximate membership (Bloom filter): Relax exact membership testing to allow false positives. Gain enormous space efficiency.

Approximate nearest neighbour: Relax exact nearest neighbour to $(1+\epsilon)$ -approximate nearest neighbour. Gain exponential speedup in high dimensions.

The pattern: Relaxation introduces error but removes structural complexity. The art is choosing the relaxation that is easy enough to solve and tight enough that rounding doesn't lose too much.

78.3 Symmetry and Invariants

Invariants are the skeleton of algorithms. Every correct algorithm maintains a property (invariant) that implies correctness when the algorithm terminates.

Finding the right invariant is often the creative step. The invariant for Dijkstra: “for every visited node v , $\text{dist}[v]$ is the true shortest distance from source to v .” This invariant is maintained at each step by the priority queue and the greedy selection.

Symmetry reduces problem size. If a problem has symmetry (rotational, reflective, permutation), solutions come in equivalence classes. Enumerate

one representative per class to avoid redundant work.

Conservation laws in flow algorithms: At every non-source, non-sink node, flow in = flow out. This conservation law constrains the solution space and enables the max-flow min-cut theorem.

Monovariance: In combinatorial game theory and termination proofs, a quantity that strictly decreases (or increases) at each step guarantees the process terminates. Dijkstra terminates because the set of settled nodes grows monotonically.

78.4 The Power of Preprocessing

Many problems become easy if the right preprocessing is done upfront.

Sorting as preprocessing: After $O(n \log n)$ sorting, most queries that would take $O(n)$ become $O(\log n)$ (binary search). Interval problems become sweepable. The amortised cost of a query drops from $O(n)$ to $O(\log n)$ if many queries share the same sorted data.

Indexing: Build an index once, query many times. A database index trades write cost for read benefit. A suffix array pays $O(n \log n)$ construction to get $O(m)$ per query.

Transform and process: Apply a mathematical transform (Fourier, Laplace, generating function) to move to a domain where operations are cheaper. Multiplication becomes addition in log space. Convolution becomes point-wise multiplication in Fourier space.

Randomised preprocessing (hashing): Build a hash table once; $O(1)$ lookup forever after. Random projection once; approximate distance queries forever.

78.5 The Role of Randomness

Randomness in algorithm design serves several distinct purposes:

Breaking adversarial cases: Randomised quicksort prevents adversarial sorted input from triggering $O(n^2)$ worst case. Random hashing prevents hash flooding attacks.

Symmetry breaking: In distributed algorithms, randomisation breaks ties that would otherwise cause deadlock or livelock.

Exploration in search: Random restarts in local search, randomised MCTS in AlphaGo, random choices in simulated annealing—randomness helps escape local optima.

Approximation with provable bounds: Randomised rounding of LP solutions, Monte Carlo estimation, random projections for dimensionality reduction—all trade determinism for efficiency with provable accuracy.

Streaming and sketching: Random hash functions enable sublinear space algorithms (HyperLogLog, Count-Min Sketch) for streaming data.

The unifying theme: randomness adds flexibility that deterministic algorithms can't have while maintaining statistical guarantees stronger than no guarantee at all.

Chapter 79

Part V: The Meta-Skills of Algorithmic Thinking

79.1 Simplify First, Generalise Later

Every algorithmic problem should be solved in its simplest version before the general case. Simplifications to try:

- What if the input were random? (Reveals average-case structure)
- What if all values were equal? (Reveals the structure without value variation)
- What if $n = 2$ or $n = 3$? (Reveals the core decision being made)
- What if there were no constraints? (Reveals the objective's structure)
- What if it were a tree instead of a graph? (Reveals tree-tractable algorithms)
- What if the problem were 1D instead of 2D? (Reveals the dimensional challenge)

Often, the simplified version has an obvious solution, and generalising it to the original problem is straightforward once the core structure is understood.

79.2 Know When to Stop

Not every problem has a polynomial-time exact solution, and not every problem requires one.

Stop and approximate: If the problem is NP-hard and the application tolerates approximation, find the best polynomial-time approximation algorithm. The approximation ratio is a feature, not a bug—it quantifies exactly how much you’re giving up.

Stop and use heuristics: If the problem is NP-hard, the instances are large, and approximation is insufficient, industrial solvers (SAT solvers, MIP solvers, constraint programming tools) combine years of engineering with theoretical insights. Use them before building a custom algorithm.

Stop and change the problem: Sometimes the stated problem is not the right problem. Real-world problems are often specified with more constraints than necessary, or the objective is a proxy for what the user actually wants. Questioning the problem formulation can transform an NP-hard problem into a tractable one.

Stop and measure: Sometimes theoretical complexity doesn’t match practical performance. An $O(n^2)$ algorithm may be fast enough for $n \leq 10,000$. An $O(n \log n)$ algorithm with a large constant factor may be slower than a $O(n^2)$ algorithm with a small constant factor for the actual n in the application. Measure before committing to an asymptotically superior algorithm.

79.3 Read the Problem’s Constraints

In competitive programming and system design alike, the constraints tell you the intended solution.

Constraint	Likely intended complexity
$n \leq 10$	$O(n!)$, backtracking, bitmask DP
$n \leq 20$	$O(2^n)$, meet in the middle
$n \leq 100$	$O(n^3)$, Floyd-Warshall, cubic DP
$n \leq 1,000$	$O(n^2)$, quadratic DP, all-pairs
$n \leq 100,000$	$O(n \log n)$, sorting, segment trees, BFS/DFS
$n \leq 1,000,000$	$O(n)$ or $O(n \log n)$, linear algorithms
$n \leq 10^9$	$O(\log n)$ or $O(1)$, math, binary search

When constraints are tight ($n = 10^5$ with a 1-second time limit), this usually means $O(n \log n)$ is intended and $O(n^2)$ will TLE (time limit exceeded). When n is small ($n = 20$), this often means exponential algorithms are intended—don't try to find a polynomial solution.

79.4 The Problem-Solving Disposition

Beyond techniques, the best algorithmic thinkers share a disposition—a way of approaching problems that is distinct from knowing any particular algorithm.

Curiosity about structure. When you see a problem, ask: what makes this interesting? What is the essential difficulty? Is there hidden structure that makes it easier than it looks?

Tolerance for not-knowing. Hard problems require sitting with confusion for a while. The impulse to start coding before fully understanding the problem is the enemy of good algorithm design.

Multiple representations. Hold the problem in multiple forms simultaneously—as a graph, as a matrix, as a string, as a DP. The right representation often makes the solution obvious.

Willingness to restart. If you've been stuck for a while on one approach, abandon it. Fresh eyes often find the structural insight that an entrenched approach misses.

Verify with small examples. Intuitions are wrong. Small examples reveal edge cases and incorrect invariants before you've invested hours in a wrong approach.

Write clean recurrences. For DP, write the recurrence before writing code. Ambiguous recurrences produce buggy code. Clear recurrences almost write themselves.

Trust the theory. When a problem is NP-hard, stop looking for a polynomial algorithm. When a lower bound applies, stop looking for a faster algorithm. The theory tells you where the walls are.

Chapter 80

Coda: The Unity of Algorithmic Thinking

Ten parts, hundreds of algorithms, dozens of data structures, multiple complexity classes, five application domains, and one capstone. What is the thread that runs through all of it?

Everything reduces to everything else. Sorting enables searching. Searching enables optimisation. Optimisation is graph traversal in a state space. Graph traversal is DP on a DAG. DP is memoized recursion. Recursion is induction. Induction is invariant maintenance. Invariant maintenance is the backbone of every algorithm.

Abstraction is the technology. A graph is an abstraction. A hash table is an abstraction. A DP recurrence is an abstraction. The power of algorithms comes not from any specific procedure but from the act of stripping a problem to its essential structure and recognising that structure as an instance of something already understood.

Hardness is as important as ease. Knowing that a problem is NP-hard tells you to stop searching for an exact polynomial algorithm and start designing approximations. Knowing that $\Omega(n \log n)$ comparisons are needed for sorting tells you that merge sort and heapsort are optimal and that further optimisation requires a different model. Hardness results are as actionable as algorithms.

Practice builds intuition. The pattern taxonomy in Part I is not something you use consciously when solving a hard problem. After enough exposure,

you see “assignment problem” and immediately think “bipartite matching,” see “optimise a subarray property” and immediately think “sliding window or DP,” see “find all occurrences in a large text” and immediately think “suffix array.” This intuition is not innate—it is the residue of having seen and solved enough problems that the patterns become reflexive.

The field is alive. Algorithm design is not a closed book. Fine-grained complexity (Part 6) is transforming our understanding of polynomial-time problems. Learning-augmented algorithms (Part 6) are combining ML with classical algorithms in principled ways. Attention mechanisms (Part 9) are architecturally transforming what “algorithm” means. Cache-oblivious design (Part 7) is rewriting what “efficient” means on real hardware. New results arrive constantly, and the tools in this series will be applied to problems that don’t exist yet.

The goal was never to memorise algorithms. The goal was to develop the judgment to recognise what kind of problem you’re facing, the vocabulary to describe its structure precisely, the toolkit to address it systematically, and the theoretical grounding to know when you’ve found the best possible answer.

That is algorithmic thinking. It is one of the most transferable and durable intellectual skills in existence—useful in software, mathematics, science, policy, economics, and anywhere that difficult problems need to be solved with precision.

Series complete. Parts 1–10 cover the full arc from the history of algorithms through to the frontier of machine learning and the meta-skill of algorithmic problem solving.

80.1 Series Index

Table 80.1: Series Index

Part	Title	Core Topics
1	History and Theory	Origins, milestones, significance, algorithm categories overview
2	Algorithm Categories and Implementations	Sorting, graphs, DP, greedy, divide & conquer, backtracking, strings, randomised, crypto, ML
3	Data Structures	Arrays, trees, hash structures, advanced and specialised structures
4	Complexity Theory	P vs NP, NP-completeness, complexity zoo, lower bounds, quantum complexity
5	Advanced Design Techniques	Network flow, linear programming, online algorithms, approximation algorithms
6	Analysis in Depth	Amortised, recurrence, probabilistic, smoothed, competitive analysis, fine-grained complexity
7	Systems and Engineering	Memory hierarchy, parallelism, distributed systems, database internals, compiler algorithms
8	Domain Deep Dives	Bioinformatics, cryptography, computer graphics, search engines
9	Machine Learning in Depth	Optimisation, architectures, regularisation, self-supervised learning, RL, LLMs
10	Problem Solving and Pattern Recognition	Pattern taxonomy, problem-solving framework, canonical reductions, meta-skills